

Web App

giovedì 16 ottobre 2025 14:57

Un'applicazione è un programma che può essere eseguito su computer, cellulari, tablet, ecc.

Le **web app** sono app che girano sul browser (o possono anche essere scaricate per accederci offline) e possono aggiornare l'interfaccia utente (UI) sia caricando una nuova pagina aggiornata o modificando dinamicamente la pagina corrente per visualizzare nuovi dati.

Le web app sono divise in due gruppi logici che dividono un sito in due livelli:

- **Back-end (Lato Server):** è un livello **nascosto all'utente** e gira su un server e si occupa della **logica** dell'applicazione usando linguaggi di programmazione (PHP, Python, C#, Go, ecc) e accedendo al **data access layer** (chiamate API, DBMS, ecc)
- **Front-end (Lato Client):** è il livello di **presentazione** che si interfaccia direttamente con l'utente e serve a **presentare** all'utente i dati raccolti dal back-end. Si usano linguaggi web e markup (HTML, CSS, Javascript) e **richieste asincrone** (AJAX)

La comunicazione dei dati tra il client e il server è effettuata con "**Web Services**" tramite **API** (Application Programming Interface) e una web app può usare molte API da server differenti

Intro a Git

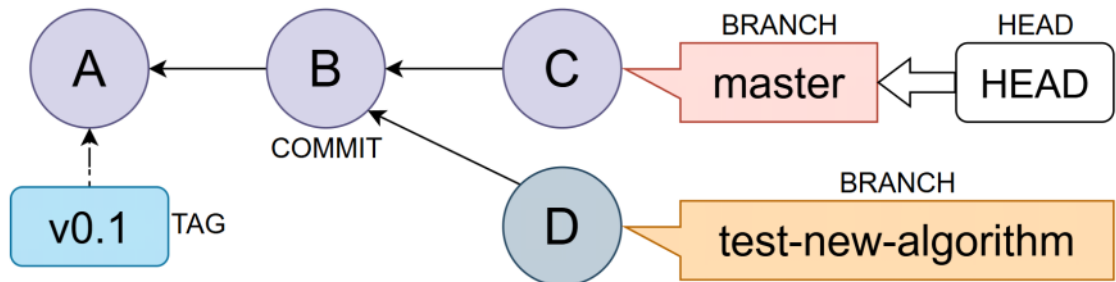
giovedì 9 ottobre 2025 15:57

Git è un sistema di **controllo di versione** di un progetto per poter gestire la cronologia delle varie modifiche che si effettuano sui file del progetto

Termini comuni:

- **Commit**: un'istantanea dei vari file del progetto in un determinato commento, identificata dallo SHA1 del commit stesso.
Necessità di un **commento obbligatorio**
- **Staging**: il commit può contenere un **sottoinsieme delle modifiche** (anche delle modifiche di un solo file) ed è necessario aggiungere i cambiamenti nella **staging area** per decidere quali file vogliamo aggiungere alla commit prima di crearla
- **Branch**: è un **set ordinato** di commit collegati in un DAG (grafo aciclico diretto). Il primo commit crea il primo branch e possiamo creare altri branch per **lavorare in parallelo a più versioni**
- **Head**: posizione corrente nella storia (può essere commit, branch o tag) → working copy. Se non punta ad un branch: detached head → no commit
- **Tag**: etichetta per un commit utile per identificare ad es la versione software
- **Repository o repo**: insieme di commit, branch e tag (assumiamo che 1 progetto = 1 repo)
- **Working copy o working directory**: set di file tracciati nella copia locale e quando cambiamo revisione cambiano i file nella working directory

Es: repo con 2 branch

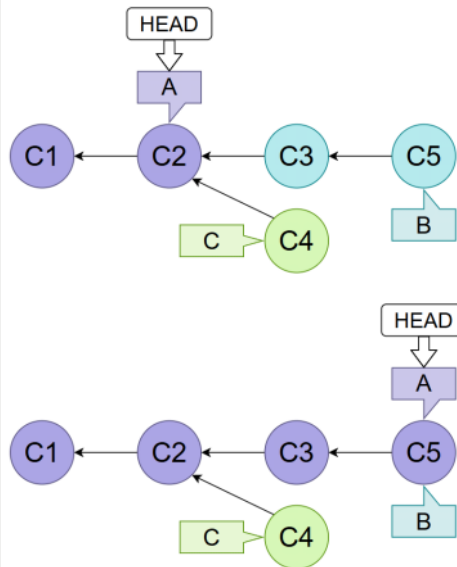


Operazioni comuni:

- **Fork**: crea un nuovo repository che è una **copia di uno esistente**
- **Pull/Merge request**: richiesta di **unire il codice** da un fork/branch alla **repo/branch padre**
- **Merge**: fonde i cambiamenti tra due branch in 3 strategie diverse per eseguire un **merge di B in A**:

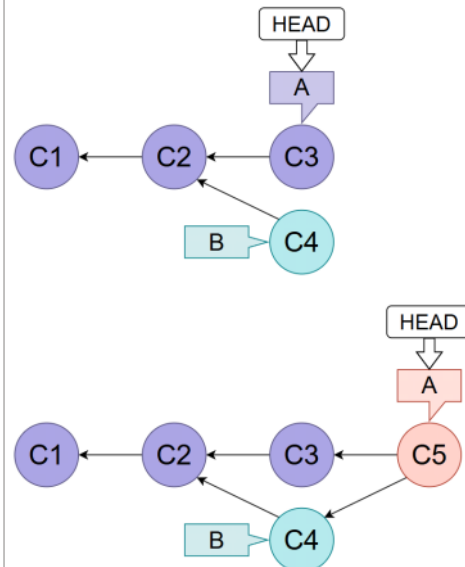
Fast-Forward

Il merge di B in A è valido solo se B è una continuazione di A



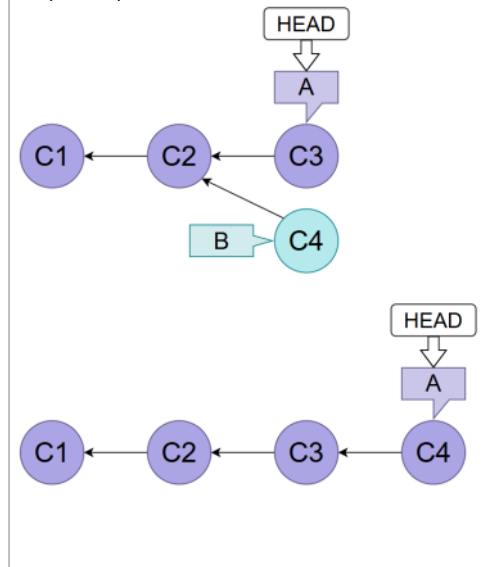
Merge Commit (Non-Fast-Forward)

Crea un nuovo commit i cui genitori sono gli ultimi commit delle due branch.



Rebase

Copia tutti i commit di B (non in comune tra A e B) e li replica dopo l'ultimo di A



Merge Conflict

Un **merge conflict** può avvenire quando eseguiamo il merge di due branch usando non -fast-forward o rebase

Esempio:

```
package main

import "fmt"

func main() {
    fmt.Println("")
}
```

Supponiamo che due sviluppatori modificano il messaggio della stampa in modo diverso allo stesso tempo:

- Il primo sviluppatore crea la commit "dev1-add-text" con il messaggio "hello!".

In questo caso il fast-forward merge va a buon fine

- Il secondo sviluppatore però crea nel frattempo la sua commit "dev2-add-text" con il messaggio "world"

`package main`

`import "fmt"`

`func main() {`

`<<<<< HEAD`

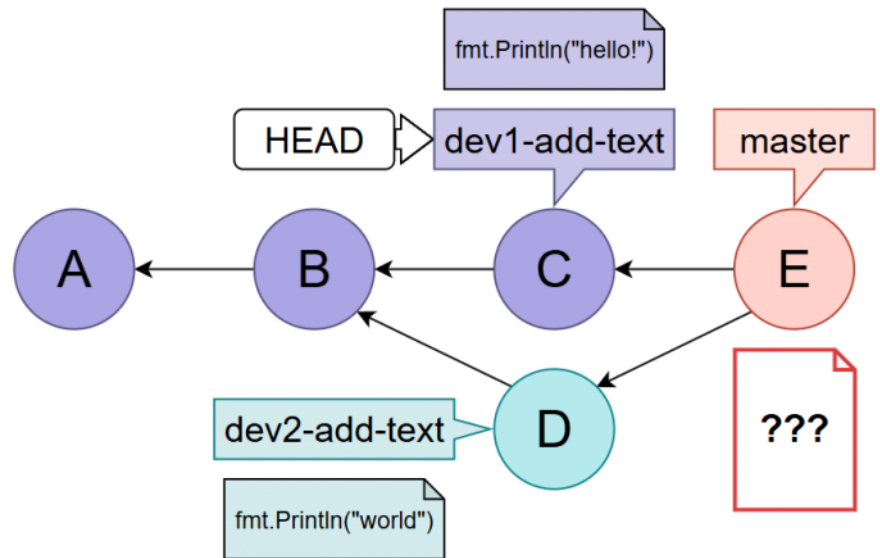
`fmt.Println("hello!")`

`=====`

`fmt.Println("world")`

`>>>>> dev2-add-text`

`}`



Adesso facendo il merge abbiamo che il **master** (l'HEAD) punta ad entrambi i commit e quindi abbiamo un **conflitto** sulla stampa del messaggio poiché abbiamo dobbiamo scegliere tra il messaggio da usare siccome sono diversi tra le due commit

Ci sono 3 modi risolvere:

- **Editare** il file **manualmente**
- **Mantenere** il file a cui abbiamo **già fatto il merge (dev1)**
- **Usare** il **nuovo file (dev2)**

Es: modifica manuale del file

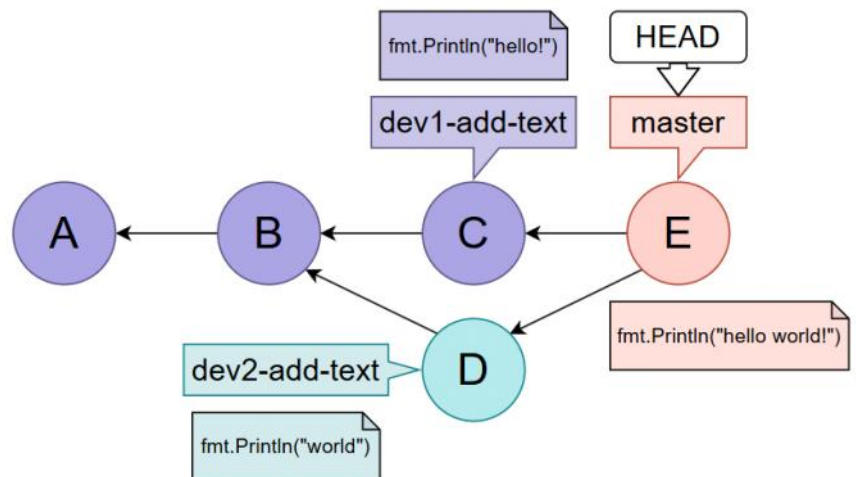
`package main`

`import "fmt"`

`func main() {`

`fmt.Println("hello world!")`

`}`



Comandi essenziali:

- **git init**: inizializza una **nuova repo** creando una cartella **.git**
- **git status**: visualizza lo **stato della working directory** per vedere i cambiamenti aggiunti rispetto all'ultimo commit
- **git add**: permette di **aggiungere file nuovi o modificati** alla working directory
- **git commit -m**: crea un **nuovo commit** salvando i cambiamenti eseguiti alla working directory ("-m" serve per aggiungere il messaggio del commit direttamente nella stessa linea di comando)
- **git log**: visualizza la **storia delle commit** create

```
HEAD <- empty
branches/
config
description
hooks/
info/ <- global exclude file for ignored patterns that you don't want to
track in a `.gitignore` file
objects/ <-- all the content for your database
refs/ <-- pointers into commit objects in that data (branches, tags,
remotes and more)
```

index <- staging area information (nothing yet)

File in una cartella .git nuova

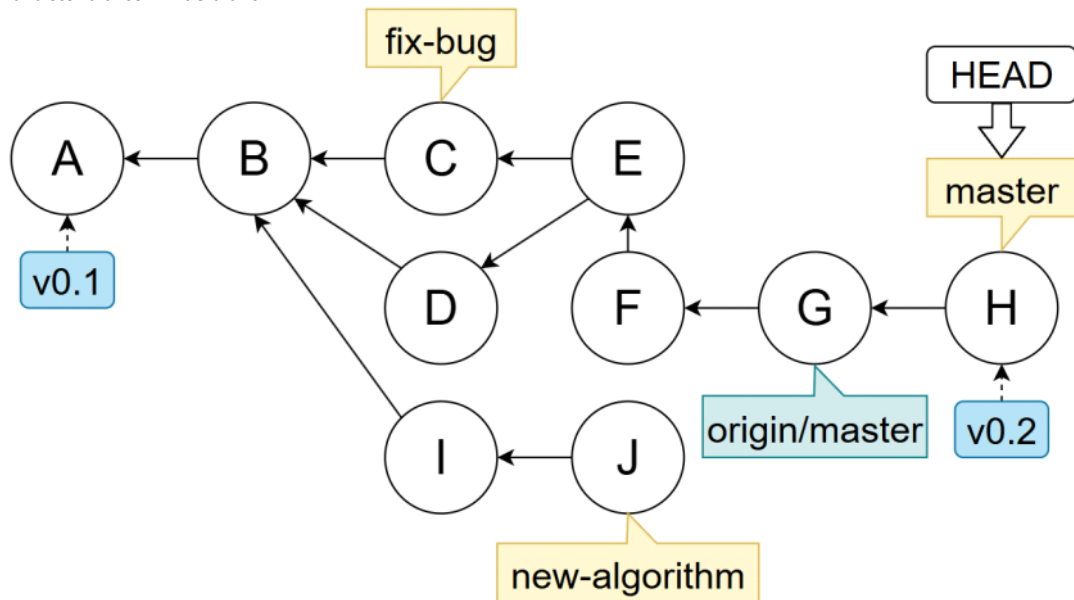
Es: creazione di una repo, aggiunta di un file "test.txt" e una cartella "thedir/" alla working directory e creazione di una nuova commit

- 1) Git richiede un nome e una email per eseguire i commit

- 2) Inizializziamo la repo con **git init**
- 3) Facendo **git status** ora ci dirà che non abbiamo ancora commit create e non ci sono file da aggiungere alla commit
- 4) Creiamo un file di testo vuoto "test.txt" e lo aggiungiamo alla working directory con **git add test.txt**
- 5) Facendo **git status** ora ci dirà che il file "test.txt" è stato aggiunto alla commit
- 6) Creiamo una cartella nuova "thedir" e aggiungiamo anche questa sempre con **git add thedir/**
- 7) Infine creiamo la commit con **git commit** aggiungendo il commento "make a dir"

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
git init
git status
touch test.txt
git add test.txt
git status
mkdir thedir
mv test.txt thedir/
git add thedir/
git commit -m 'made a dir'
```

Esempio completo di una storia di commit e branch



Abbiamo:

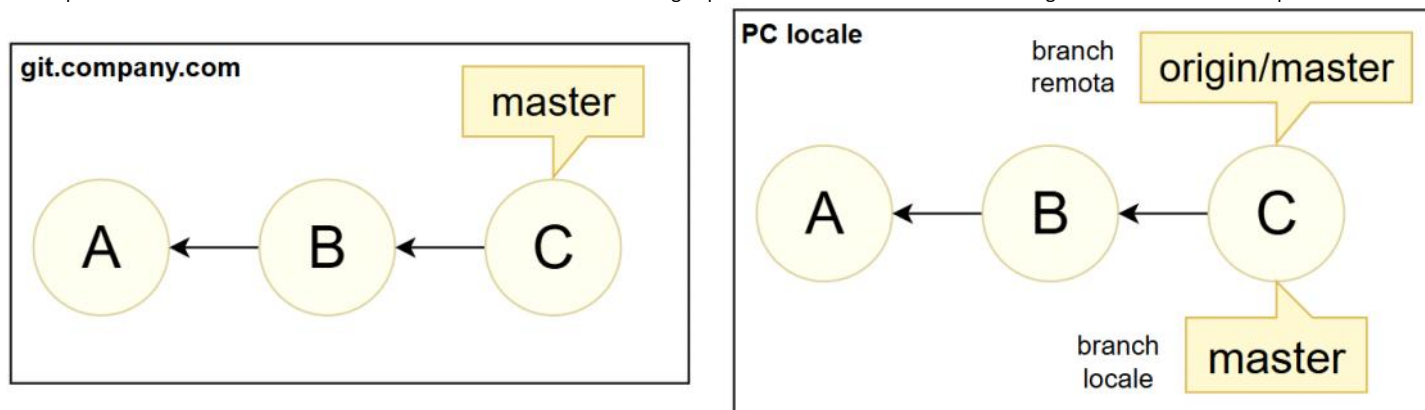
- A è la **radice** della repo (la prima commit) ed è la versione 0.1
- A e B sono condivisi da tutte le branch
- master contiene C, D (parallela di C), E, F, G e H (che è sia la versione 0.2 che la testa della repo)
- fix-bug e master condividono C
- La branch new-algorithm contiene solo A, B, I e J
- L'**origine remota** ha una vecchia copia del branch master fino a G, come vediamo con l'etichetta origin/master

Repositories Remote

Git può inviare e ricevere commits, tag e branch a **repo remoto** di utenti remoti (chiamati **remotes**) tipo un server.

Il nome remoto predefinito è **origin** e i nomi delle branch remote sono visualizzate nel formato **<remote>/<branch>** (es **origin/master**)

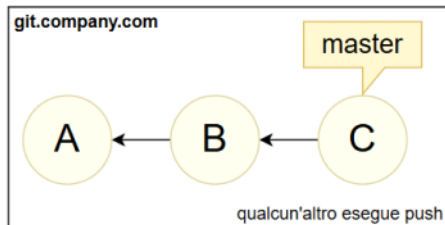
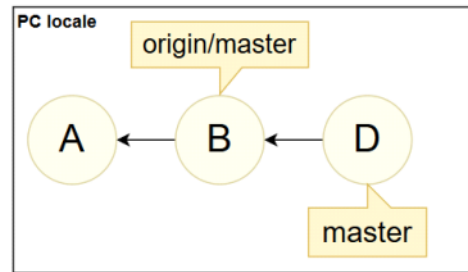
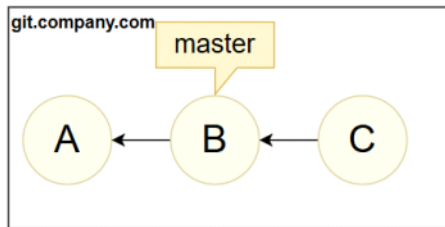
Non si possono creare direttamente delle commit sul branch remoto ma bisogna prima creare una **branch "locale"** collegata in relazione diretta a quella remota



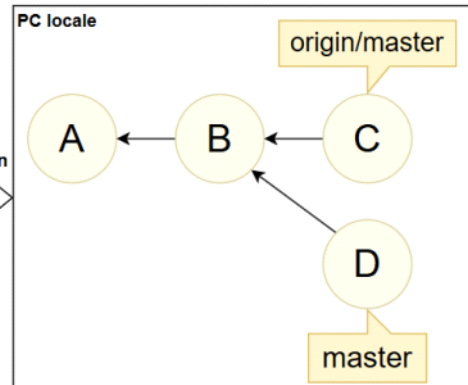
Alcune primitive basiche per sincronizzare le repo locali/remote sono:

- **Clone**: scarica una copia completa della repo remota per la prima volta
- **Fetch**: sincronizza i dati (commit e oggetti) dal repo remoto **senza modificare il branch locale**





git fetch origin



- **Pull:** scarica il contenuto del remoto e lo **unisce immediatamente** nel branch locale corrente per facilitare l'update del branch locale a quello remoto è l'**unione di git fetch e git merge** (cioè nell'esempio del fetch invece di creare due branch avrebbe fatto anche il merge tra C e D, chiedendo quale dei due commit tenere come master)

```
git pull <remote> <branch-remoto>
# Esempio: Scarica origin/main e lo unisce in main locale
git pull origin main
```

- **Push:** carica i commit locali sul branch remoto

```
git push <remote> <branch-locale>
# Esempio: Carica il branch main su origin
git push origin main
```

Undo

Git dà la possibilità di **annullare azioni o modifiche** effettuate facilmente. Si possono annullare:

- **Staging:** se non vuoi committare un file aggiunto con **git add** puoi rimuovere il file dall'area di staging con
`git reset HEAD -- <file>`
- **Modifiche locali:** se hai modificato un file e vuoi ripristinarlo alla versione dell'ultimo commit puoi **scartare le modifiche locali non committate** con
`git checkout -- <file>` (le modifiche vengono perse)
- **Commit pubblicato:** per annullare un commit **già pubblicato in remoto**, con
`git revert <hash-commit>`
viene creato un **nuovo commit opposto** che annulla le modifiche senza riscrivere la cronologia delle commit con
- **Commit locale:** per annullare un commit **non ancora pubblicato in remoto**, con
`git reset --soft HEAD~1`
viene riscritta la storia locale spostando il puntatore HEAD e il branch
 - **HEAD~1:** indica il commit **direttamente precedente** a HEAD (HEAD~2 indica il penultimo, HEAD~3 indica il terzultimo, ecc)
 - **--soft:** rimuove l'ultimo commit **mantenendo le modifiche** nell'area di staging
 - **--hard:** rimuove l'ultimo commit e **cancella le modifiche** (le modifiche vengono perse)

Si può anche **rimpiazzare** l'ultimo commit con uno nuovo riscrivendo la storia delle commit con

`git commit --amend`

(esempi pratici di queste funzioni di git stanno sul libro del prof "Introduction to full stack web development with Go and V ue.js" a pagina 30 o su <https://github.com/danrs/git-practice> oppure su <https://antonz.org/git-by-example> (in modo interattivo))

HTTP

lunedì 10 novembre 2025 10:41

HTTP (HyperText Transfer Protocol) è un protocollo fondamentale per la comunicazione su Internet

Esso è applicato al liv. **applicazione** nello stack di protocolli internet, nato nel 1991 e cambiato negli anni fino alla versione HTTP/3 del 2022.

HTTP è basato sul modello **client/server**:

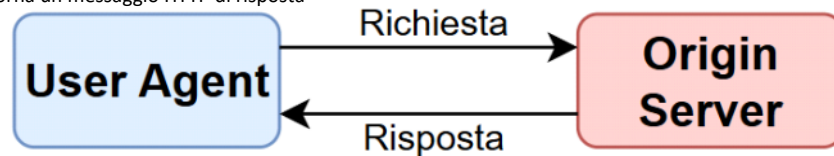
- **Client o User Agent (UA)**: qualsiasi programma che **avvia una richiesta** ad un server (es browser, app, ecc)
- **Server o Origin Server (O)**: un programma che ascolta per le richieste e risponde con una data risorsa o effettua una azione richiesta (es sito web)

Un'applicazione può assumere entrambi i ruoli: può fare da tramite per un client inviando una richiesta ad un altro server

Richiesta/Risposta

I dati vengono scambiati tramite sequenze di messaggi **richiesta/risposta** inviati durante una sessione di connessione attiva. Es:

- 1) L'**UA** invia una richiesta al **O**
- 2) L'**O** elabora la richiesta e ritorna un messaggio HTTP di risposta



Server Proxy

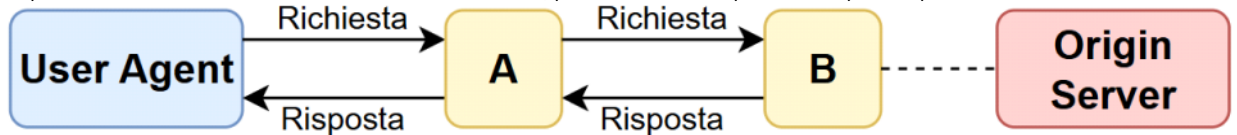
HTTP supporta anche molteplici **host intermediari** chiamati **proxy** che inoltrano i messaggi tramite **tunnel**.

Queste proxy possono scannerizzare/alterare/rifiutare richieste e risposte

I proxy possono anche fare da **cache**, salvando le risposte HTTP e inoltrandola alla richiesta del client senza coinvolgere il server originale

Il server deve però dichiarare una risposta come **cacheable** (memorizzabile nella cache) per fare in modo che le proxy salvino la risposta

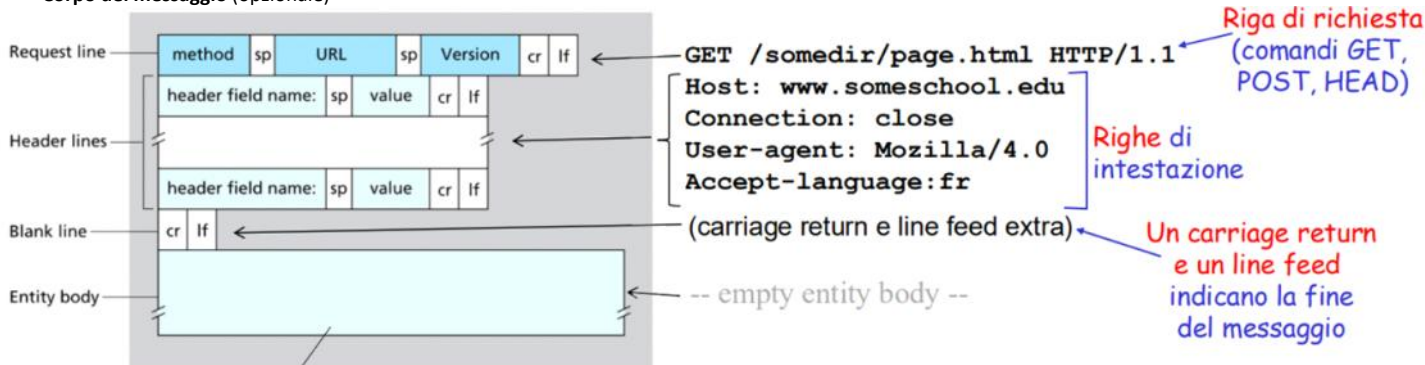
Es: Il proxy B risponde direttamente all'UA senza inoltrare la richiesta all'O, poiché ha salvato in precedenza quella risposta nella sua cache



Richiesta

Una **richiesta HTTP** è divisa in tre parti:

- **Riga di richiesta**: contiene il metodo HTTP, il percorso alla risorsa e la versione HTTP usata
- **Righe di intestazione**
- **Corpo del messaggio** (opzionale)



Campo vuoto per il GET, utilizzato per il POST

Intestazione	Descrizione
User-agent	Indica il programma client utilizzato
Accept	Indica il formato dei contenuti che il client è in grado di accettare
Accept-charset	Famiglia di caratteri che il client è in grado di gestire
Accept-encoding	Schema di codifica supportato dal client
Accept-language	Linguaggio preferito dal client
Authorization	Indica le credenziali possedute dal client
Host	Host e numero di porta del client
Date	Data e ora del messaggio
Upgrade	Specifica il protocollo di comunicazione preferito
Cookie	Comunica il cookie al server (verrà spiegato successivamente)
If-Modified-Since	Invia il documento solo se è più recente della data specificata

Intestazione della richiesta

Altri metodi:

- **DELETE**: richiede al server di rimuovere l'associazione tra la risorsa target e la sua funzionalità attuale
- **CONNECT**: stabilisce un tunnel verso il server identificato dalla risorsa target
- **OPTIONS**: descrive le opzioni di comunicazione per la risorsa target
- **TRACE**: esegue un test di loop-back del messaggio lungo il percorso verso la risorsa target
- **PATCH**: applica aggiornamenti parziali di una risorsa (invece di sovrascrivere l'intera risorsa, modifica solo i campi specificati)

GET	E' usato quando il client vuole scaricare un documento dal server. Il documento richiesto è specificato nell'URL. Il server normalmente risponde con il documento richiesto nel corpo del messaggio di risposta.
HEAD	E' usato quando il client non vuole scaricare il documento ma solo alcune informazioni sul documento (come ad esempio la data dell'ultima modifica). Nella risposta il server non inserisce il documento ma solo degli header informativi.
POST	E' usato per fornire input al server (contenuto dei campi di un form). L'input arriva al server nel corpo dell'entità.
PUT	E' utilizzato per memorizzare un documento nel server. Il documento viene fornito nel corpo del messaggio e la posizione di memorizzazione nell'URL.

Per inviare info al server

E' possibile anche usare GET

GET www.somesite.com/animalsearch?monkeys&banana

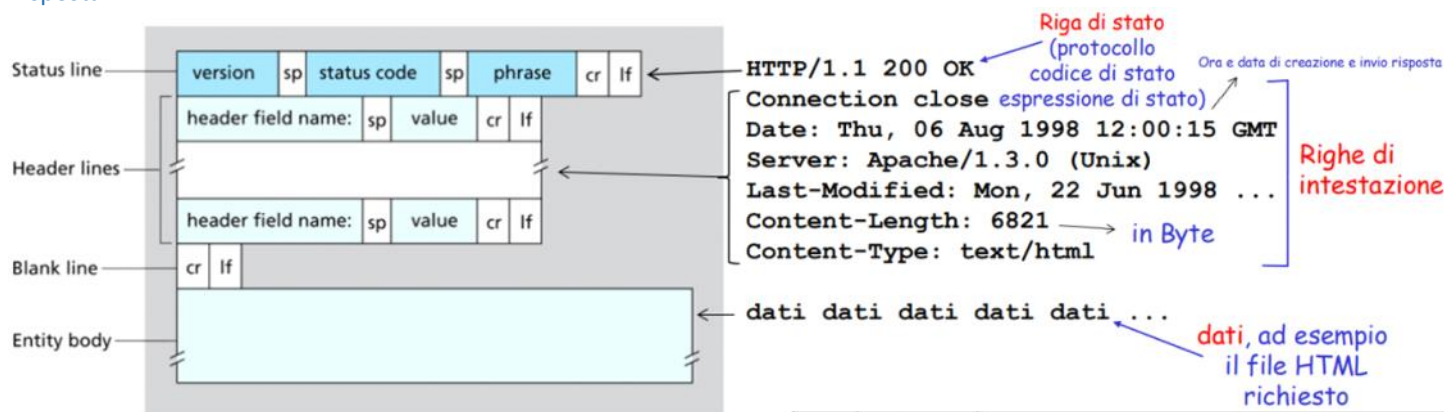
Metodi più comuni

Proprietà dei metodi:

- **Safe** (sicuro): il metodo non ha effetti collaterali sulla risorsa (è **solo lettura**). Può però cambiare lo stato del server in altri modi
I metodi sicuri sono: **GET, HEAD, OPTIONS e TRACE**
- **Idempotent** (idempotente): richieste identiche multiple con quel metodo hanno lo stesso effetto di una singola richiesta
I metodi idempotenti sono: **PUT e DELETE** e anche gli altri metodi sicuri
 - PUT aggiorna una risorsa esistente o ne crea una nuova tramite l'URL preciso (PUT /users/1) quindi **modifica sempre la stessa risorsa**
 - POST crea una nuova risorsa in una collezione (POST /users). **Più richieste POST creano più risorse identiche**
- **Cacheable** (memorizzabile in cache): metodi che possono consentire ad una cache di archiviare e usare una risposta
I metodi memorizzabili sono: **GET, HEAD e POST** (in determinate condizioni)

Metodo di richiesta	Payload Richiesta	Payload Risposta	Sicuro	Idempotente	Cacheable
GET	Opzionale	Si	Si	Si	Si
HEAD	Opzionale	No	Si	Si	Si
POST	Si	Si	No	No	Si
PUT	Si	Si	No	Si	No
DELETE	Opzionale	Si	No	Si	No
PATCH	Si	Si	No	No	No
CONNECT	Opzionale	Si	No	No	No
OPTIONS	Opzionale	Si	Si	Si	No
TRACE	No	Si	Si	Si	No

Risposta



Intestazione	Descrizione
Date	Data corrente
Upgrade	Specifica il protocollo preferito
Server	Indica il programma server utilizzato
Set-Cookie	Il server richiede al client di memorizzare un cookie
Content-Encoding	Specifica lo schema di codifica
Content-Language	Specifica la lingua del documento
Content-Length	Indica la lunghezza del documento
Content-Type	Specifica la tipologia di contenuto
Location	Chiede al client di inviare la richiesta a un altro sito
Last-modified	Fornisce data e ora di ultima modifica del documento

Intestazione della risposta

Code	Meaning	Examples
1xx	Information	100 = server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 = try again later

Alcuni codici di stato della risposta

Richieste e Risposte

Metodo	Descrizione	Sicuro	Idempotente	Risposta
GET	Leggere dati	Si	Si	200 OK
HEAD	Leggere informazioni (no body)	Si	Si	200 OK
POST	Inviare dati per creare una risorsa	No	No	201 Created (ritorna la risorsa creata)
PUT	Inviare dati per sostituire una risorsa esistente	No	Si	204 No Content
DELETE	Eliminare un dato esistente	No	Si	204 No Content
PATCH	Aggiorna parzialmente una risorsa	No	Si	200 OK

cURL

cURL è un comando che ci permette di vedere in azione una richiesta HTTP

Metodo	Scopo	Sintassi cURL
GET	Scarica il contenuto della risorsa	curl https://swapi.dev/api/people/1/
HEAD	Richiede solo le intestazioni (header)	curl -I https://swapi.dev/api/people/1/
POST	Invia dati per creare una nuova risorsa	curl -X POST -H "Content-Type: application/json" -d '{"title": "Test"}' https://jsonplaceholder.typicode.com/posts
PUT	Invia dati per sostituire una risorsa esistente	curl -X PUT -H "Content-Type: application/json" -d '{"id": 1, "title": "New Title"}' https://jsonplaceholder.typicode.com/posts/1

DELETE	Elimina una risorsa	curl -X DELETE https://jsonplaceholder.typicode.com/posts/1
HEADERS	Visualizza tutte le intestazioni	curl -i https://swapi.dev/api/people/1/

Es del libro

```
$ curl -v --http1.1 https://httpbin.org/user-agent
> GET /user-agent HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/7.74.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Thu, 27 Jul 2023 20:48:41 GMT
< Content-Type: application/json
< Content-Length: 34
< Connection: keep-alive
< Server: gunicorn/19.9.0
< Access-Control-Allow-Origin: *
< Access-Control-Allow-Credentials: true
<
{
  "user-agent": "curl/7.74.0"
}
```


JSON e YAML

lunedì 10 novembre 2025 13:01

JSON (JavaScript Object Notation)

JSON è formato di testo leggero per scambiare/archiviare dati (file .json) in modo strutturato. Ha la proprietà di essere facilmente leggibile per gli umani e facilmente analizzabile dalle macchine.

I dati in JSON sono organizzati in coppia **chiave-valore** e sono permessi diversi tipi di dati

Vengono usati due strutture dati principali:

Object (Oggetto)

collezione **non ordinata** di coppie chiave-valore racchiusa tra parentesi graffe "{}"

- **Chiave** (o nome): **stringa univoca** nell'oggetto racchiusa tra virgolette "
- **Valore**: scalari (int, float, bool), array, dizionari, strutture dati annidate

Es:

```
{
  "WASA": {
    "name": "Web and Software Architecture",
    "semester": 1
  }
}
```

Array

collezione **ordinata** di valori, racchiusa tra parentesi quadre "[]". I valori possono essere di diverso tipo nello stesso array e i tipi sono: scalari (int, float, bool), array, dizionari, strutture dati annidate

Es:

```
{
  "wasaWeekdays": ["Tuesday", "Thursday"]
}
```

YAML (YAML Ain't Markup Language)

YAML è un linguaggio di **serializzazione dei dati** facilmente leggibile per gli umani usato per configurazione/trasferimento/archiviazione dati (file .yaml)

Usa un formato di testo basato sull'indentazione di Python. I file JSON sono YAML validi

Un documento YAML è definito con:

- **Mappe (oggetti)**: racchiusi tra parentesi graffe "{}"
- **Liste (array)**: racchiusi tra parentesi quadre "[]" o usando il simbolo "-" per ogni oggetto dell'array
- Le **stringhe** non richiedono di essere delimitate a meno che non sono ambigue e richiedono in quel caso le virgolette "
- Qualsiasi cosa dopo un simbolo cancelletto "#" è un **commento**
- un singolo file YAML può contenere **più documenti YAML** separati da "---" o "..."
- Le **stringhe multi-linea** possono essere specificate in vari modi in base al simbolo:
 - ">" indica che le righe a capo sono rimosse (trasforma la stringa in una singola linea)
 - "|" indica che le righe a capo sono mantenute
 - Di default solo la stringa a capo alla fine della stringa è mantenuta: "-" la rimuove, "+" mantiene tutte le righe a capo alla fine della stringa

Json

```
{
  "anObject": {
    "aNumber": 42,
    "aString": "This is a string",
    "aBoolean": true,
    "nothing": null,
    "anArray": [
      1,
      {
        "name": "value",
        "anotherName": 12
      },
      "something else"
    ]
  }
}
```

YAML

```
anObject:
  aNumber: 42
  aString: This is a string
  aBoolean: True
  nothing: null
  anArray:
    - 1
    - anotherObject:
        name: value
        anotherName: 1234
    - something else
```

REST

mercoledì 12 novembre 2025 18:47

REST (Representational State Transfer) è uno stile di architettura software per sistemi web per fare in modo che Client e Server comunichino in modo ordinato e scalabile.

REST si basa sull'idea del trasferimento delle **rappresentazioni di risorse** da un computer ad un altro (es da server a client). Un sistema **RESTful** implementa tutti i principi **REST** e soddisfa tutti i vincoli

I concetti fondamentali di REST sono:

- **Risorsa**: qualunque informazione che può essere nominata (es: documento, immagine, ecc) ed è un **set di elementi o valori** che variano nel tempo.
Due risorse possono anche indicare allo stesso valore (es "versione v2.1" e "ultima versione" di un programma)
Es: Utente n.45
- **URI**: un indirizzo che ci permette di accedere e usare una risorsa
Es: negozio.com/users/45
- **Rappresentazione della risorsa**: per accedere alla risorsa il server invia lo **stato corrente** di una risorsa definita in dati e metadati e il formato dei dati è noto come **"media type"**
Es: { "nome": Mario, "id": 45 }

URI (Uniform Resource Identifier)

Un **URI** è una stringa di caratteri usata per identificare e usare una risorsa logica o fisica (es: <http://example.com/users>)

Gli URI hanno due specializzazioni:

- **URL (Uniform Resource Locator)**: usato per localizzare risorse su internet e specificare come accederci
- **URN (Uniform Resource Name)**: usato per dare un nome univoco globalmente ad una risorsa

È buona prassi usare **sostantivi singolari** per una singola risorsa e **sostantivi plurali** per una collezione di risorse

La convenzione di notazione per le URI sono:

- **Slash in avanti "/"**: per esprimere la gerarchia
- **Preferire i trattini "-"** agli underscore "_"
- Usare solo **lettere minuscole**
- Non usare **estensioni di file** (il media type è comunicato nell'header)
- Usare **componenti query** per filtrare

La definizione formale di un URI da RFC 3986 è:

URI = schema ":" ["//" autorità] percorso ["?" query] ["#" frammento]
autorità = [userinfo "@"] host [":" port]

Dove:

- **Schema**: protocollo o metodo di accesso (es https, ftp, mailto, ecc)
- **Autorità**: identità del server/host (eventualmente con credenziali e porta) (es user:pass@example.com:8080)
- **Percorso**: posizione della risorsa sul server (es "/cartella/pagina.html")
- **Query**: parametri per filtrare o passare dati alla risorsa (es "?id=45&lang=it")
- **Frammento**: riferimento interno nella risorsa (es "#sezione3" in una pagina HTML)

Vincoli REST

REST impone dei vincoli per costruire un sistema:

- 1) **Client-Server**: impone la separazione di responsabilità (Client: UI - Server: data). Migliora la portabilità e la scalabilità
- 2) **Stateless**: ogni richiesta del client deve avere tutte le info necessarie per comprendere la richiesta senza usare contesti sul server
Così lo **stato della sessione** è mantenuto dal client e lo **stato delle risorse** dal server
- 3) **Cacheable**: il client può usare una **copia** della rappresentazione delle risorse (dati)
- 4) **Uniform-Interface**: L'uniformità **semplifica** ma riduce l'efficienza, garantendo però la comunicazione uguale per tutti i componenti.

Le implementazioni sono separate dai servizi che forniscono, poiché un client deve solo sapere come chiamare un servizio (e non come è implementato)

Ci sono **quattro vincoli** per le interfacce:

- Identificazione della risorsa (**URI**)
- Manipolazione della risorsa tramite rappresentazioni
- Messaggi auto-descrittivi che contengano tutte le informazioni necessarie
- Hypermedia come motore dello stato applicativo: il client non deve conoscere tutti i link di un sito ma solo partire da una pagina e seguire i link forniti

- 5) **Layered System**: in un'architettura a livelli ci sono diversi componenti (gli intermediari fanno sia da client sia da server) e ogni componente vede solo il livello con cui interagisce

HTTP vs REST

In un'architettura RESTful, i **metodi HTTP** (noti come "**verbi**") definiscono l'**azione** che si desidera **eseguire sulla risorsa** (URI)

Gli URI identificano la **risorsa**, i metodi HTTP definiscono **cosa fare** con quella risorsa. Es:

- **GET (Corrispondenza CRUD: Read)**: recupera una risorsa o una collezione
- **POST (Corrispondenza CRUD: Create)**: crea una nuova risorsa in una collezione
- **PUT (Corrispondenza CRUD: Update/Replace)**: sostituisce completamente una risorsa esistente
- **DELETE (Corrispondenza CRUD: Delete)**: rimuove una risorsa specifica
- **PATCH (Corrispondenza CRUD: Update/Modify)**: applica modifiche parziali a una risorsa

Codici di Successo RESTful Comuni (2xx)

- **200 OK**: usato per richieste GET, PUT, PATCH o DELETE eseguite con successo
- **201 Created: DEVE** essere restituito dopo un **POST** che crea una nuova risorsa e la risposta include la nuova risorsa creata
- **204 No Content**: quando l'azione ha avuto successo ma non c'è corpo di risposta da restituire al client (tipico dopo DELETE o PUT)

Codici di Errore Cliente Comuni (4xx)

- **400 Bad Request**: la richiesta non può essere soddisfatta a causa di un errore del client (es JSON malformato, dati mancanti)
- **401 Unauthorized**: richiesta rifiutata perché manca l'autenticazione
- **403 Forbidden**: utente autenticato ma non ha i permessi per accedere alla risorsa
- **404 Not Found**: la risorsa richiesta non esiste

Azione o Risorsa

- **Azione 1: ottenere dati**

- **Obiettivo:** ottenere tutti i prodotti in vendita
- **Design corretto:** GET /products
- **Motivazione:** stiamo recuperando la collezione di prodotti
- **Azione 2: creare una nuova risorsa**
 - **Obiettivo:** aggiungere un nuovo utente
 - **Design corretto:** POST /users (corpo della richiesta contiene i dati del nuovo utente)
 - **Risposta attesa:** 201 Created. L'API dovrebbe restituire la nuova risorsa e l'URI per accedervi
- **Azione 3: eseguire una funzione specifica**

Se un'azione **non è mappabile a CRUD** (es "Pubblica un documento" o "Cambia password") ci sono due approcci accettati:

 - 1) **Modeling come Risorsa:** modellare l'azione come una risorsa
Es: pubblicare un documento
URI: POST /documents/{id}/publish
 - 2) **PATCH (modifica stato):** usa PATCH per aggiornare un attributo di stato
Es: pubblicare un documento aggiornando il suo stato
URI: PATCH /documents/{id}{Corpo: {"Status": "published"}}

Il secondo approccio è preferito perché mantiene un modello più puro, in cui si **agisce** sullo **stato della risorsa**

API

mercoledì 12 novembre 2025 18:29

Una **API** (Application Programming Interface) è un **insieme di regole e protocolli** che permettono a due software di interagire tra di loro. L'API funge da **contratto di interazione** tra **Consumer** (client) e **Provider** (servizio).

Essa specifica:

- Le **richieste** possibili che possiamo effettuare
- I **parametri** da inviare nelle richieste
- I **valori di ritorno** (non viene specificato come funzionano le funzioni)
- Qualsiasi **formato di dato** richiesto (es. JSON, XML, YAML)

Per stabilire un contratto, l'API deve essere definita esplicitamente attraverso **documentazione** o **linguaggi di descrizione** standardizzati

I vantaggi dell'uso delle API sono:

- **Interfaccia esplicita**: viene definito chiaramente le aspettative e le modalità di interazione
- **Contratto infrangibile**: stabilisce un insieme di regole che entrambi le parti devono rispettare
- **Nascondimento delle informazioni**: la logica delle funzioni del Provider è nascosta al Consumer che conosce solo l'interfaccia

Esistono API **locali** (per linguaggi di programmazione, OS, librerie software, hardware) e **remote (Web API)** (per interfacce basate su protocolli di rete)

In più le API possono essere:

- **Pubbliche**: condivise liberamente senza autorizzazione disponibili per l'uso del pubblico. L'accesso può essere **limitato** ad alcuni utenti tramite **API Tokens**
- **Private**: usate internamente da aziende o sistemi chiusi. L'accesso è limitato ai componenti interni

OpenAPI e OAS

Per standardizzare la documentazione delle varie API è stato creato lo standard **OAS (OpenAPI Specification)** per definire e documentare Web APIs basate su HTTP

È un formato **vendor-neutral** (indipendente dal fornitore) e rappresenta lo **standard industriale** per descrivere le API moderne

Un documento OAS può essere definito con YAML (più comune) o JSON

Es struttura base di un documento OAS

openapi: 3.0.0

info:

title: An example OpenAPI document

description: |

This API allows writing down marks on a Tic Tac Toe board

and requesting the state of the board or of individual cells

version: 0.0.1

paths: {} # Gli endpoint dell'API verrebbero definiti qui

OAS 3.x Cheat Sheet

Struttura e Informazioni Generali

Componente Principale	Tipo	Descrizione	Note Chiave
openapi	string	Versione della specifica	Usa 3.1.0 per la piena compatibilità con JSON Schema
info	Object	Metadata sull'API	Contiene title , version (API) e description (CommonMark supportato)
servers	Array	Elenco degli URL base per l'API	Ogni elemento ha url , description e variables
paths	Object	Definizione degli endpoint (risorse)	Mappa i percorsi (es /users/{id}) ai Path Item Objects
components	Object	Contenitore per elementi riutilizzabili	Schemi, risposte, parametri, security schemes, ecc

Percorsi e Operazioni

Path Item Object

Proprietà	Tipo	Descrizione
summary	string	Breve riepilogo del percorso
description	string	Descrizione dettagliata
get, put, post, delete, ecc	Operation	Corrispondono ai verbi HTTP

Operation Object

Proprietà	Tipo	Descrizione
operationId	string	ID univoco per l'operazione (utile per la generazione di codice client/server)
summary/description	string	Riepilogo/descrizione dell'operazione
tags	Array	Usato per raggruppare operazioni nella documentazione
parameters	Array	Elenco di oggetti Parameter
requestBody	RequestBody	Payload per i metodi che lo supportano (POST, PUT, PATCH)
responses	Responses	Definizioni di tutte le risposte possibili (codici HTTP)
security	Array	Sovrascrive la sicurezza globale per questa operazione

Parametri e Corpo della Richiesta

Parametro (Parameter)

Definito sotto l'Operation o in components/parameters

Proprietà	Valore in	Descrizione
-----------	-----------	-------------

name	string	Nome del parametro
in	query	Parametro nell'URL (es ?limit=10)
	path	Parametro nel percorso (es /users/{id}) Richiede required: true
	header	Parametro nell'header HTTP
	cookie	Parametro nel cookie
required	boolean	Indica se il parametro è obbligatorio
schema	Schema	Definizione del tipo di dato del parametro
style	string	Stile di serializzazione (es. form, simple)
content	Media Type	Alternativa a schema per payload complessi nel parametro (raro)

Corpo della Richiesta (RequestBody)

Proprietà	Tipo	Descrizione
description	string	Spiegazione del payload
required	boolean	true se il corpo è obbligatorio
content	Map[string:MediaType]	Mappa i tipi di media (es application/json) ai loro schemi
content > Media Type Object		Contiene lo schema e opionalmente example o examples

Risposte (Responses Object)

Codice HTTP	Tipo	Descrizione
200, 201, 404, ecc	Response	Un'istanza di Response Object
default	Response	Risposta per tutti i codici non documentati esplicitamente
Response Object		Contiene description, headers (header della risposta) e content

Schemi (Schemas) e Tipi di Dato

Gli schemi sono definiti sotto **components/schemas** e usano parole chiave da JSON Schema (OAS 3.1 supporta JSON Schema 2020-12)

Parola Chiave	Tipo	Descrizione
type	string	Tipo di dato principale
format	string	Formato specifico (affina il type)
properties	Object	Per schemi di tipo object
items	Schema	Per schemi di tipo array
enum	Array	Restringe i valori a un elenco specifico
minimum/maximum	number	Restrizioni numeriche
required	Array	Elenco delle proprietà obbligatorie (per gli oggetti)

Tipi Primitivi e Formati

Tipo	Formato	Note
integer	int32, int64	
number	float, double	
string	byte, binary, date, date-time, password	
boolean		
∅	N/A	(OAS 3.1+) Tipo esplicito per valori JSON null

Polymorphism e Combinazione di Schemi

Usato per l'**ereditarietà** e per definire tipi complessi in cui sono possibili schemi multipli

Parola Chiave	Logica	Significato
allOf	AND	Il dato deve soddisfare TUTTI gli schemi elencati (Utile per ereditarietà)
oneOf	XOR	Il dato deve soddisfare ESATTAMENTE UNO degli schemi elencati
anyOf	OR	Il dato deve soddisfare UNO O PIU degli schemi elencati
not	NOT	il dato NON deve soddisfare lo schema elencato

Riutilizzo di Componenti (\$ref)

Il componente **\$ref** viene usato per fare riferimento a uno schema, una risposta, un parametro, ecc definito in **components**

Target	Esempio \$ref
Interno	#/components/schemas/User
URL remoto	https://example.com/models.yml#/components/schemas/User

Sicurezza

Definisci gli schemi di sicurezza in **components/securitySchemes** e applicali usando la parola chiave **security** a livello di documento o di singola Operation

Tipi consentiti:

- **http**: Basic o Bearer Token (specificato tramite **scheme**)
- **apiKey**: chiave API (specificato tramite **in**: header, query o cookie)
- **oauth2**: flussi OAuth 2.0 (specificato in **flows**)
- **mutualTLS** (OAS 3.1+)
- **openIdConnect**

Applicazione:

```
components:
  securitySchemes:
    # Definizione
    ApiKey:
      type: apiKey
      name: X-API-KEY
      in: header

  security:
    # Applicazione globale: richiede ApiKey per tutte le operazioni
    - ApiKey: []

  paths:
    /widgets:
      post:
        # Applicazione locale: sovrascrive,
        # rimuovendo la sicurezza se l'array è vuoto
        security: []
```


API Best Practices

giovedì 27 novembre 2025 12:12

Required

In OpenAPI, il campo **required** definisce le regole di validazione del **payload** e non il comportamento logico dell'API

La chiave **required** in uno schema determina se un campo deve **essere presente** nel payload **in ingresso** (es nel requestBody di POST o PUT)

Contesto	Campo id (readOnly: true)	Regola di required
POST /fountains	L'ID non può essere inviato	Deve essere omesso nel requestBody
PUT /fountains/{id}	L'ID non deve essere modificato dal client	Deve essere inviato (per la sostituzione completa) ma il server lo ignora/valida
GET /fountains/{id}	L'ID è restituito	Deve essere presente nello schema di risposta

I campi opzionali (quelli non elencati in **required: []**) e i campi in sola lettura (**readOnly: true**) dovrebbero essere gestiti in modo specifico negli esempi

- **Es requestBody (POST):**
 - L'esempio **NON DEVE** includere il campo **id**
 - **Obiettivo:** guidare lo sviluppatore client a inviare il payload minimo e corretto
- **Es responseBody (201 Created):**
 - L'esempio **DEVE** includere il campo **id**
 - **Obiettivo:** mostrare a client la struttura completa della risorsa creata

Definizione dello schema (in components/schemas/Fountain):

```
Fountain:
  type: object
  properties:
    id:
      $ref: "#/components/schemas/FountainIdSchema"
    state:
      type: string
  required: # State è richiesto, l'ID no (è generato)
  - state
```

Uso nell'operazione **POST /fountains**: l'es per la richiesta (requestBody) dovrebbe riflettere i campi richiesti (state, latitude, longitude) ma **escludere l'ID**:

```
requestBody:
  content:
    application/json:
      example:
        state: "good"
        latitude: 41.89025
        longitude: 12.49237
```

URI

URI rappresentano **risorse**, non azioni.

Bisogna usare **sostantivi** nelle URI poiché identificano oggetti (risorse) e non si deve **MAI includere verbi** nell'uri della risorsa.

L'azione (cosa fare) è definita dal **metodo HTTP**

Metodo	Esempio (Corretto)	Esempio (Errato)
GET	/managed-devices/{id}	/get-managed-devices/{id}
PUT	/managed-devices/{id}	/update-managed-devices/{id}
POST	/users	/create-user
DELETE	/managed-devices/{id}	/remove-managed-devices/{id}

È buona prassi usare **sostantivi singolari** per una singola risorsa e **sostantivi plurali** per una collezione di risorse

- Singolo: /users/admin
- Collezione: /users

Gerarchia Logica e Separatori

Gerarchia: usa lo **slash "/"** per esprimere una **gerarchia logica** nelle relazioni tra risorse. Questo aiuta l'API a crescere in modo ordinato

Separatore: lo **slash finale** è comunemente usato per indirizzare a una collezione di risorse (es /users/)

Es annidamento logico:

/users/{userid}/devices/{deviceid}

(il dispositivo {deviceid} è una risorsa contenuta nella collezione di dispositivi dell'utente {userid})

Gestione Errori e Filtri

Codici di Stato HTTP

REST ≠ OK/Errore. Molti errori possono essere mappati direttamente a specifici codici di **stato HTTP**

- Parametri **mancanti** o valori **non validi**: **400 Bad Request**
- Risorsa **non trovata**: **404 Not Found**
- **Successo**: **200 OK**, **201 Created**, **204 No Content**

Corpo di errore: se necessario, conviene fornire dettagli aggiuntivi (per gli sviluppatori) in un corpo di risposta comune (es JSON con campi specifici sull'errore)

Query per Filtri e Paginazione

USA i componenti **query string** della URI per filtrare e impaginare i risultati

MAI usare il corpo (body) o l'URI el percorso (path) per i filtri. La collezione è unica, i parametri la limitano

Es:

/managed-devices/?region=USA

Dati Binari e Corpo della Richiesta

Evita embedding in JSON: è sconsigliato incorporare grandi dati binari (es. foto) nei JSON per motivi di performance

è consigliato:

- 1) **Invio (Client → Server)** con due strategie:
 - **Approccio standard:** Usa **multipart/form-data** per inviare dati strutturati (JSON) e il file in **un'unica richiesta**
 - Carica il file in una **API dedicata** (POST /images/) e usa l'**URL/ID** dell'immagine nella richiesta JSON principale
- 2) **Ricezione (Server → Client):**
 - Restituisci un JSON con l'informazione strutturata e l'**URL completo** per **accedere** al file binario in un **endpoint separato** (es /images/{identifier})

Multipart/form-data

L'approccio standard con multipart/form-data permette di caricare insieme sia il JSON che il File in un'unica richiesta al server

Es: aggiungere una nuova fontana (JSON) e caricarne le foto (File)

Corpo della richiesta:

- **Media type:** multipart/form-data
- **Parte 1 (JSON):** Content-Disposition: name = "fountain_data" (contiene il JSON { "state": "good", ...})
- **Parte 2 (File):** Content-Disposition: name = "photo" (contiene i byte binari dell'immagine)

Quando il server risponde deve **evitare** di inviare direttamente il contenuto nel JSON

- 1) **Risposta diretta del file:** Se un endpoint serve **solo** il file, la risposta non è JSON. Es:
GET /images/fountain_123.jpg
 - **Status:** 200 OK
 - **Content-Type:** image/jpeg
 - **Body:** contiene i byte binari dell'immagine
- 2) **Referenziamento nel JSON:** Quando si recuperano le risorse, il JSON deve contenere un riferimento (URL/ID) al file. Es:

```
{
  "id": 1,
  "state": "good",
  "latitude": 41.89025,
  "longitude": 12.49237,
  "photo_url": "/api/v1/images/fountain_123.jpg" # riferimento al file
}
```

Esempio con cURL:

Il comando **cURL** semplifica l'uso di multipart/form-data grazie all'opzione **-F** (o **--form**), che esegue 3 azioni cruciali automaticamente:

- 1) Imposta l'intestazione **Content-Type:** multipart/form data
- 2) Genera e imposta un **valore di boundary unico** per la richiesta
- 3) Formatta il corpo della richiesta, separando correttamente i dati

Il comando cURL richiede un **parametro -F** per **ogni "parte"** che deve essere inviata

Esempio con l'aggiunta della nuova fontana e la sua foto:

Parte	Opzione cURL	Descrizione
Dati strutturati (JSON)	-F "fountain_data=@data.json;type=application/json"	Carica il contenuto di data.json e ne specifica il Content-Type
File binario (immagine)	-F "photo=@image.jpg;type=image/jpeg"	Carica il file binario image.jpg

In pratica:

- 1) **Preparazione del file JSON** (data.json): creiamo un file JSON che rappresenta i dati strutturati della nuova fontana (senza ID perché è generato dal server)

```
{
  "state": "good",
  "latitude": 41.89025,
  "longitude": 12.49237,
}
```

- 2) **Comando cURL completo:**

```
curl -X POST https://api.nasoniroma.it/v1/fountains \
-F "fountain_data=@data.json;type=application/json" \
-F "photo=@percorso/alla/foto.jpg;type=image/jpeg"
```

Risultato: cURL crea una **singola richiesta HTTP POST**, contenente sia i dati JSON che i byte dell'immagine, pronti per essere elaborati dal server

Gestione delle Sotto-Collezioni

Se una risorsa (o azione) ha effetto **solo sull'utente autenticato** (es mutare Bob), **non annidarla** direttamente sotto la risorsa che rappresenta l'entità target (Bob)

Esempio sbagliato:

/users/{userid}/mute

sembra che tu stia mutando l'utente {userid} per tutti

Soluzione (Collezione Relazionale): tratta l'azione come una **collezione di risorse** relative all'utente autenticato

Esempio corretto:

Per mutare qualcuno lo inseriamo nella **collezione personale muted** (gente che ho messo a tacere)

/me/muted/{mutedid}

mutedid rappresenta l'ID dell'utente che vogliamo mutare

- **POST/PUT** a questo percorso: mette a tacere un utente
- **DELETE** a questo percorso: toglie il muto all'utente

Questo approccio garantisce che le azioni siano **idempotenti**, che l'operazione di "unmuting" sia nativamente gestita da DELETE, e che la risorsa sia chiaramente **per-utente-autenticato**

Aggiornamento delle Risorse: PUT vs PATCH

Per **modificare** una risorsa **esistente** si usano principalmente due metodi HTTP con semantiche molto diverse: **PUT** e **PATCH**

PUT (Sostituzione Completa)

- **Scopo:** Sostituire la risorsa bersaglio con il contenuto completo inviato nel corpo della richiesta
- **Idempotenza:** **Si**. Eseguire la stessa richiesta PUT più volte produce lo stesso stato finale sul server
- **Dati richiesti:** L'intera **rappresentazione** della risorsa. Se si omette un campo questo dovrebbe essere eliminato o azzerato dal server
- **Uso tipico:** Modifiche **complete**, come la riscrittura di un documento

Es: aggiornamento fontana con PUT

Il client invia l'intera risorsa, anche se ha modificato solo lo stato

```
{
  "id": 1
  "state": "faulty", # modificato
  "latitude": 41.89025, # Non modificato, ma deve essere inviato
  "longitude": 12.49237 # Non modificato, ma deve essere inviato
}
```

PATCH (Aggiornamento Parziale)

- **Scopo:** Applicare una serie di **modifiche parziali** alla risorsa bersaglio
- **Idempotenza:** **Non intrinsecamente**. Una richiesta PATCH è idempotente solo se il server la implementa per esserlo
- **Dati richiesti:** **Solo i campi da modificare**. I campi omissi rimangono invariati
- **Uso tipico:** Modifiche **mirate**, come cambiare solo lo stato/flag di una risorsa (es: mutare utente, cambiare stato di una fontana, ecc)

Es: aggiornamento fontana con PATCH

Il client invia **solo** il campo modificato, riducendo il traffico

```
{
  "state": "faulty" # invia solo la modifica
}
```

Esempio OpenAPI per PATCH

```
/fountains/{id}:
  patch:
    operationId: updateFountainState
    summary: Segnala un guasto o correggi le coordinate
    description: Applica aggiornamenti parziali ai campi forniti
    requestBody:
      description: |
        Proprietà della fontana da aggiornare
        (solo i campi specificati vengono modificati)
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              state:
                $ref: "#/components/schemas/Fountain/properties/state"
              latitude:
                $ref: "#/components/schemas/Fountain/properties/latitude"
              longitude:
                $ref: "#/components/schemas/Fountain/properties/longitude"
    responses:
      "200":
        description: Aggiornamento applicato con successo
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Fountain"
      "404":
        description: Fontana non trovata
```

Confronto con le API dei Social

Piattaforma	Naming delle Risorse	Esempio (Path GET)	Scelta REST
Meta/Facebook (Graph)	Nodi e Edges. Orientato al grafico	/v18.0/{user-id}/posts	Focalizzazione sull'Oggetto. Gli Edges gestiscono le relazioni
Twitter/X (API v2)	Nomi plurali. Versionamento esplicito	/2/users/{id}/tweets	Stile REST classico. Collezioni ben definite. Nomi intuitivi
LinkedIn	Risorse plurali. Uso di me	/v2/me	Pratic comune. Non serve ID utente per l'utente corrente. Pulizia

Osservazione: nomi **plurali** per le collezioni. ID univoci usati

Azione	Metodo HTTP	Facebook Graph	Twitter/X API v2	Principio REST
Creazione	POST	POST /{user-id}/feed	POST /2/tweets	Aggiunge una nuova risorsa
Lettura	GET	GET /{user-id}	GET /2/tweets/{id}	Recupera la rappresentazione. Sicura, idempotente
Aggiornamento	PATCH/PUT	POST /{post-id}	PUT /2/tweets/{id}	Punto di rottura: Facebook usa POST per aggiornamenti. Compromesso azione/stato
Eliminazione	DELETE	DELETE /{post-id}	DELETE /2/tweets/{id}	Rimuove la risorsa. Pulito

Scelta progettuale: POST per aggiornare è un compromesso frequente. Semplifica la gestione degli stati complessi

Caratteristica	Facebook Graph	Twitter/X API v2	Vantaggio
Selezione	Parametro fields	Parametri user.fields,	Efficienza massima. Riduce il payload. Riduce il carico

campi		tweet.fields	sul server
Paginazione	Cursor-based: after/before	Cursor-base: next_token/prev_token	Scalabilità superiore. Più veloce di offset/limit sui grandi dati
Versionamento	Versione nel path (v18.0)	Versione nel path (/2/)	Stabilità assicurata. Garantisce la retrocompatibilità
Gestione errori	Codici HTTP standard (4xx, 5xx). Oggetto JSON di errore esplicito	Codici HTTP. Array di oggetti errors	Messaggi self-descriptive. Chiare ragioni di fallimento

Hi-Lo Game 1

martedì 18 novembre 2025 17:37

Con questo gioco vedremo l'uso delle **API**, principi **REST**, **Risorse** e **Operazioni HTTP**

Nel gioco devi indovinare un num. segreto tra 1 e 100, con un massimo di 10 tentativi. Ad ogni tentativo, si riceve una risposta che ti dice se il numero che hai detto è **più alto** (hi) o **più basso** (lo) rispetto al num. segreto

Requisiti di gioco: questi requisiti sono la base per il design dell'API:

1. **Avviare** una nuova partita
2. **Accettare** un tentativo (fino a 10) e restituire **hi, lo, correct**
3. **Resettare** una partita, generando un nuovo numero segreto
4. **Ottenere** la lista di ogni tentativo di una partita, con i risultati correlati
5. **Ottenere** la lista di tutte le partite, con il risultato finale (win/lose) e il num. di tentativi

Parte 1: Definizione del Servizio

Il design dell'API segue i principi **REST**, mappando i requisiti ai concetti di **Risorse (sostantivi)** e **Operazioni (verbi HTTP)**

- **Risorsa collezione:** /games
 - Una collezione di partite
 - Usata per i requisiti 1 (POST) e 5 (GET)
- **Risorsa singola:** /games/{id}
 - Una singola partita, identificata da {id}
 - Usata per i requisiti 2, 3, 4

Struttura di base del documento di specifica **OpenAPI 3.0**

```
openapi: 3.0.0
info:
  title: Hi-Lo Game
  description: |
    This API allows playing hi-lo game
    and requesting the state of a game or the history
    of all the games
  version: 0.0.1
paths:
  /games:
    # Metodi GET e POST sulla collezione
    ...
  /games/{id}:
    # Metodi POST, GET, PATCH sulla singola risorsa
    ...
```

Parte 2: Metodi Aggiuntivi e Riutilizzo

Definizione Endpoint: /games

Metodo POST

Questo metodo implementa il requisito **1: Avviare una nuova partita**

- **Metodo:** POST su /games (crea una nuova risorsa partita)
- **Riassunto:** Start a new game
- **Descrizione:** Start a new game generating the secret number and return the created game id
- **Risposta 200 (OK):** la risposta deve contenere l'ID della partita
 - **Schema:** type: integer (ID della partita)

Lo **schema** definisce un **tipo primitivo** (integer, string), array o oggetto a seconda del campo **type**

```
post:
  summary: Start a new game
  description: |
    Start a new game generating the secret number and return the created game id
  responses:
    "200":
      description: new game created
      content:
        application/json:
          schema:
            type: integer
```

Metodo GET

Questo metodo implementa il requisito **5: Ottenere una lista di tutte le partite**

- **Metodo:** GET su /games (prende le partite giocate)
- **Riassunto:** List all game
- **Descrizione:** Obtain the list of all games, with the final result (win/lose) and the number of guesses
- **Risposta 200 (OK):**
 - **Schema:** type: array di oggetti
 - **Proprietà oggetto:**
 - **id:** type: integer
 - **outcome:** type: string (enum: win, lose)
 - **guesses:** type: integer (num. di tentativi)

```

get:
  summary: List all games
  description: |
    Obtain the list of all games, with the final result (win/lose) and the number
    of guesses
  responses:
    "200":
      description: |
        An array of object, each containing the game id, the final outcome, and
        the number of guesses
      content:
        application/json:
          schema:
            type: array
            items:
              type: object
              properties:
                id:
                  type: integer
                outcome:
                  type: string
                enum:
                  - win
                  - lose
                guesses:
                  type: integer

```

Definizione Endpoint: /games/{id}

Parametri

Dobbiamo definire il parametro **id** di tipo integer, necessario per il path.

Un parametro nel path è sempre obbligatorio

```

/games/{id}:
  # Metodi POST, GET, PATCH sulla singola risorsa
  parameters:
    - name: id
      in: path
      required: true
      description: this is the game id
      schema:
        type: integer
        description: e.g., /games/1234

```

Metodo PUT

Questo metodo implementa il requisito **3: Resetare una partita**, generando un nuovo id

- **Metodo:** PUT su /games/{id} (sostituisce la risorsa della partita precedente)
- **Riassunto:** Start or reset a game
- **Descrizione:** Start a game with gived id, generating the secret number; or reset an existing game, re-generating the secret number and zeroing the guess counter
- **Risposta 200 (OK):** reseta una partita
- **Risposta 201 (Created):** crea una nuova partita

```

put:
  summary: Start or reset a game
  description: |
    Start a game with gived id, generating the secret number; or reset an existing
    game, re-generating the secret number and zeroing the guess counter
  responses:
    "200":
      description: An existing game is reset
    "201":
      description: A new game is created

```

Metodo POST

Questo metodo implementa il requisito **2: Accettare un tentativo**. È l'interazione principale del gioco

- **Metodo:** POST su /games/{id} (crea una risorsa di risposta al tentativo)
- **Riassunto:** Make a guess
- **Descrizione:** Try to guess the secret number; return hi, lo or correct, plus the guess count
- **Parametri (path):** {id}
- **Corpo Richiesta (requestBody):**
 - **Proprietà:** guess-number (type: integer) (il numero tentato)
 - **Validazione:** minimum: 1, maximum: 100
- **Risposta 200 (OK):** Risultato del tentativo
 - **Proprietà:**
 - guess-count: type: integer (il contatore dei tentativi da 1 a 10)
 - guess-outcome: type: string (enum: hi, lo, correct)
- **Risposta 403 (Forbidden):** Gestisce il caso di Game Over (perche dopo il Game Over, l'utente non è più autorizzato a fare nuovi tentativi anche se potrebbe)
 - **Proprietà:** id, outcome (win/lose), guesses
- **Risposta 404 (Not Found):** Gestisce il caso di partita inesistente

```

post:
  summary: Make a guess
  description: |
    Try to guess the secret number; return hi, lo or correct, plus the guess count
  parameters:
    - name: guess
      in: query
      required: true
      description: the guess
      schema:
        type: integer
        minimum: 1
        maximum: 100
  responses:
    "200":
      description: Guess accepted, the result is in the content
      content:
        application/json:
          schema:
            type: object
            properties:
              guess-count:
                type: integer
                minimum: 1
                maximum: 10
              guess-outcome:
                type: string
                enum:
                  - hi
                  - lo
                  - correct
    "403":
      description: game over
      content:
        application/json:
          schema:
            type: object
            properties:
              id:
                type: integer
              outcome:
                type: string
                enum:
                  - win
                  - lose
              guesses:
                type: integer
    "404":
      description: game not found

```

Metodo GET

Questo metodo implementa il requisito **4: Ottenere la lista dei tentativi**

- **Metodo:** GET su /games/{id}
- **Riassunto:** List guesses
- **Descrizione:** Return a list of all guesses in reverse chronological order, including the responses received (hi/lo/correct)
- **Risposta 200 (OK):**
 - **Schema:** type: array di oggetti
 - **Proprietà Oggetto:**
 - guess-count: type: integer
 - guess-outcome: type: string (enum: hi, lo, correct)

```

get:
  summary: List guesses
  description: |
    Return a list of all guesses in reverse chronological order, including the
    responses received (hi/lo/correct)
  responses:
    "200":
      description: Request accepted, all guesses listed in content
      content:
        application/json:
          schema:
            type: array
            items:
              type: object
              properties:
                guess-count:
                  type: integer
                  minimum: 1
                  maximum: 10
                guess-outcome:
                  type: string
                  enum:
                    - hi
                    - lo
                    - correct

```


Parte 3: Riutilizzo e Schemi OpenAPI

Siccome alcuni oggetti JSON nelle risposte sono ricopiati più volte tra i vari metodi, possiamo sostituirli con un **riferimento a un Componente** definito centralmente. Questo ci permette di **ridurre**:

- La dimensione del codice
- La manutenzione (modifica una sola volta)
- Gli errori (coerenza garantita)

Il **nodo components** è la sezione in un documento OpenAPI dove si definiscono gli **oggetti riutilizzabili**

paths → components → schemas

Gli elementi definiti qui possono essere referenziati ovunque tramite **\$ref**

Esempio:

Possiamo creare un componente **guess-result** per riusare l'oggetto che usa guess-count e guess-outcome (usato in POST /games/{id} - codice 200 e GET /games/{id} - codice 200)

Stessa cosa possiamo farla per il **risultato finale di una partita** (usato in GET /games e POST /games/{id}/guesses - errore 403)

```
components:
  schemas:
    guess-result:
      type: object
      properties:
        guess-count:
          type: integer
          minimum: 1
          maximum: 10
        guess-outcome:
          type: string
          enum:
            - hi
            - lo
            - correct
    final-result:
      type: object
      properties:
        id:
          type: string
          enum:
            - win
            - lose
        guesses:
          type: integer
```

Ora possiamo chiamare i componenti guess-result e final-result con \$ref. Esempio:

```
get:
  summary: List guesses
  description: |
    Return a list of all guesses in reverse chronological order, including the
    responses received (hi/lo/correct)
  responses:
    "200":
      description: Request accepted, all guesses listed in content
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: "#/components/schemas/guess-result"
```

File YAML finale

<http://gamificationlab.uniroma1.it/notes/hilo-game-final.yaml>

Il progetto Nasoni include una app e un sito che permettono ai cittadini/turisti di esplorare le fontanelle vicine a loro (posizione e stato). Permette anche di cambiare stato o posizione di ogni fontanella, aggiungere fontanelle mancanti e rimuovere quelle che non sono più presenti.

Analisi e Schemi

Usando la sintassi YAML per OpenAPI, definiamo e documentiamo le risorse necessarie nel blocco **components**

Requisiti per lo schema principale della risorsa **"Fountain"** in components/schemas:

- 1) La risorsa deve avere un **campo ID (id)** di tipo intero, sola lettura (readOnly true) e con un esempio valido
- 2) Un campo **Stato (state)** che accetta solo due val: "good" e "faulty" (usando enum)
- 3) I campi **Latitudine (latitude)** e **Longitudine (Longitude)** entrambi di tipo number formato float. Bisogna applicare i **vincoli minimum e maximum appropriati** per le coordinate (Latitudine: ± 90 , Longitudine: ± 180)
- 4) Aggiungi **title e description** pertinenti per lo schema e ogni sua proprietà

Requisiti per il **componente riutilizzabile FountainID** in components/parameters (parametro usato nei path parameters):

- 1) Definire un parametro di tipo integer, chiamato **id**, con un campo description per chiarezza
- 2) Deve essere contrassegnato come readOnly: true (aiuta a mantenere la coerenza con la risorsa)

```
components:
  schemas:
    # Nuovo Schema per l'ID, ora riutilizzato in Fountain e FountainID Parameter
    FountainIdSchema:
      type: integer
      format: int64
      description: ID univoco assegnato dal server.
      readOnly: true
      example: 12345
    #...

  parameters:
    # Task 1.2: Definizione del Parametro "FountainID"
    FountainID:
      name: id
      in: path
      description: ID univoco della fontana (Nasone) nel percorso.
      required: true
      # USA IL RIFERIMENTO ALLO SCHEMA ID
      schema:
        $ref: '#/components/schemas/FountainIdSchema'
      # L'esempio del parametro è definito nello schema referenziato.
```

Definizione dei Path

Usando gli schemi e i parametri definiti in components, documenta tutti i percorsi (**path**) e le relative operazioni nel blocco **paths** come richiesto dalla traccia. Definizione del percorso **/fountains/** e le sue operazioni:

1) POST - Creazione di una nuova fontana:

- o **operationId**: createFountain
- o Deve accettare un corpo di richiesta JSON contenente lo schema Fountain (senza ID, come spiegato nella traccia)
- o Deve rispondere con **201 Created**, restituendo la risorsa Fountain **completa** (incluso l'ID assegnato dal server)

```
post:
  operationId: createFountain
  summary: Creazione di una Nuova Fontana
  description: Registra una nuova fontana nel sistema. L'ID viene assegnato dal server.
  requestBody:
    description: Dati della nuova fontana. L'ID non deve essere incluso.
    required: true
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Fountain'
  responses:
    '201':
      description: Fontana creata con successo
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Fountain'
    '400':
      $ref: '#/components/responses/BadRequest'
```

1) GET - Elenco delle fontane (con filtri di vicinanza):

- o **operationId**: listFountains
- o Deve includere tre parametri opzionali di tipo query: latitude, longitude e range (per trovare le fontane vicino)
- o Definiamo gli schemi per latitude e longitude come riferimenti a schemi locali per riutilizzo (es definire schemi separati Latitude e Longitude in components/schemas se non sono stati già definiti in precedenza)
- o Deve rispondere con **200 OK**, restituendo un array di oggetti Fountain

```
get:
  operationId: listFountains
  summary: Elenco delle Fontane (con Filtri di Vicinanza)
  description: Restituisce l'elenco delle fontane. Può essere filtrato per vicinanza a un punto specifico.
```

```

get:
  operationId: listFountains
  summary: Elenco delle Fontane (con Filtri di Vicinanza)
  description: Restituisce l'elenco delle fontane. Può essere filtrato per vicinanza a un punto specifico.
  parameters:
    - name: latitude
      in: query
      description: Latitudine del punto di riferimento per il calcolo della vicinanza.
      required: false
      schema:
        $ref: '#/components/schemas/Latitude'
    - name: longitude
      in: query
      description: Longitudine del punto di riferimento per il calcolo della vicinanza.
      required: false
      schema:
        $ref: '#/components/schemas/Longitude'
    - name: range
      in: query
      description: Raggio in metri (o unità definite) entro cui cercare le fontane (richiede latitude e longitude).
      required: false
      schema:
        $ref: '#/components/schemas/Range'
  responses:
    '200':
      description: Elenco delle fontane restituito con successo
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/Fountain'
    '400':
      $ref: '#/components/responses/BadRequest'

```

Definizione del percorso `/fountains/{id}` e le sue operazioni:

- 1) **PUT - Aggiornamento completato:** (aggiorna lo stato di una fontanella)
 - **operationId:** updateFountain
 - Deve usare il riferimento a FountainID per l'ID nel percorso
 - Deve accettare un corpo di richiesta JSON contenente lo schema Fountain (il client invia l'intera risorsa per sostituirla)
 - Deve rispondere con **200 OK**, restituendo la risorsa Fountain **aggiornata** (Altrimenti va bene 204 No Content, ma 200 è preferibile per un feedback)
 - Deve includere una risposta **404 Not Found** nel caso in cui la fontanella da aggiornare **non esista**
- 2) **DELETE - Eliminazione:**
 - **operationId:** deleteFountain
 - Deve usare il riferimento a FountainID per l'ID nel percorso
 - Non richiede un corpo di richiesta (deve solo eliminare una risorsa)
 - Deve rispondere con **204 No Content** in caso di successo
 - Altrimenti, se non si trova la fontanella da eliminare deve rispondere con **404 Not Found**

```

put:
  operationId: updateFountain
  summary: Aggiornamento Completo di una Fontana
  description: Sostituisce completamente la risorsa fontana con l'ID specificato con i dati forniti.
  requestBody:
    description: Dati completi della fontana per la sostituzione (l'intero oggetto).
    required: true
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Fountain'
  responses:
    '200':
      description: Risorsa fontana aggiornata e restituita.
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Fountain'
    '404':
      $ref: '#/components/responses/NotFound'
    '400':
      $ref: '#/components/responses/BadRequest'

delete:
  operationId: deleteFountain
  summary: Eliminazione di una Fontana
  description: Elimina la risorsa fontana con l'ID specificato.
  responses:
    '204':
      description: Fontana eliminata con successo (No Content)
    '404':
      $ref: '#/components/responses/NotFound'

```

API Finale

```
openapi: 3.0.0
info:
  description: Fontanelle app backend REST APIs
  title: Fontanelle APIs
  version: 1.0.0
servers:
  - url: http://localhost:3000

tags:
  - name: "List fountains"
    description: Endpoints for listing fountains
  - name: "Manage fountains"
    description: Endpoint for managing fountains

components:
  responses:
    BadRequest:
      description: |
        The request was not compliant with
        the documentation (eg. missing fields, etc).

    InternalServerError:
      description: |
        The server encountered an internal error.
        Further info in server logs.

  parameters:
    FountainID:
      schema:
        type: integer
        example: 1
        readOnly: true
      name: id
      in: path
      required: true
      description: Fountain ID

  schemas:
    Latitude:
      description: Location latitude in decimal degrees format.
      type: number
      format: float
      example: 12.34
      minimum: -90
      maximum: 90

    Longitude:
      description: Location longitude in decimal degrees format.
      type: number
      format: float
      example: 56.78
      minimum: -180
      maximum: 180

    Fountain:
      title: Fountain
      description: "This object represent a single Fountain (\\"nasone\\")."
      type: object
      properties:
        id:
          description: Unique fountain identifier.
          type: integer
          example: 1
          readOnly: true
        status:
          description: |-
            Describe the status of the fountain.
```

```

    readOnly: true
    status:
      description: |-
        Describe the status of the fountain.
        If it's "good", the fountain is in working condition;
        if it's "faulty", the fountain is broken.
      type: string
      enum: ["good", "faulty"]
      example: good
    latitude: { $ref: "#/components/schemas/Latitude" }
    longitude: { $ref: "#/components/schemas/Longitude" }

paths:
  /fountains/:
    get:
      Try it
      tags: ["List fountains"]
      operationId: listFountains
      summary: Get the list of available fountains
      description: |-
        Return the list of all fountains in the system.
        If the client specifies a set of coordinates for a point,
        the list will be sorted by distance from that point
        and filtered for a range.
        Optionally, the client can specify a custom range.
      parameters:
        - name: latitude
          in: query
          required: false
          description: Latitude for sorting/finding fountains.
          schema: { $ref: "#/components/schemas/Latitude" }
        - name: longitude
          in: query
          required: false
          description: Longitude for sorting/finding fountains.
          schema: { $ref: "#/components/schemas/Longitude" }
        - name: range
          in: query
          required: false
          description: |-
            Range for the location filter. Default: 10.
            If coordinates are not specified, this parameter is ignored.
          schema:
            type: number
            format: float
            example: 12
            minimum: 1
            maximum: 200
      responses:
        "200":
          description: List of fountains
          content:
            application/json:
              schema:
                description: List of fountains that matches the request
                type: array
                items: { $ref: "#/components/schemas/Fountain" }
                minItems: 0
                maxItems: 1000
        "400": { $ref: "#/components/responses/BadRequest" }
        "500": { $ref: "#/components/responses/InternalServerError" }
    post:
      Try it
      tags: ["Manage fountains"]
      operationId: createFountain
      summary: Create a new fountain
      description: |-
        Create a new fountain using the properties in the request body.
        The server will create a new unique ID, the client can find it
        in the response.
      requestBody:

```

```

    Create a new fountain using the properties in the request body.
    The server will create a new unique ID, the client can find it
    in the response.
  requestBody:
    content:
      application/json:
        schema: { $ref: "#/components/schemas/Fountain" }
  responses:
    "201":
      description: Fountain created successfully
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Fountain" }
    "400": { $ref: "#/components/responses/BadRequest" }
    "500": { $ref: "#/components/responses/InternalServerError" }

/fountains/{id}:
  parameters:
    - $ref: "#/components/parameters/FountainID"
  put:
    Try it
    tags: ["Manage fountains"]
    operationId: updateFountain
    summary: Update fountain properties
    description: |-
      Replaces all fountains properties with the Fountain specified
      in the body of the request.
    requestBody:
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Fountain" }
    responses:
      "200":
        description: Fountain updated successfully
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Fountain" }
      "400": { $ref: "#/components/responses/BadRequest" }
      "404": { description: Fountain not found }
      "500": { $ref: "#/components/responses/InternalServerError" }
  delete:
    Try it
    tags: ["Manage fountains"]
    operationId: deleteFountain
    summary: Remove a fountain
    description: Remove a fountain from the system.
    responses:
      "204": { description: Fountain deleted successfully }
      "400": { $ref: "#/components/responses/BadRequest" }
      "404": { description: Fountain not found }
      "500": { $ref: "#/components/responses/InternalServerError" }

```


Comandi Principali Go

giovedì 27 novembre 2025 16:56

Go è un linguaggio di programmazione **open source** progettato per essere facile da imparare, leggibile e produttivo offrendo comunque ottime funzionalità Gestisce automaticamente l'allocazione di memoria e il supporto integrato per la concorrenza e una libreria standard robusta e un sistema di gestione di dipendenze

Pacchetti (Packages)

I file nella **stessa directory** fanno parte dello stesso **pacchetto** che deve essere dichiarato all'inizio dei vari file nel pacchetto (es: package main)

Il pacchetto **main** è importante perché il compiler di Go si aspetta di trovarci la funzione iniziale **main()**, che è la prima che viene eseguita

All'interno dello stesso pacchetto si può usare qualsiasi funzione/variabile globale (anche quelle **private**) **indipendentemente dal file che la implementa**, come se fosse un singolo file

Pero quando si fa riferimento a **pacchetti esterni** bisogna **importarli** appena dopo la linea del package

es **import "fmt"** ci permette di usare qualsiasi funzione/struttura/interfaccia/variabile globale **pubblica** del pacchetto **fmt** **prefissando il nome della funzione dopo il nome del pacchetto**, ad es **fmt.Println()**

Per definire una funzione/struttura/interfaccia/variabile globale **pubblica** essa deve avere il **primo carattere del nome in maiuscolo**

Es **Println()** è pubblica ma **println()** non lo è

Esempio stampa

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

Cheat Codes

Si possono **importare più pacchetti insieme** usando un solo import e inserendo tra parentesi i vari pacchetti da importare

Questo funziona anche per le **variabili multiple**

In più è possibile specificare il tipo per più **variabili insieme** scrivendo le variabili separate da una virgola e alla fine il loro tipo

```
import ( // import multipli di pacchetti
    "fmt"
    "complex"
)

var ( // inizializzazione di più variabili
    toBe bool = false
    MaxInt uint64 = 1 << 64 - 1
    z complex128 = complex128(complex.Sqrt(-5+12i))
)

x, y, z int // specifica del tipo per più variabili
```

Moduli

Un **modulo** è una collezione di **uno o più pacchetti**

Viene usato per la **gestione delle dipendenze** e controllo di versione e viene definito dal file **go.mod**

Si può **importare** usando il percorso completo (nome-modulo/pacchetto)

Per **creare pacchetti** all'interno del progetto:

- 1) Nella directory del progetto **go mod init nome-modulo** crea la radice del progetto
- 2) Dopo si possono creare i **sotto-moduli** (sotto-directory) per i pacchetti
- 3) Si possono importare col percorso completo: nome-modulo/sotto-dir

Es:

```
main.go
go.mod
go.sum
package1/
    functions.go
    other-things.go
subpackage/
    file.go
```

Contenuto package1/functions.go

```
package package1

// Nota: Se non si usa la lettera MAIUSCOLA come prima lettera
// allora la funzione sarà disponibile solo all'interno di package1
func Dummy() {} // funzione pubblica

func internalFunction() {} // funzione privata (non esportata)
```

Contenuto di main.go

```
package main
import "nome-modulo/package1" // importa il pacchetto interno

func main() {
    package1.Dummy() // chiama la funzione pubblica
    package1.internalFunction() // Errore: non è esportata
}
```

Moduli Esterni

Per ottenere **moduli esterni** bisogna prima scaricarli con **go get nome-package** che scarica il pacchetto e lo aggiunge a go.mod per poterlo poi importare

Creazione Progetto Go

Creazione Modulo

Il comando **go mod** è fondamentale per inizializzare e gestire il progetto

Comando	Descrizione
go mod init <path/nome>	Inizializza un nuovo modulo Go. Crea il file go.mod nella dir corrente, definendo il percorso radice del modulo
go mod tidy	Pulisce e Aggiorna. Aggiunge eventuali dipendenze mancanti (trovate nel codice) e rimuove quelle inutilizzate dal file go.mod e go.sum
go mod download	Scarica tutti i moduli richiesti in go.mod nella cache locale

Struttura Standard (Layout)

La convenzione standard di Go è:

- **File sorgente:** i file Go finiscono con .go
- **Package main:** contiene la funzione main() che è il punto di partenza per un programma eseguibile
- **Package (Nome personalizzato):** contiene librerie e logica di business riutilizzabile

Es con linux:

- 1) **Crea la cartella:** mkdir myproject && cd myproject
- 2) **Inizializza il modulo:** go mod init example.com/myproject
- 3) **Crea il file principale:** touch main.go

Toolchain

Esecuzione Rapida

Per eseguire il codice senza creare un eseguibile finale:

Comando	Descrizione
go run main.go	Compila ed esegue il codice sorgente specificato. Utile per testare rapidamente durante lo sviluppo
go run .	Compila ed esegue il package main nella directory corrente

Compilazione e Build

Per creare un eseguibile da distribuire

Comando	Descrizione
go build .	Compila il package corrente e genera un file binario nella dir corrente, chiamato come il modulo/directory
go build -o app_name .	Compila e specifica il nome del file di output (es app_name)
go install .	Compila e installa l'eseguibile nella cartella binari predefinita di Go (\$GOPATH/bin o \$GOBIN)

Test

Framework di testing integrato

Comando	Descrizione
go test	Esegue i test (file con suffisso _test.go) nella dir corrente
go test ./...	Esegue i test in tutte le sottodirectory
go test -v	Esegue i test in modalità verbosa (mostra i dettagli di ogni test)

Formattazione Codice

Go impone uno stile di **formattazione standardizzato**

Comando	Descrizione
go fmt .	Riformatta automaticamente i file Go nella dir corrente in modo standardizzato, usando lo stile canonico di Go

Analisi e Linting (Controllo Qualità)

Strumenti per l'analisi statica del codice

Comando	Descrizione
go vet .	Analizza il codice sorgente per identificare, potenziali bug o costrutti sospetti (es errori di formattazione o assegnazioni inutili)
go doc <package/func>	Mostra la documentazione per un pacchetto o una funzione specifica (es go doc fmt.Println)

Altri Strumenti Utili

Gestione e Ispezione dell'Ambiente

Comando	Descrizione
go version	Mostra la versione di Go installata
go env	Stampa le variabili d'ambiente di Go (es \$GOPATH, \$GOBIN)

Gestione Dipendenze Esterne

Comandi per gestire le dipendenze esterne richieste dal codice

Comando	Descrizione
go get <path/package>	Aggiunge un nuovo pacchetto esterno (o aggiorna una esistente) al file go.mod e lo scarica
go get -u ./...	Aggiorna tutte le dipendenze del modulo alla versione patch o minor più recente
go get -u=patch ./...	Aggiorna solo alle versioni patch più recenti (più sicuro)

Ispezione e Blocco (Go.sum)

Il file go.sum contiene gli **hash crittografici** per le dipendenze, garantendo integrità

Comando	Descrizione
go list -m all	Elenca tutte le dipendenze del modulo, incluse quelle transitive (di cui dipendono le tue dipendenze)
go mod vendor	Crea una cartella vendor/ contenente le copie locali di tutte le dipendenze, per build isolate o per reti limitate
go mod verify	Verifica che i moduli scaricati nella cache corrispondano agli hash in go.sum

Pulizia

Per mantenere pulito l'ambiente del modulo

<i>Comando</i>	<i>Descrizione</i>
go clean -modcache	Cancella la cache dei moduli (\$GOPATH/pkg/mod) forzando il download di tutte le dipendenze alla prossima build/run

Variabili e Strutture

venerdì 5 dicembre 2025 14:06

Variabili

Tipi di dati di base:

- **bool**
- **string**
- **int**: interi segnati
- **int8, int16, int32, int64**: interi segnati di grandezza esplicita
- **uint**: interi non segnati (solo val. positivo o zero)
- **uint8, uint16, uint32, uint64**
- **uintptr**: puntatore
- **byte**: tipo a 8-bit e rappresenta singolo byte (alias di uint8)
- **rune**: tipo a 32-bit e rappresenta un codepoint Unicode (tipo **char** in C)
- **float32, float64**
- **complex64, complex128**: numeri complessi di grandezza esplicita

int, uint e uintptr dipendono dalla grandezza della piattaforma (es sono a 64 bit sui sistemi a 64 bit)

La sintassi di Go permette:

- **Inferire** il tipo delle variabili **senza dichiararne il tipo**
- **Conversione del tipo (type casting)** è possibile prefissando il valore tra parentesi col tipo di variabile (es: int64(5))
- **Dichiarazione multipla** di variabili sulla stessa linea quando sono dello stesso tipo.
- Dichiarare una variabile **senza specificare un tipo esplicito** usando "!=" assegnandogli direttamente un valore e inferendo il tipo, senza dover usare **var**
- Dichiarare **costanti** usando la parola chiave **const** invece di **var**

```
var i = 3 // tipo inferito
var i int = 3 // tipo definito

var testo = "questa è una stringa"
var check = true // var booleana

// dichiaro una var come int64 e gli do valore 5
var x = int64(5)

// dichiaro una var con tipo inferito intero
var y = 10

// uso la conversione del tipo per allineare i tipi
var sum = x + int64(y)

// dichiaro due contatori di tipo int
var cont1, cont2 int

// dichiaro una variabile assegnandogli un valore
// il tipo viene inferito
valore := 100 // tipo int

const Pi = 3.14 // costante di tipo float
```

Tipo Rune

Avviando

```
package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    s := "Ciao, Mondo! 🌍" // contiene un emoji
    fmt.Println("Len (Byte):", len(s))
    fmt.Println("Rune Count:", utf8.RuneCountInString(s))
}
```

Vediamo che:

- **Len (Byte) = 17**
- **Rune Count = 14**

Questo perché l'emoji è un Rune (di valore U+1F30D) che viene codificato in UTF-8 usando **4 byte** (0xF0, 0x9F, 0x8C, 0x8D)

Byte per Rune (in UTF-8)	Intervallo Code Point (Hex)	Esempio carattere	Descrizione
1 Byte	U+0000 a U+007F	A, 1, !	Caratteri ASCII di base (inglese)
2 Byte	U+0080 a U+07FF	è, ñ, £	Caratteri latini estesi (accenti, tilde), valute
3 Byte	U+0800 a U+FFFF	字, π, €	Caratteri CJK (cinese, giapponese, coreano), simboli matematici, maggior parte delle valute
4 Byte	U+10000 a U+10FFFF	👉, 🌐, ✓	Emoticon, pittogrammi, caratteri rari

Cicli for

In Go esiste un solo tipo di **ciclo**: **for**

Non serve definire i controlli all'interno delle parentesi e le istruzioni da eseguire nel ciclo vanno inserite tra le parentesi graffe

L'**inizio dell'intervallo** (init), la **condizione da verificare** e il **passo di avanzamento** (post) sono **opzionali**.

Lasciando solo la condizione da verificare avremmo un ciclo **while**, mentre togliendo tutte le opzioni del ciclo avremmo un ciclo infinito

```
// due for che danno lo stesso output
sum := 0
for i := 0; i<10; i++ { // semplice ciclo da 0 a 10 (non incluso)
    sum += i
}

sum = 0
i := 0 // inizio
for {
    if i >= 10 {break} // condizione
    sum += i
    i++ // passo
}

// ciclo while
sum = 1
for sum < 1000 {
    sum += sum
}

// ciclo infinito
for {
}
```

Controllo if

Come negli altri linguaggi di programmazione usiamo if per controllare delle condizioni

Come per il for non serve inserire la condizione tra parentesi e bisogna usare le parentesi graffe per racchiudere le istruzioni da eseguire

if supporta la dichiarazioni di istruzioni eseguite **prima della condizione (dichiarazione breve)** e queste variabili sono **visibili** solo **all'interno** del blocco if/else

```
func timeParity() {
    if p := time.Now().Second(); p%2 == 0 { // p è dichiarata e inizializzata qui
        fmt.Println(p, "è Pari")
    } else {
        fmt.Println(p, "è Dispari")
    }
    // p non è visibile qui, dopo i blocchi if/else
}
```

Switch

Similmente agli altri linguaggi di programmazione lo switch valuta i **case dall'alto verso il basso** e esegue il **primo case** che combacia con la condizione dello switch

Però rispetto ad altri linguaggi, ogni case **termina automaticamente** con un break implicito, quindi i case successivi non vengono eseguiti

```
func randomFloor() {
    var floor = rand.Intn(5) // numero random da 0 a 4
    switch floor {
    case 0:
        fmt.Println("Piano terra")
    case 1:
        fmt.Println("Primo piano")
    case 2:
        fmt.Println("Secondo piano")
    default:
        fmt.Println("Piani successivi")
    }
}
```

Defer (Posticipa)

La parola chiave **defer** è usata per **posticipare l'esecuzione** di una funzione fino a quando la funzione circostante non ritorna. Si possono usare insieme istruzioni **defer multiple** che verranno eseguite nell'ordine **LIFO** (l'ultima posticipata viene eseguita per prima).

```
func test() {
    defer fmt.Println("Quarta") // (4) eseguita prima del ritorno di test()
    fmt.Println("Terza") // (3) eseguita immediatamente
}

func main() {
    defer fmt.Println("Quinta") // (5) eseguita per ultima (LIFO) (3° defer eseguito in main)
    fmt.Println("Prima") // (1) eseguita immediatamente
    defer test() // (3 e 4) eseguita per penultima (2° defer eseguito in main)
    defer fmt.Println("Seconda") // (2) eseguita per secondultima (1° defer eseguito in main)
}

func ordineLifo() {
    // per ordine LIFO
    defer fmt.Println("Quarta")
    defer fmt.Println("Terza")
    defer fmt.Println("Seconda")
    defer fmt.Println("Prima")
}
```

Puntatori

I **puntatori** sono variabili che memorizzano l'**indirizzo di memoria** di un valore. Per dichiarare e usare un puntatore si usa l'asterisco prima di fronte alla variabile.

Es: *T è un puntatore a un valore di tipo T

Il valore di **default** di un puntatore è **nil** (assenza di valore).

Si usa:

- **Operatore di indirizzo &** per generare un puntatore a una variabile
- **Operatore di deferenziazione *** per leggere o usare il valore a cui punta

```
var original int = 42 // var di tipo intero
var pointer *int // puntatore a una var intera
pointer = &original // assegna l'indirizzo di "original" al puntatore "pointer"
// si può usare anche direttamente "pointer := &original" invece delle 2 righe precedenti
fmt.Println(original) // stampa il valore originale
fmt.Println(pointer) // stampa l'indirizzo di memoria di "original"
fmt.Println(*pointer) // stampa il valore di "original" tramite il puntatore
*pointer = 21 // imposta "original" a 21 tramite il puntatore
fmt.Println(*pointer) // stampa il nuovo valore 21
```

Struct (Strutture)

La parola chiave **struct** si usa per creare delle **strutture**, cioè una collezione di campi che possono essere di tipo diverso.

L'accesso ai campi si esegue tramite il punto tra il nome della struttura e il campo (es: rubrica.Numero).

L'accesso tramite puntatore (*p).campo può essere abbreviato in p.campo.

È possibile inizializzare un sottoinsieme di campi specificandoli con **NomeCampo: Valore**.

Le struct possono essere dichiarate anche **anonimamente**, cioè senza un nome di tipo esplicito se devono essere usate **solo localmente**.

```

type Vertex struct {
    X, Y int
}
func main() {
    var (
        v1 = Vertex{1, 2} // ha tipo Vertex
        v2 = Vertex{X: 1} // Y:0 è implicito
        v3 = Vertex{} // X:0 e Y:0 sono impliciti
        p = &Vertex{3, 2} // puntatore a Vertex
    )
    a := struct { // struttura anonima
        i int
        b bool
    }{1, true} // inizializzata sul momento
    if v1.Y == p.Y && a.b{
        // siccome v1.Y e p.Y hanno entrambi val 1 e a.b è true
        fmt.Println("Vero") // viene stampato "Vero"
    } else {
        fmt.Println("Falso")
    }
}

```

Array e Mappe

venerdì 5 dicembre 2025 12:50

Array e Slices

Go nativamente supporta due tipi di strutture dati per una lista ordinata di elementi dello stesso tipo: **array** e **slices**

La differenza principale è che l'array ha la **grandezza definita** al momento di creazione, mentre gli slices possono essere **estesi**

Array

Lista di **dimensione fissa**

Usiamo **[n]T** per definire un array di **n valori di tipo T** (es var arr [10]int)

Si possono inizializzare gli elementi direttamente alla creazione dell'array usando le parentesi graffe per racchiudere i vari valori da inserire

Non è necessario riempire di valori l'array poiché i valori non specificati vengono inizializzati al valore di default

```
voti := [6]int{18, 25, 23, 29} // voti = [18, 25, 23, 29, 0, 0]
fmt.Println(voti[2]) // stamperà 23
fmt.Println(voti[4]) // stamperà 0 (val di default del tipo int)
fmt.Println(len(voti)) // stamperà 6
fmt.Println(voti[6]) // darà errore "out of bounds"
```

Slices

Una **slice** è una lista di **dimensione variabile**.

Uno slice è una **astrazione di un array** (fa riferimento a una sezione di un array) e per aggiungere un nuovo elemento viene **rimpiazzato l'array sottostante** con un altro più grande.

Essa ha 3 componenti:

- 1) **Puntatore**: l'indirizzo del primo elemento referenziato **nell'array sottostante**
- 2) **Lunghezza (len(s))**: numero di elementi che contiene
- 3) **Capacità (cap(s))**: numero di elementi che può contenere prima di essere ridimensionata

Quindi la lunghezza è il num. di elementi nella slice e la capacità è il num. di elementi nell'array sottostante

La sintassi a[low: high] indica l'intervallo semi-aperto da **low incluso** a **high escluso**

Es:

```
a := [5]int{10, 20, 30, 40, 50}
```

```
s := a[1:4]
```

- **Array a:**

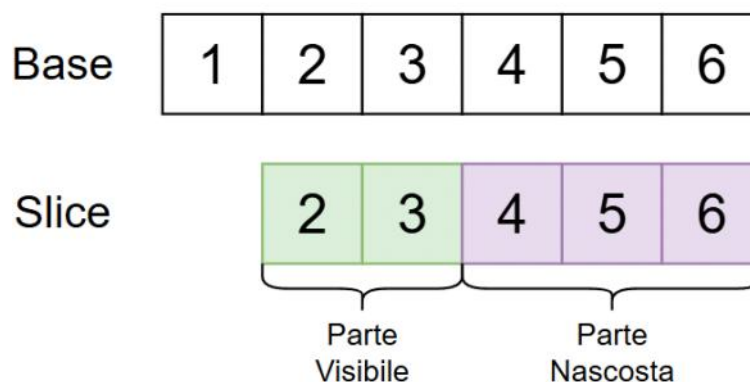
- Val: [10, 20, 30, 40, 50]
- Len: 5

- **Slice s:**

- Val: [20, 30, 40]
- Puntatore: &a[1]
- Len = 3
- Cap = 4 (da a[1] fino a a[4])

Quindi lo slice **non memorizza dati**, ma modifica l'array originale e si possono avere **più slice** che puntano allo **stesso array**

```
Base := [6]int{1, 2, 3, 4, 5, 6}
Slice = Base[1:3] // len = 2, cap = 5
```



Si può estendere uno slice (`s = s[:nuova_len]`) solo fino al **val max** della sua cap

Lo slice si definisce con **[]T** (senza specificare le dimensioni)

Di default ha `len=0` e `cap=0` se non si specificano le dimensioni/inizializza lo slice

Quando si crea uno slice viene creato un array e poi viene costruito uno slice che lo referencia

Es: `[]bool{true, true, false}`

È possibile **estendere** una slice **solo verso destra** (aumentando la sua lunghezza) **fino** alla **capacità massima**

Non si può estendere verso **sinistra** perché non si può cambiare l'indice di partenza e si può indicizzare **oltre la len** se si effettua un **re-slicing**

Si può controllare la capacità iniziale di uno slice usando **make()**, una funzione che accetta 3 argomenti:

- Il tipo dello slice
- La lunghezza iniziale (i valori assumeranno il val. di default) (se la lunghezza è 0, lo slice sarà vuoto)
- La capacità (se omessa la capacità sarà uguale alla lunghezza data)

Con **append(slice, serie di val)** invece possiamo aggiungere nuovi valori alla slice (devono essere del tipo definito per la slice) e **allocare un array più grande** se la capacità è esaurita.

```
s := []int{2, 3, 5, 7, 11, 13} // len = 6, cap = 6
// affetta la slice per darle lunghezza 0
s = s[:0] // s = [], len = 0, cap = 6
// estendiamo nuovamente la sua lunghezza
s = s[:4] // s = [2, 3, 5, 7], len = 4, cap = 6
fmt.Println(s[4]) // Errore: indice 4 fuori dalla lunghezza 4 (arriva da 0 a 3)
s = s[:cap(s)] // crea una nuova slice grande quanto la capacità totale (len = 6)
fmt.Println(s[4]) // non dà più errore perché ora s = [2, 3, 5, 7, 11, 13]

s = append(s, 17) // s = [2, 3, 5, 7, 11, 13, 17], len = 6, cap = 12

a := make([]int, 3, 5) // a = [0, 0, 0, 0, 0], len = 3, cap = 5
b := make([]int, 5) // a = [0, 0, 0, 0, 0], len = cap = 5
c := make([]int, 0, 5) // a = [], len = 0, cap = 5 (zero elementi)
```

Domanda Append:

```
s := []int{9000}[:0]
s = append(s, 0)
```

Inizialmente s è [9000] (len = 1, cap = 1). s[:0] riduce la grandezza a [] (len = 0, cap = 1) e append(s, 0) usa la capacità esistente per inserire l'elemento 0 e restituire una slice [0] (len = 1, cap = 1)

Esempio **condivisione di array tra slice**:

```
Base := [8]int{10, 20, 30, 40, 50, 60, 70, 80}
A := Base[1:5] // A = [20, 30, 40, 50], len = 4, cap = 7
B := Base[3:6] // B = [40, 50, 60], len = 4, cap = 7
// A e B puntano allo stesso array

A[2] = 99 // modifica l'array a cui puntano A e B
// Base = [10, 20, 30, 99, 50, 60, 70, 80]
// A = [20, 30, 99, 50]
// B = [99, 50, 60]
// La modifica viene effettuata su entrambi gli slice

A = append(A, 100, 110, 120, 130) // aggiunta di nuovi 4 elementi
// Siccome A ha len = 4 e cap = 7, non abbiamo spazio nell'array sottostante
// L'append CREA un nuovo array per A
// B siccome è ancora collegata alla Base rimarrà invariata rispetto ad A

// Base = [10, 20, 30, 99, 50, 60, 70, 80]
// A = [20, 30, 40, 50, 100, 110, 120, 130] len = 8, cap = 16
// B = [99, 50, 60] len = 3, cap = 7
```

Mappe (o dizionari)

Le **mappe** possono contenere **valori** indicizzati da una **chiave** (coppia **chiave-valore**)

Il suo valore di default di una mappa non definita è **nil** (e non si possono aggiungere chiavi ad una mappa nil)

Per inizializzare una mappa si deve usare la funzione **make(map[tipo_chiave]tipo_val)**

Si possono anche inizializzare usando i letterali, cioè definendo le coppie chiave-valore all'inizializzazione della mappa

Modifica sulle mappe:

- **Inserimento/aggiornamento:** m[key] = val
- **Lettura:** val := m[key]
- **Cancellazione:** delete(m, key)
- **Test di presenza (due val):** val, ok := m[key]
 - **ok è true** se la chiave è **presente**, altrimenti è **false**
 - **val** prende il **val** associato alla chiave, altrimenti prende il **val default** del tipo se la **chiave non è presente**

```
var dict map[string]int // mappa dichiarata ma non allocata (nil)
// aggiungere un val risulta in crash

dict = make(map[string]int)
dict["roma"] = 42 // dict = map[roma:42]
fmt.Println(dict["roma"]) // stampa "42"

dict = map[string]int{ // inizializzazione letterale
    "roma": 37,
    "milano": 40,
} // dict = map[roma:37 milano:40]
val, check := dict["roma"] // val = 37, check = true
delete(dict, "roma") // dict = map[milano:40]
val, check = dict["roma"] // val = 0, check = false
```

Range

La parola chiave **range** è una forma del ciclo **for** per iterare sugli array, slice, stringhe o mappe

Restituisce **due valori** per l'iterazione: L'**indice** e la **copia dell'elemento** a quell'indice (opzionale) (se si itera una mappa: indice = chiave, elemento = valore)


```
slice := []int{2, 3, 5, 7, 11}

for i, val := range slice {
    // i è l'indice, val è la copia dell'elemento
}
for i, _ := range slice {
    // per usare solo l'indice e ignorare l'elemento (obbligatorio "_")
}
for i := range slice {
    // forma abbreviata per usare solo l'indice
}
for _, val := range slice {
    // per ignorare l'indice e usare solo il valore
}
```

Funzioni, Metodi, Interfacce ed Errori

lunedì 1 dicembre 2025 12:58

Funzioni

Le **funzioni** vengono definite usando la parola chiave **func** seguita dal **nome** della funzione, i **parametri** (chiusi tra parentesi), definiti usando il nome del parametro seguito dal tipo, e i **valori di ritorno** (anche multipli)

I valori di ritorno possono essere definiti in due modi:

- **Non-nominati**: si stabilisce una **lista dei tipi** dei valori di ritorno e Go si aspetta lo stesso ordine per ogni **return**
- **Nominati**: si definiscono direttamente i **nomi** per ogni variabile di ritorno e queste variabili sono ritornate quando un **return vuoto** viene incontrato

Le variabili passate alle funzione vengono **copiate** per i tipi base (int, string, struct) mentre per i **puntatori** viene passata la **copia dell'indirizzo di memoria** (*T), quindi nella funzione è possibile **modificare il val originale** tramite l'indirizzo passato.

Stessa cosa per le **slice**, **map** e **channel**, dove la loro **struttura è copiata** ma il dato sottostante a cui fanno **riferimento** è **condiviso**

Possono essere passate anche le **funzioni stesse** come valori

```
func divide1(x int, y int) (int, int) { // return non-nominato
    div = x / y
    mod = x % y
    return div, mod
}

func divide2(x int, y int) (div int, mod int) { // return nominato
    div = x / y
    mod = x % y
    return // restituisce automaticamente div e mod
}

func useBase(v int){
    a += 5 // modifica solo la COPIA locale
}

func usePointer(v *int) { // passaggio di un puntatore
    *a += 5 // dereferenzia e modifica il valore ORIGINALE
}

func useFunc(fn func(float64, float64) float64) float64 {
    // fn = funzione che prende 2 float e ritorna un float
    // useFunc = funzione che prende una funzione e ritorna un float
    return fn(3, 4)
}

func main() {
    fmt.Println(divide1(7, 3)) // stampa "2 1"
    fmt.Println(divide2(7, 3)) // stampa "2 1"

    a := 5
    useBase(a) // dopo a sarà ancora 5
    modifyPointer(&a) // dopo a sarà 10 (modifica col puntatore)

    fmt.Println(useFunc(math.Pow)) // stampa 81 (cioè Pow(3, 4))
}
```

Closure

Una **closure** è una funzione che permettono il riferimento a variabili dall'esterno del suo corpo anche dopo che la funzione è finita

Ogni istanza di closure è **legata alla propria copia** delle variabili esterne, quindi se si creano più oggetti della stessa closure le variabili interne alla closure saranno **distinte tra i vari oggetti**

```
func adder() func(int) int { // adder restituisce una closure
    sum := 0 // variabile "catturata" dalla closure
    return func(x int) int {
        sum += x // la closure modifica sum anche dopo l'uscita da adder()
        return sum
    }
}

func main() {
    pos, neg := adder(), adder() // due closure diverse, ognuna con la propria "sum"
    for i := 0; i < 10; i++ {
        fmt.Println(
            pos(i), // accumula i
            neg(-2*i), // accumula -2*i indipendentemente da pos
        )
    }
}
```

Metodi

I **metodi** sono funzioni **associate** con un tipo personalizzato o **uno struct** e che prendono come argomento un **receiver** (ricevente) speciale su cui operano. Le strutture passate **vengono copiate** nei metodi quindi ogni modifica alla struttura passata **non** andrà a **modificare** la struttura **originale**. Se vengono definite con **puntatori** si possono anche **modificare i valori della struttura originale**.

I metodi possono essere **definiti su qualsiasi tipo** (tranne puntatori e interfacce).

```
type Rubrica struct {
    Indirizzo string
}

// Receiver per Valore: opera sulla COPIA della struct
func (item Rubrica) HaIndirizzo() bool {
    return item.Indirizzo != "" // ritorna true se ha l'indirizzo
}

// Receiver per Puntatore: opera sulla struct ORIGINALE
func (item *Rubrica) CancellaIndirizzo() {
    *item.Indirizzo = "" // andrà a modificare il valore dell'indirizzo
}

func main() {
    rubr := Rubrica{"Via del Corso"}
    fmt.Println(rubr.HaIndirizzo()) // ritorna true
    rubr.CancellaIndirizzo() // modifica il valore nella struct
    fmt.Println(rubr.HaIndirizzo()) // ritorna false
}
```

Interfacce

Un'interfaccia definisce un **insieme di firme di metodi** senza provvedere l'implementazione di questi metodi.

Invece definisce la specifica del comportamento che un tipo concreto deve implementare.

Un tipo implementa un'interfaccia se implementa **tutti i metodi** definiti nell'interfaccia.

Rispetto ad altri linguaggi l'implementazione dell'interfaccia è **implicita** (non è richiesto "implements"), basta solo usare i metodi dell'interfaccia per implementarla.

```
type Shape interface { // interfaccia che implementa 2 metodi
    Area() float64
    Perimeter() float64
    // ogni tipo che usa questa interfaccia deve implementare questi 2 metodi
}

type Circle struct { // tipo che implementa l'interfaccia Shape
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.Radius
}

func main() {
    var s Shape
    s = Circle{Radius: 5}

    fmt.Println("Area: ", s.Area())
    fmt.Println("Perimetro: ", s.Perimeter())
}
```

Le interfacce consentono il **polimorfismo** e il **disaccoppiamento**, permettendo a funzioni come `fmt.Fprintf` di lavorare con qualsiasi `Writer`.

Implementazioni di Interfacce Comuni

- **io.Writer**: fondamentale per l'I/O
- **os.Stdout**: standard output della console
- **bytes.Buffer**: un buffer di byte in memoria
- **os.File**: qualsiasi file su disco
- **net.Conn**: una connessione di rete (socket)

Composizione Vs Ereditarietà

Go non ha l'ereditarietà di classe ma favorisce il riuso del codice tramite **composizione (embedding)**. Cioè si incorpora una struct all'interno di un'altra:

- **Embedding**: inserire una struct **senza nome di campo**
- **Promozione**: i metodi della struct interna sono **promossi** alla struct esterna
- Simula il riutilizzo **senza** ereditarietà di tipo

```

type Logger struct {
    LogLevel string
}

func (l logger) Log(msg string) {
    // logica di logging
}

// Server "ha" un Logger (composizione)
type Server struct {
    Logger // Embedding
    Host string
}

func main() {
    s := Server(Logger: Logger{LogLevel: "INFO"})

    // Il metodo Log è ora accessibile direttamente da "s"
    s.Log("Avvio del server")
}

```

Gestione degli Errori

Go usa **valori di ritorno** rappresentati dal tipo **error**, un'interfaccia col singolo metodo **Error() string** che ritorna una stringa che rappresenta il **messaggio di errore**. Quando una funzione fallisce, ritorna **error** come **ultimo valore**.

Si possono anche **creare errori** custom usando la funzione **errors.New("messaggio di errore")**.

Un errore può un **tipo specifico** (es una struct o una var error) che implementa l'interfaccia error e indica la **causa/natura** del fallimento (es. *os.PathError).

Possiamo eseguire anche dei **controlli logici** specifici come l'**uguaglianza** con **errors.Is** o la **conversione** con **error.As**.

```

var ErrInvalidInput = errors.New("input non valido")

func Process(data string) (string, error) {
    if data == "" {
        // restituisce il valore specifico ErrInvalidInput
        return data, ErrInvalidInput
        // oppure potevamo fare direttamente
        // return data, errors.New("input non valido")
    }
    // se l'input è valido continuiamo col processo
    return data, nil
}

func main() {
    result, err := Process("")

    // controlla subito se c'è stato un errore
    // controllo sul TIPO (errore specifico)
    if errors.Is(err, ErrInvalidInput) {
        fmt.Println("Errore logico: input mancante")
        return
    } else if err != nil { // controllo errore generico
        fmt.Println("Errore generico: ", err)
        return
    } // se non c'è stato errore, stampa il risultato
    fmt.Println("Risultato: ", result)
}

```

Web Backend

venerdì 5 dicembre 2025 14:06

Server HTTP Integrato

Go include una libreria HTTP nella libreria standard **net/http**

- **http.Handler**: interfaccia usata per gestire le richieste
- **http.HandleFunc**: associa un percorso a una **funzione gestore**
- **http.ListenAndServe**: avvia il server HTTP

Il gestore di richieste ha la firma:

```
func handler(w http.ResponseWriter, r *http.Request)
```

Il parametro **w** (di tipo **http.ResponseWriter**) è l'interfaccia usata per costruire e inviare la risposta al client

Il parametro **r** (di tipo ***http.Request**) contiene tutte le informazioni inviate dal client

Es endpoint che genera un num. casuale e restituisce la parità

- **http.ResponseWriter (w)**: usato per scrivere la risposta inviata al client (**corpo della risposta**)
- **w.Header().Set(...)**: usato per impostare gli **header** della risposta (es Content-Type). **Deve essere chiamato prima di scrivere il corpo**
- **fmt.Fprintf(w, ...)**: funzione utile che accetta **io.Writer** (come **w**) per scrivere la risposta

```
import (
    "fmt"
    "time"
    "math/rand"
    "net/http"
)

func evenRandomNumber(w http.ResponseWriter, r *http.Request){
    w.Header().Set("Content-Type", "text/plain") // imposta l'header prima di scrivere il corpo
    var p = rand.Int()
    if p % 2 == 0 {
        fmt.Fprintf(w, "%d is even", p)
    } else {
        fmt.Fprintf(w, "%d is odd", p)
    }
}

func main(){
    http.HandleFunc("/", evenRandomNumber)
    fmt.Println("Starting web server at http://localhost:8090")
    http.ListenAndServe(":8090", nil)
}
```

Lettura dei Parametri (Query String)

Per accettare dati da un client tramite l'URL (metodo **GET**) si usano i **Query String Parameters**.

Accesso ai parametri:

- 1) La richiesta **r** contiene l'URL: **r.URL** (di tipo ***url.URL**)
- 2) Si usa **.Query()** per ottenere la mappa dei parametri (di tipo **url.Values**)
- 3) Si usa **.Get("nome_parametro")** per estrarre il valore

```
import (
    "fmt"
    "net/http"
)

func hi(w http.ResponseWriter, r *http.Request){
    name := r.URL.Query().Get("name") // estrae il valore del parametro "name"
    w.Header().Set("Content-Type", "text/plain")
    fmt.Fprintf(w, "Hi %s!", name) // stampa il saluto, usando il nome ricevuto
}

func main(){
    http.HandleFunc("/", hi)
    fmt.Println("Starting web server at http://localhost:8090")
    http.ListenAndServe(":8090", nil)
}
```

Esempio URL: <http://localhost:8090/?name=John+Doe>

Questi parametri sono parte dell'URL e vengono usati per passare dati semplici o opzioni di filtraggio (GET [/api/risorse?id=123&sort=asc](#))

- **r.URL.Query()** (di tipo **url.Values** (mappa di stringhe)): una mappa che associa i nomi dei parametri ai loro valori

Es: estrae il valore del parametro **user_id**

```
Id := r.URL.Query().Get("user_id")
```

Questi parametri sono integrati nel percorso dell'URL ([/users/101](#))

Componente	Package	Descrizione
Standard Library (net/http)		Il gestore deve analizzare manualmente la stringa r.URL.Path (es usando strings.Split) per estrarre i segmenti variabili. NON raccomandato per API complesse

		Es: <code>segments := strings.Split(r.URL.Path, "/")</code> (suddivide il percorso (es <code>/users/101</code>) in segmenti per estrarre l'ID
Router esterno (es Gorilla Mux)	<code>mux.Vars(r)</code>	Restituisce una mappa (<code>map[string]string</code>) contenente i parametri estratti dal path. Questo è l'approccio standard nell'industria Go

Ricezione Dati (POST Body)

Per ricevere dati strutturati (come JSON o testo) inviati con richieste **POST**, è necessario leggere il corpo della richiesta

Lettura del body:

- 1) Il corpo della richiesta è accessibile tramite **r.Body** (di tipo `io.ReadCloser`)
- 2) Usiamo **io.ReadAll(r.Body)** per leggere tutto il contenuto in un array di byte (`[]byte`)

```
import (
    "fmt"
    "io"
    "net/http"
)

func hi(w http.ResponseWriter, r *http.Request){
    body, _ := io.ReadAll(r.Body) // legge l'intero corpo della richiesta come []byte
    name := string(body) // converte il body (si assume sia il nome in text/plain) in stringa
    w.Header().Set("Content-Type", "text/plain")
    fmt.Fprintf(w, "Hi %s!", name)
    // in un server reale si deve sempre controllare l'errore di io.ReadAll
    // e l'header Content-Type della richiesta per dati strutturati
}

func main(){
    http.HandleFunc("/", hi)
    fmt.Println("Starting web server at http://localhost:8090")
    http.ListenAndServe(":8090", nil)
}
```

(l'underscore nel 2° val. di ritorno per `io.ReadAll()` significa che vogliamo ignorare quel valore)

Per testare una richiesta POST si usa tipicamente curl:

```
curl -d "John Doe" http://localhost:8090
```

(si usa **-d** per inviare il corpo della richiesta)

Per i metodi che inviano dati nel corpo (es POST E PUT), l'input si trova in **r.Body**

Componente	Tipo	Descrizione
r.Body	<code>io.ReadCloser</code>	Stream di dati grezzi della richiesta
io.ReadAll(r.Body)	<code>[]byte, error</code>	Legge l'intero corpo in un array di byte. Fondamentale controllare l'errore

Gli header devono essere **impostati prima** di scrivere il corpo della risposta

- **w.Header().Set("Name", "Value")**: imposta un header
Es: `w.Header().Set("Content-Type", "application/json")`
- **w.WriteHeader(code)**: imposta lo status code HTTP. Se non chiamata, Go usa di default 200 OK
Es: `w.WriteHeader(http.StatusCreated) (201)`

Dopo aver impostato gli header, si scrive il corpo

- `w.Write([]byte)`: scrive un array di byte sul corpo. Usato per dati grezzi o JSON codificato
- `fmt.Fprintf(w, format, args...)`: funzione che scrive direttamente su un `io.Writer` (come `w`). Usato spesso per risposte semplici in testo semplice

Gestione di Dati Strutturati: JSON

Normalmente, JSON è il formato standard per scambiare dati. Go usa il package **encoding/json** per mappare i dati tra le struct di Go e le stringhe JSON

Decoding (Leggere dal Corpo di Richiesta)

Per leggere un corpo JSON in una struct Go, si usa **json.NewDecoder**

```
type UserInput struct {
    Name string `json:"name"`
    Age int `json:"age"`
}
// ...
var input UserInput
err := json.NewDecoder(r.Body).Decode(&input)
// ...
```

Encoding (Scrivere nella Risposta)

Per inviare una struct Go come risposta JSON, si usa **json.Marshal** o **json.NewEncoder**

```
type UserResponse struct {
    Message string `json:"status"`
}
// ...
response := UserResponse{Message: "Success"}
w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(response)
```

Concorrenza e Canali

lunedì 22 dicembre 2025 00:22

Concorrenza

Riguarda la **struttura** del programma. Permette di gestire molte cose **contemporaneamente** (passando da un compito all'altro)

Parallelismo

Riguarda l'**esecuzione**. Permette di fare molte cose **contemporaneamente** (eseguendo su core CPU diversi)

La concorrenza è diversa dal parallelismo. Es: un programma può avere un design concorrente che permette l'esecuzione parallela per alcune Goroutine. Go rende semplice la concorrenza, che potrebbe portare all'esecuzione in parallelo (gestito dal runtime)

Goroutine

Le **Goroutine** sono delle versioni più leggere dei **threads** (migliaia di Goroutine possono coesistere) e sono l'unità fondamentale della concorrenza in Go

- **Definizione:** funzioni eseguite in modo **concorrente**, gestite dal runtime di Go
- **Avvio:** si avvia una Goroutine antepoendo la parola chiave **go** alla chiamata di funzione

```
func main(){
    go func(){ // Goroutine separata (eseguita concorrentemente)
        for i := 0; i < 10; i++ {
            fmt.Println("Goroutine: ", i)
        }
    }()

    for j := 0; j < 10; j++ { // Goroutine principale (main)
        fmt.Println("Main: ", j)
    }

    // ATTENZIONE: il main NON aspetta la Goroutine secondaria
    // È necessaria la sincronizzazione (es un canale o WaitGroup)
}
```

Mentre i thread sono programmati direttamente dal kernel, le Goroutine sono gestite dallo **Scheduler di Go**, che si trova all'interno del runtime:

- **M:N Scheduling:** lo Scheduler multiplexa (assegna) molte Goroutine (M) su un num. minore di thread dell'OS (N)
- **Blocco Non Fatale:** se una Goroutine si blocca (es aspettando un I/O) il runtime di Go sposta solo quella Goroutine in attesa, **non blocca l'intero thread** dell'OS su cui era in esecuzione. Il thread viene immediatamente riassegnato a un'altra Goroutine pronta per l'esecuzione
- **Equità:** la commutazione di contesto delle Goroutine è **molto più veloce** di quella dei thread dell'OS e permette a Go di garantire equità e prevenire la **starvation** in modo efficiente

Sincronizzazione e Canali

Per garantire che le Goroutine completino il loro lavoro prima che il programma principale termini, è necessario un meccanismo di **sincronizzazione**

Ci sono due modi principali per sincronizzare le Goroutine:

- 1) Usare i **sync.WaitGroup** (per attendere il completamento. Il **WaitGroup** è specificatamente progettato per aspettare che un insieme di Goroutine abbia completato le proprie operazioni
 - **wg.Add(N):** incrementa un contatore per il num. di Goroutine da attendere
 - **wg.Done():** decrementa il contatore all'interno della Goroutine (tipicamente con defer)
 - **wg.Wait():** blocca la Goroutine chiamante (main) finché il contatore non torna a zero

```
func main(){
    var wg sync.WaitGroup
    wg.Add(1) // diciamo al WaitGroup di aspettare 1 Goroutine

    go func(){
        // garantisce che il contatore sia decrementato, anche in caso di panico
        defer wg.Done()
        for i := 0; i < 5; i++ {
            fmt.Println("Goroutine: ", i)
            time.Sleep(100*time.Millisecond)
        }
    }()

    fmt.Println("Main in attesa...")
    wg.Wait() // il main si blocca qui finche la goroutine non chiama wg.Done()
    fmt.Println("Programma completato")
}
```

- 1) Usare i **canali (channels)**. Un canale è una **coda FIFO** dove le Goroutine possono **scrivere/leggere dati** di un tipo specifico per scambiare dati tra di loro. Il canale non solo trasferisce dati, ma blocca l'esecuzione finché sia l'invio che la ricezione non sono pronti. Se il main attende di ricevere sul canale, attenderà finché la Goroutine non lo invierà
 - **Creazione:**
 - **make(chan Type):** canale **non bufferizzato**, il produttore è bloccato finché il consumatore non legge e viceversa
 - **make(chan Type, N):** canale **bufferizzato** con **capacità N**, il produttore può scrivere finché il buffer non è pieno, poi è bloccato finché il consumatore non legge almeno un valore. Stessa cosa per il consumatore
 - **Operazioni (bloccanti):**
 - **Invio:** channel <- value
 - **Ricezione:** v := <-channel


```
func main(){
    // canale vuoto, usato solo come segnale
    done := make(chan bool) // canale di tipo booleano

    go func(){
        for i := 0; i < 5; i++ {
            fmt.Println("Goroutine: ", i)
            time.Sleep(100*time.Millisecond)
        }
        done <- true // invia un segnale al main: "Ho finito!"
    }()

    fmt.Println("Main in attesa del segnale...")
    <-done // il main si blocca in attesa di ricevere un valore dal canale "done"
    fmt.Println("Programma completato")
}
```

I canali non bufferizzati garantiscono che invio e ricezione **avvengano simultaneamente (rendezvous)**

```
func main(){
    // canale non bufferizzato: richiede incontro (rendezvous)
    var ch = make(chan int)

    go func(){
        var i = 0
        for i < 10 {
            ch <- i // 1) Invio: si blocca finche qualcuno non riceve
            i++
        }
    }()

    var j = 0
    for j < 10 {
        fmt.Println(<-ch) // 2) Ricezione: si blocca finche qualcuno non invia
        j++
    }
    // vengono garantiti che i messaggi sono consumati in ordine
}

func main(){
    // 1) viene creata una struttura dati per il canale
    // "channel" detiene un puntatore a questa struttura
    var channel = make(chan int)

    // 2) la Goroutine 1 riceve una COPIA del puntatore
    go senderGoroutine(channel)

    // 3) la Goroutine 2 riceve una COPIA del puntatore
    go receiverGoroutine(channel)

    //
}

func senderGoroutine(ch chan int){
    ch <- 42 // invia sulla struttura CONDIVISA
}

func receiverGoroutine(ch chan int){
    v := <-ch // riceve sulla struttura CONDIVISA
}
```

Sincronizzazione Manuale con Mutex

Quando bisogna **condividere la memoria** (variabili) invece di comunicare tramite canali, si usano i **Mutex** per evitare race condition. Un Mutex **protegge** una **sezione critica** di codice e solo una Goroutine alla volta può ottenere il **lock**.

- **mu.Lock()**: acquisisce il lock o blocca se già detenuto da un'altra Goroutine
- **mu.Unlock()**: rilascia il lock così che le altre Goroutine possono accedere alla memoria condivisa

```
var counter int
var mu sync.Mutex // mutex per proteggere "counter"

func Increment(){ // rendiamo Increment() sicura per l'uso concorrente
    mu.Lock() // Blocco: inizia la sezione critica
    defer mu.Unlock() // Defer: garantisce il rilascio del lock, anche in caso di panico
    counter++ // accesso sicuro alla risorsa condivisa
}
```

Multiplexing I/O

L'istruzione **select** permette a una Goroutine di **attendere su più operazioni** di canali contemporaneamente.

- **Comportamento**: select **blocca** finché uno dei suoi **case** (operazioni su canale) è pronto
- **Scelta casuale (equità)**: se **più canali** sono pronti, select ne **sceglie** uno a **caso** per l'esecuzione. L'ordine dei case nella sintassi non influisce sulla priorità
- **default**: se presente, select **non blocca** ma **esegue il default** se nessun canale è immediatamente pronto


```
select {
case v1 := <-chan1:
    fmt.Printf("Ricevuto %v dal chan 1\n", v1)
case chan2 <- 1:
    fmt.Println("Valore inviato a chan 2")
default:
    fmt.Println("Nessun canale è pronto")
}
```

Un uso comune di select è l'implementazione di **timeout** usando **time.After(duration)** che restituisce un canale che riceve un val. dopo la durata specificata. Se il canale di timeout è pronto prima del canale dati, l'**operazione scade**.

```
select {
case v1 := <-dataChannel:
    fmt.Printf("Ricevuto dato: %v\n", v1) // operazione riuscita
case <-time.After(5*time.Second):
    fmt.Println("Timeout: non è stato ricevuto nessun dato") // operazione fallita per timeout
}
```

Esempi Backend

domenica 21 dicembre 2025 23:29

Backend per "Hi-Lo"

Specifiche API (Requisiti Minimi)

Definiamo gli endpoint che il server deve esporre

Metodo	Percorso	Azione	Risposta (JSON)
POST	/games	Avvia una nuova partita	{"game_id": "ID_UNIVOCO_GIOCO"}
POST	/games/{id}/guess	Invia un tentativo per una partita specifica	{"result": "hi"} (o "lo", "correct")
GET	/games/{id}	Mostra lo stato attuale della partita	{"guesses_made": 5, "status": "in_progress"}

Suggerimenti Pacchetti

- **net/http**: per il server e i gestori (handler)
- **encoding/json**: per **NewDecoder** (leggere JSON) e **NewEncoder** (scrivere JSON)
- **math/rand**: per generare il numero segreto all'avvio della partita
- **time**: per inizializzare **rand.Seed()**
- **sync**: per il **Mutex**

Esercizio: HandleNewGame (POST /games)

- 1) Generare un numero segreto (es `rand.Int(100) + 1`)
- 2) Generare un ID univoco per la partita (es un contatore o `fmt.Sprintf`)
- 3) Creare la nuova **GameState**
- 4) **Bloccare il Mutex** (`mu.Lock()`)
- 5) Salvare la nuova partita nella mappa **games**
- 6) **Sbloccare il Mutex** (`mu.Unlock()`)
- 7) Rispondere al client con il JSON contenente il `game_id`

Esercizio: HandleGuess (POST /games/{id}/guess)

- 1) Ottenere `{id}` dal percorso (usando `mux.Vars(r)`)
- 2) Leggere il JSON dal corpo (`r.Body`) per ottenere il tentativo del client
- 3) **Bloccare il Mutex** (o `mu.RLock()`) se si usa **RWMutex**
- 4) Recuperare lo stato della partita (`games[id]`)
- 5) **Sbloccare il Mutex** (o `mu.RUnlock()`)
- 6) Controllare se lo stato esiste (errore 404 se non esiste)
- 7) Confrontare il tentativo con `SecretNumber` e definire la risposta ("hi", "lo", "correct")
- 8) (Opzionale: aggiornare lo stato con il nuovo tentativo, richiederà `mu.Lock()`)
- 9) Rispondere al client con il JSON contenente il risultato

Backend per "Nasoni"

Ci basiamo sugli endpoint che abbiamo definito in [API Finale](#)

```
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
    "strconv"
)

type Fountain struct {
    ID int `json:"id"`
    State string `json:"state"`
    Latitude float64 `json:"latitude"`
    Longitude float64 `json:"longitude"`
}

func getFountainsFromDataStorage(lat, lng, filterRange float64) []Fountain {
```

```

}

func getFountainsFromDataStorage(lat, lng, filterRange float64) []Fountain {
    // Recupera la lista delle fontane da qualche parte
}

func listFountains(w http.ResponseWriter, r *http.Request) {
    // Prima, controlliamo il metodo HTTP
    // siccome il router Go di default non lo controlla
    if r.Method != http.MethodGet {
        w.WriteHeader(http.StatusMethodNotAllowed)
        return
    }

    // Ora, controlliamo per la stringa query
    // Prima, controlliamo se i valori richiesti sono impostati
    if !r.URL.Query().Has("latitude") {
        w.WriteHeader(http.StatusBadRequest)
        return
    }

    if !r.URL.Query().Has("longitude") {
        w.WriteHeader(http.StatusBadRequest)
        return
    }

    var lat, lng, filterRange float64
    var err error

    lat, err = strconv.ParseFloat(r.URL.Query().Get("latitude"), 64)
    if err != nil {
        // La latitudine non è un float valido
        w.WriteHeader(http.StatusBadRequest)
        return
    }

    lng, err = strconv.ParseFloat(r.URL.Query().Get("longitude"), 64)
    if err != nil {
        // La longitudine non è un float valido
        w.WriteHeader(http.StatusBadRequest)
        return
    }

    // Controlliamo il parametro opzionale "range"
    if r.URL.Query().Has("range") {
        filterRange, err = strconv.ParseFloat(r.URL.Query().Get("range"), 64)
        if err != nil {
            // Il valore non è un float valido, rifiutiamo la richiesta
            w.WriteHeader(http.StatusBadRequest)
            return
        } else if filterRange < 1 || filterRange > 200 {
            // Il valore è fuori dal range, rifiutiamo la richiesta
            w.WriteHeader(http.StatusBadRequest)
            return
        }
    } else {
        filterRange = 10
    }
}

```

```

    filterRange = 10
}

// Recupera la lista delle fontane
fountains := getFountainsFromDataStorage(lat, lng, filterRange)
// Invia la lista all'utente
w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(fountains)
}

func main() {
    http.HandleFunc("/fountains/", listFountains)
    fmt.Println("Starting web server at http://localhost:8090")
    http.ListenAndServe(":8090", nil)
}

```

Best Practice

lunedì 22 dicembre 2025 12:46

I prof di questo corso hanno fatto un template opzionale ("Fantastic Coffee (decaffeinato)") che combina alcune best practices per Go <https://github.com/sapienzaapps/fantastic-coffee-decaffeinated>

```
cmd/  
demo/  
doc/  
go.mod  
go.sum  
LICENSE  
open-npm.sh  
README.md  
service/  
vendor/  
webui/
```

Struttura del template

cmd/

cmd/ contiene gli eseguibili

Il codice di Go in questa cartella dovrebbero solo presentare package in maniera usabile

Es:

- 1) Leggere opzioni CLI
- 2) Importare packages e usare funzioni per registrare gli **handler API**
- 3) Far partire il web server
- 4) Aspettare per il segnale di terminazione

cmd/healthcheck

È un esempio di un daemon per controllare la **salute dei daemons** del server

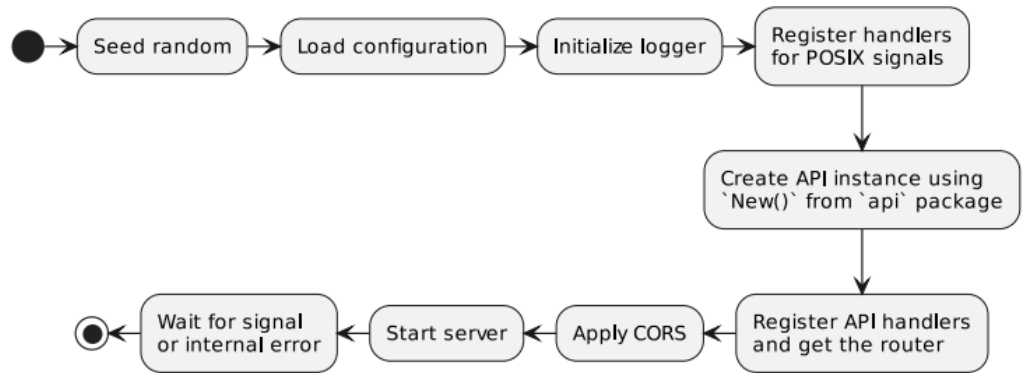
Utile quando l'hypervisor non provvede controlli di liveness/readiness HTTP

cmd/webapi

Contiene un esempio di un daemon per un web server API

```
cmd/webapi/cors.go  
cmd/webapi/load-configuration.go  
cmd/webapi/main.go  
cmd/webapi/register-web-ui.go  
cmd/webapi/register-web-ui-stub.go
```

[cmd/webapi/main.go](#)



doc/

Contiene la **documentazione** (normalmente per l'APIs)

service/

Tutti i pacchetti dovrebbero essere qui, come sotto-directory. L'annidamento è possibile

Es: services/globaltime is the globaltime package

vendor/

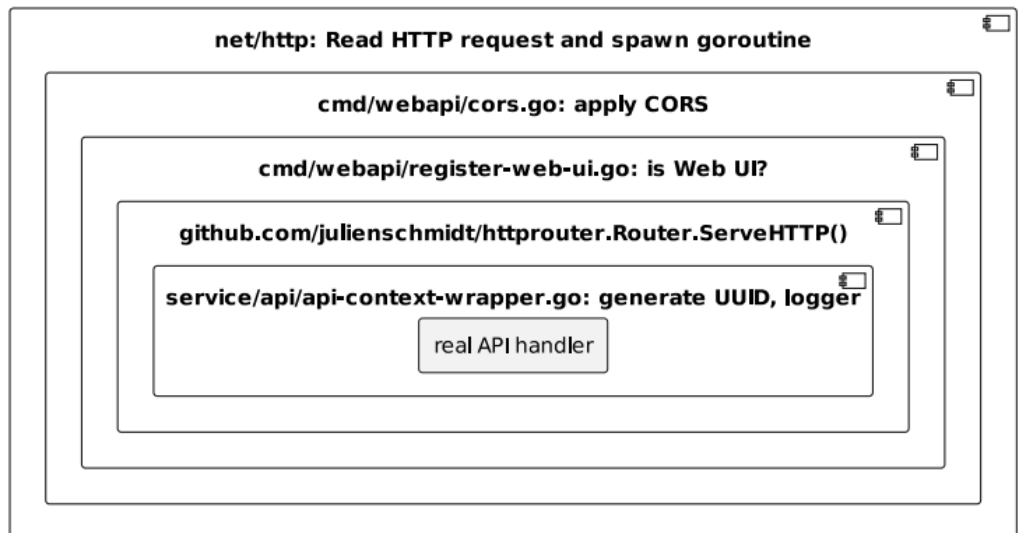
Contiene le risorse per tutti i packages esterni ("dependencies")

Bisogna aggiornarla quando si aggiunge/aggiorna/rimuove una dependencie: **go mod vendor**

webui/

Contiene il progetto di **Vue.js**

HTTP Request Wrappers



Appunti Veloci Javascript

mercoledì 7 gennaio 2026 11:58

(Scusate skippo HTML)

JavaScript è un linguaggio eseguito principalmente nel browser e compilato just-in-time all'avvio di una pagina web

I modi per aggiungere Javascript ad un documento HTML sono:

- 1) **Interno**: nel tag `<script>` nella sezione `<head>` o `<body>`
- 2) **Esterno**: collegando un file .js usando il tag `<script src="file.js"></script>`
- 3) **HTML Event Handler**: direttamente nei tag HTML (es `onclick="..."`) (sconsigliato)

Tipi e Variabili

JavaScript ha una **tipizzazione dinamica** (il tipo della variabile non deve essere specificato e può cambiare) e **debole** (conversione del tipo automatica)

Le variabili possono essere dichiarate con:

- **var**: **scope di funzione**, cioè accessibili ovunque nella funzione
- **let**: **scope di blocco**, cioè accessibili solo nel blocco di codice (delimitato da parentesi graffe) in cui sono definite
- **const**: sempre a scope di blocco e **non può essere riassegnata**

```
function scopeFunzione() {
  var a = 1; // function-scoped
  a = "hi"; // riassegnazione del tipo, ora è string
  if (true) {
    var b = 2; // function-scoped
    console.log(a) // log: "hi"
  }
  // b è accessibile fuori dal blocco if
  console.log(b) // log: 2
}

function scopeBlocco() {
  let c = "hello" // block-scoped
  c += 123 // conversione automatica
  if (true) {
    const d = "ciao"; // block-scoped, non può essere modificato
    console.log(c) // log: "hello123"
  }
  // d NON è accessibile fuori dal blocco if
  console.log(d) // Errore: ReferenceError
}
```

Undefined è un tipo di dato primitivo e un valore e indica che una variabile è stata **dichiarata ma non** ha un valore **assegnato**

Operatori

- **Aritmetici**: +, -, *, /, % (modulo), ++ (incremento), -- (decremento)
- **Assegnazione**: =, +=, -=, *=, /=, %=-
- **Stringa**: + (concatenazione e assegnazione automatica nel caso di tipi diversi) ("hello" + "world" → "helloworld" | "hello" + 123 → "hello123")
- **Logici**: && (AND), || (OR), ! (NOT)
- **Comparazione**:
 - == (**uguaglianza debole**, con coercizione di tipo)
 - === (**uguaglianza forte**, senza coercizione di tipo)
 - != (disuguaglianza debole), !== (disuguaglianza forte)
 - >, <, >=, <=

```
let x = 5 + 3; // x = 8
x += 2; // x = 10

let str = "hello" + " " + "world" // str = "hello world"

// disuguaglianza debole
console.log(5 == "5") // true
// disuguaglianza forte
console.log(5 === "5") // false
```

(skippo i cicli for/while)

Oggetti e Funzioni

Gli **oggetti** sono una collezione **mutabile** di proprietà, dove:

- Le **chiavi** delle proprietà sono stringhe
- I **valori** delle proprietà sono qualsiasi tipo di Javascript

Si possono accedere i valori delle proprietà con la notazione **punto**

Anche le **funzioni** sono oggetti e possono restituire i valori con **return**.

Il return è **opzionale** e se non viene specificato la funzione restituisce **undefined**

Un altro modo per definire una funzione è con la **funzione freccia** (=>)

let/var/const nome_funz = (arg1, arg2, ... argN) => espressioni

```
var user = {
  name: "Fra",
  age: 22
}

function multiply(a, b){
  return a*b;
}

const x = multiply(4, 3); // x è 12

// sintassi concisa per le funzioni con funzione freccia
const sum = (a, b) => {
  return a+b;
}

// sintassi concisa per singola espressione con funzione freccia
const square = (a) => a*a;
```

DOM (Document Object Model)

JavaScript può modificare il contenuto della pagina HTML accedendo al **DOM**, un modello di programmazione per documenti HTML e XML che fornisce una **rappresentazione ad albero** del documento permettendo tramite un'interfaccia di **interagire con l'albero** e manipolare i suoi elementi

L'oggetto JS **document** rappresenta il DOM e da esso possiamo accedere ad ogni nodo del DOM (altri oggetti JS) per modificarli.

Accedere agli Elementi del DOM

- **document.getElementById("id")**: trova un elemento per **id**
- **document.getElementsByTagName("name")**: trova elementi per nome **tag** (restituisce htmlcollection)
- **document.getElementsByClassName("name")**: trova elementi per nome **classe** (restituisce htmlcollection)
- **document.querySelector("css selector")**: trova il **primo** elemento che corrisponde a un selettore css
- **document.querySelectorAll("css selector")**: trova **tutti** gli elementi che corrispondono a un selettore css (restituisce nodelist)

Modificare gli Elementi del DOM

- **element.innerHTML = new html content**: modifica il **contenuto html** di un elemento
- **element.innerHTML = new value**: modifica il **valore** di un **attributo** di un elemento
- **element.style.property = new style**: modifica lo stile di un elemento

Aggiungere e Rimuovere Elementi DOM

- **document.createElement(element)**: crea un nuovo elemento html
- **document.removeChild(element)**: rimuove un elemento html
- **document.appendChild(element)**: aggiunge un elemento html
- **document.replaceChild(new, old)**: sostituisce un elemento html
- **document.write(text)**: scrive direttamente nell'output html

```
// modifico il contenuto di un link con l'ID "my-link"
document.getElementById("my-link").href = "new-url.html";

// prendo il primo elemento tra tutti quelli che hanno la classe "my-class"
document.getElementsByClassName("my-class")[0].innerHTML = "nuovo testo";

// creo un nuovo elemento <span>
const myElem = document.createElement("span");

// imposto l'id, lo stile e il contenuto
myElem.id = "bar";
myElem.style = "color:red";
myElem.innerHTML = "questo è rosso";

// aggiunge il nuovo elemento al corpo del documento
document.body.appendChild(myElem);
```

Import e Export

Per dividere il codice in **moduli** differenti si può usare **import** per importare variabili e funzioni da altri moduli e **export** per esportarli appunto ad altri moduli. Aggiungendo **type="module"** al tag <script> il runtime interpreta il file html come un modulo


```

export let name1, name2, /*, ... */ nameN;
export function functionName() { /* ... */ }
export { variable1 as name1, variable2 as name2, /* ..., */ nameN };
export default function functionName() { /* ... */ }
export * from "module-name"; // aggregazione di moduli

// ESEMPIO IMPORT/EXPORT
// file homework.js
export function hwAvg(grades) {
  var sum = 0;
  for (let grade of grades) { sum += grade; }
  return sum / grades.length;
}

// file myWebapp.html
<>
<p id="avg"></p>
<script type="module">
  import {hwAvg} from "../homework.js";
  document.getElementById("avg").innerHTML = hwAvg([30, 28, 30, 29])
</script>
</>

```

La parola chiave **default** è importante per i moduli di javascript:

- 1) **Definisce l'esportazione principale:** ogni modulo può avere **una e una sola** esportazione di default. Questa è l'esportazione principale che ci si aspetta venga usata da chi importa il modulo
- 2) **Semplifica l'importazione:** quando si importa l'esportazione di default, non è necessario usare le parentesi graffe {} e si può **scegliere un nome a piacere** per l'entità importata

```

// file test_export.js
const k = 12;
export default k; // export di default

// file test_import.js
import m from "../test_export.js";
// possiamo usare m invece di k, perche k è l'export di default
console.log(m);

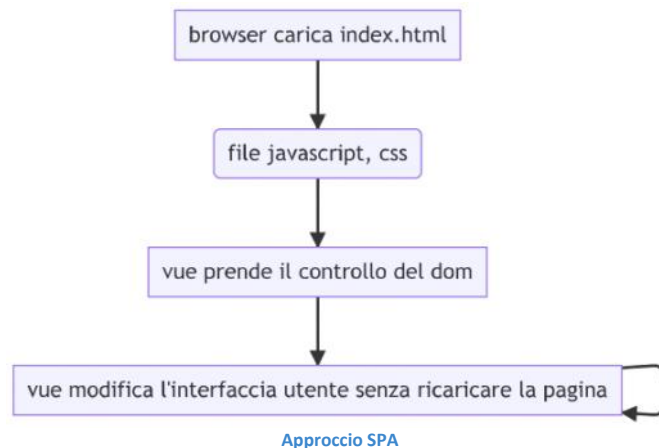
```

<i>Tipo di Export</i>	<i>Sintassi Export</i>	<i>Sintassi Import</i>	<i>Regola Naming</i>
Nominata	export const foo = 1;	import {foo} from "../file";	Devi usare lo stesso nome (foo)
Default	const bar = 2; export default bar;	import qualsiasiNome from "../file";	Puoi usare qualsiasi nome

Intro a Vue.js

mercoledì 7 gennaio 2026 16:43

Vue.js è un **framework** (libreria di componenti javascript per creare applicazioni più velocemente) per la creazione di interfacce utente. È facile da integrare con altri progetti o librerie e può essere usato come alternativa a jquery o **progetti SPA** (Single Page Application)



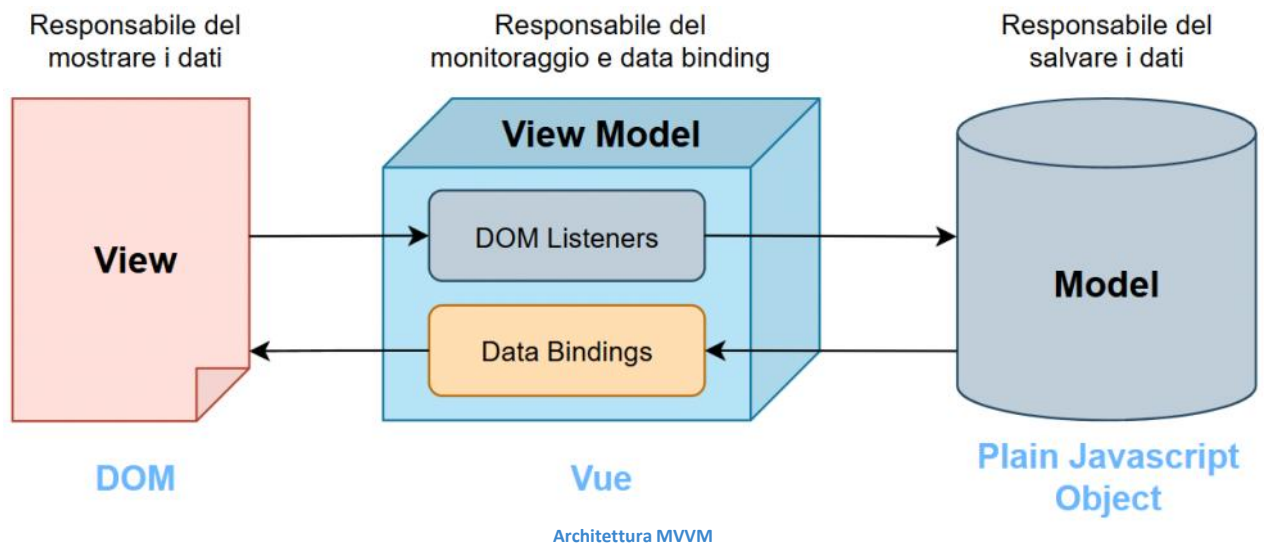
Le funzionalità principali sono:

- **Rendering dichiarativo e basato su componenti** che usando una sintassi di template permette di descrivere dichiarativamente l'output html in base allo JS
- **Reattività**: il sistema traccia automaticamente le modifiche allo stato javascript e **aggiorna il DOM** (la rappresentazione ad albero dell'HTML) in modo efficiente senza ricaricare la pagina

Si può utilizzare Vue.js in numerosi modi poiché è focalizzato sull'architettura **MVVM (Model-View-ViewModel)**

In più Vue offre due approcci differenti per scrivere i componenti:

- **Options API**: definisce la logica del componente usando un **oggetto con opzioni** come **data**, **methods**, **mounted**. È considerato più amichevole per i principianti
- **Composition API**: definisce la logica importando funzioni API. Offre maggiore flessibilità e migliore riutilizzo del codice



Esempio minimo con l'approccio **options API** senza un sistema di build:

Parte JS	Template HTML
<pre>import { createApp } from 'vue' createApp({ // la funzione data restituisce lo stato reattivo data() { return { count: 0 } } }).mount('#app')</pre>	<pre><div id="app"> <!-- ascolta l'evento click e incrementa count --> <button @click="count++"> Count is: {{ count }} </button> </div></pre>

Componenti Vue.js

L'applicazione di Vue è fatta di vari **componenti** e usa un **sistema di template** che estendono la sintassi HTML per aggiungere vari componenti nuovi (es attributi personalizzati per eventi, elementi custom per inserire componenti Vue.js, ecc)

I componenti Vue hanno un'estensione **.vue** e solitamente contengono tre blocchi:

- **<template>**: contiene il markup HTML (con sintassi speciale Vue)

- **<script>**: contiene la logica JS del componente
- **<style>**: contiene il css specifico per il componente

View/Vista

La **vista** (o view o pagina) è la rappresentazione visiva dell'app (DOM, cioè HTML+CSS)

- È **passiva**: non prende decisioni
- Si occupa solo di **come mostrare i dati** all'utente
- Intercetta le interazioni (click, input) e le passa al **viewmodel**

In vue corrisponde alla sezione **<template>**

ViewModel

Il **viewmodel** fa da mediatore tra la vista e la logica di presentazione

- Agisce da ponte tra **vista e model** (dati grezzi)
- Espone dati pronti per la visualizzazione (reattività)
- gestisce i comandi dell'utente e aggiorna i dati

In vue corrisponde alla sezione **<script>**

Un componente a **file singolo** (SFC, Single File Component) è un file simile all'HTML che consente di riusare i componenti e contiene:

- Javascript
- HTML (template)
- CSS

Es: estendere il pulsante HTML standard con un comportamento personalizzato

```
<script>
export default {
  data() {
    return { count: 0 }
  }
}
</script>
<template>
<button @click="count++">Count is: {{ count }}</button>
</template>
<style scoped>
button { font-weight: bold; }
</style>
```

Elementi Principali dell'App

Pagina Index HTML

index.html contiene l'HTML per la single page. Può contenere codice non-vue come js/css esterni o custom

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Example app</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <div id="app"></div>
    <!-- il punto di ingresso per i moduli javascript -->
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

Main JS

main.js è il punto d'ingresso dell'app poiché **crea e monta l'app** stessa

```
import {createApp, reactive} from 'vue'
import router from './router'
import App from './App.vue'
import Component1 from './components/Component1.vue'
const app = createApp(App)
app.component("Component1", Component1);
app.use(router)
app.mount('#app')
```

App.vue

In **app.vue** abbiamo il componente principale della **spa**

```
<script setup>
import { RouterView } from 'vue-router'
</script>
<script>
export default {}
</script>

<template>
  <header>App header</header>
  <main><RouterView ></RouterView></main>
</template>

<style>
</style>
```

[Router](#)

Il **router** permette lo scambio dei vari componenti e gestire le rotte per le viste nell'app

[Installazione](#)

Usiamo vue con **vite**, un sistema di build per lo sviluppo web
npm create vue@latest

Sintassi Template

lunedì 12 gennaio 2026 12:32

Interpolazione del Testo da Variabili

Per inserire i valori di una variabile nel DOM con la sintassi mustache `{{ data }}` e grazie alla **reattività** il valore della variabile si **aggiorna automaticamente** quando il valore cambia.

È possibile anche usare le **espressioni javascript**, tramite il controllo di flusso o dichiarazioni

Il testo viene interpolato dal valore della variabile a una stringa semplice, quindi l'utente vede solo il testo e non il nome della variabile

```
<span>Message: {{ msg }} </span>
{{ number + 1 }} <!-- operazioni aritmetiche -->
{{ ok ? "YES" : "NO" }} <!-- operatore ternario per condizioni -->
{{ list.length }} <!-- proprietà degli oggetti -->
{{ message.split("").reverse().join("") }} <!-- chiamate di funzione -->
{{ var a = 1 }} <!-- non funziona -->
{{ if (ok) { return message } }} <!-- non funziona -->
```

Direttive

Le **direttive** sono attributi personalizzati di vue per i tag HTML e permettono di **controllare e interagire col DOM**

Le direttive iniziano con "v-" (si può usando le scorciatoie)

Es struttura:

- v-on:submit.prevent="onSubmit"
- **Argomento:** segue i due punti o il simbolo della scorciatoia
- **Modificatori:** denotati dal punto iniziale
- **Valore:** interpretato come espressione JS

Le direttive principali sono:

- **v-text:** imposta il **contenuto testuale**
- **v-html:** imposta il **contenuto HTML** (non sicuro, meglio v-text)
- **v-on:** registra un **event listener** (scorciatoia: "@" al posto di "v-on:")
- **v-bind:** lega un **attributo a una variabile** (scorciatoia: si può omettere "v-bind" e lasciare i ":")
- **v-model:** **binding bidirezionale** con una variabile (solo per `<input>`, `<select>` e `<textarea>` e componenti vue.js)
- **v-show:** commuta la **visibilità** dell'elemento
- **v-if**, **v-else-if** e **v-else:** commuta il **rendering** dell'elemento
- **v-for:** **ripete l'elemento** corrente (e il sottoalbero)

v-bind Vs v-model

v-bind serve per mostrare da javascript all'utente e basta

v-model invece mostra i dati e permette all'utente di modificarli (è simile a v-bind + v-on)

	Mustache <code>{{ data }}</code>	v-bind	v-on	v-model
Target DOM	Contenuto Testuale dell'elemento	Attributo dell'elemento (href, class, id, ...)	Eventi dell'elemento (click, submit, ...)	Valore/Proprietà dell'input
Sintassi	<code>{{ ... }}</code>	"v-bind:" oppure ":"	"v-on:" oppure "@"	"v-model:"
Flusso Dati	JS → DOM (solo output)	JS → DOM (solo output)	DOM → JS (evento input)	DOM ↔ JS (Input/Output)
Esempio	<code><div>Ciao {{ name }}</div></code>	<code><div :class="myClass"></div></code>	<code><button @click="save"></code>	<code><input v-model="name"></code>

```

<span v-text="msg"></span> <!-- contenuto testuale -->

<button v-on:click="sendMsg('Hello!')">
  Chiama la funzione sendMsg() con il parametro "Hello!"
</button>
<span @click="lightsOff">
  Scorciatoia con "@". Se non mandiamo parametri non servono le parentesi
</span>
<button @click="count++"> Aumenta la variabile count ad ogni click</button>

<span v-bind:id="dynamicId">
  L'attributo id sarà impostato al valore della variabile JS dynamicId
</span>
<div :id="elementId"></div> <!-- scorciatoia rimuovendo "v-bind" -->

<input type="text" v-model="name"></input> <!-- binding bidirezionale -->

<p v-show="showMessage">
  Questa parte è visibile solo quando showMessage è vero
</p> <!-- l'elemento è renderizzato ma è invisibile -->

<p v-if="checkOk">
  Se checkOk è true allora questa parte è visibile
</p>
<p v-else>
  Altrimenti se è false viene mostrata questa
</p>

<div v-for="item in items"></div>
<!-- se l'array items è ["a", "b", "c"] l'HTML risultante è -->
<div>a</div>
<div>b</div>
<div>c</div>

<div v-for="(item, index) in items"></div> <!-- per prendere l'index -->

```

Reattività

lunedì 12 gennaio 2026 16:44

La programmazione **reattiva** è un paradigma di programmazione dichiarativo e si basa sul **flusso di dati** (data streams) e **propagazione del cambiamento**. Nei linguaggi reattivi, le variabili reattive vengono **ricalcolate** quando una dipendenza cambia. Es:

```
reactive var $b = a + 1
```

La variabile reattiva **b** cambia quando **a** cambia, quindi se anche dopo il calcolo di **b**, **a** cambia valore, **b** viene ricalcolato

Linguaggio	Codice	Output Iniziale	Output Finale	Descrizione
Imperativo	<pre>var a = 2; var b = a + 1; fmt.Println(b); a = 1; fmt.Println(b);</pre>	3	3	b non cambia , il calcolo è eseguito una sola volta
Reattivo	<pre>var a = 2; reactive var b = a + 1; fmt.Println(b); a = 1; fmt.Println(b);</pre>	3	2	b è reattivo e si aggiorna al cambiare di a

In Vue la funzione **data()** permette di definire una **lista di variabili reattive**, che vengono restituite da **data()** (usa **undefined o null** se non c'è un valore)

Vue è **deep-reactive**: le modifiche negli oggetti annidati vengono rilevate

Si possono accedere alle variabili reattive di JS usando la parola chiave **this**, mentre non è necessario usarla nei template

Nota che le variabili della funzione **data()** non sono "pure", sono **avvolte nel sistema reattivo**, quindi sono differenti da variabili non reattive (es di **someObject**)

```
<script>
export default {
  data() { // definiamo una lista di variabili reattive
    return {
      count: 0,
      someObject: {}
    }
  },
  mounted() { // mounted() è un hook del ciclo di vita
    console.log(this.count) // => 0
    // puoi cambiarla come una normale variabile JS
    // questo attiverà l'aggiornamento nel DOM e
    // tutte le computed properties o watchers che la usano
    this.count = 1

    const newObject = {} // oggetto normale
    this.someObject = newObject // assegniamo l'oggetto reattivo a newObject
    console.log(newObject === this.someObject) // => falso
  }
}
</script>
<template>
  <button @click="count++">Contatore: {{ count }}</button>
</template>
```

Computed Properties

Le **computed properties** sono proprietà che si **aggiornano automaticamente** quando una **dipendenza** (variabile reattiva) **cambia**

Quando lo stato reattivo muta, Vue aggiorna il DOM automaticamente, ma **non sincronamente**. I cambiamenti sono **bufferizzati fino al prossimo "tick"** (ciclo di update)

Una proprietà **computed** è un **getter** con caching e viene ricalcolata solo quando le dipendenze reattive cambiano.

Se le dipendenze **non cambiano**, viene restituito il valore precedente **senza eseguire la funzione**

Attenzione: una funzione regolare definita in **methods** viene rieseguita a **ogni render del componente**, anche se i suoi input non sono cambiati

Oltre a definire il **getter**, possiamo anche definire i **setter** nell'oggetto, permettendoci di andare a modificare i suoi valori

Esempio:

```

<script>
  export default {
    data() {
      return {
        firstName: "John",
        lastName: "Doe"
      }
    },
    computed: {
      // viene letto come una normale proprietà
      fullNameSimple() { // sola lettura
        return this.firstName + " " + this.lastName
      },
      fullName: { // sia lettura che scrittura
        // Getter chiamato quando viene letto fullName
        get() {
          return this.firstName + " " + this.lastName
        },
        // Setter chiamato quando viene assegnato this.fullName = "..."
        set(newValue) {
          // non si usa "let", si usano le proprietà reattive di firstName e lastName
          [this.firstName, this.lastName] = newValue.split(" ")
        }
      }
    }
  }
}
</script>

```

Nell'esempio se eseguiamo:

- `console.log(this.fullName);` ⇒ chiama il `get()` e esegue l'output del nome
- `this.fullName = "Mario Rossi";` ⇒ chiama il `set("Mario Rossi")`, esegue lo split del valore inviato e assegna alle variabili reattive i rispettivi valori

Metodi (Methods) e Listener dello Stato Reattivo (Watchers)

I **metodi** sono funzioni, definite nell'opzione **methods**, usate per eseguire logica o cambiare lo stato. Vengono chiamate da JS o nel template (es con v-on)

Per le funzioni in methods **non si può usare l'arrow notation**

I metodi **watchers (osservatori)** sono definiti nell'opzione **watch** e sono utili per eseguire **operazioni in risposta** a cambiamenti reattivi dello stato e per eseguire **effetti collaterali**, spesso asincroni, come il **fetch di dati**

```

<script>
  export default {
    data() {
      return { count: 0 }
    },
    watch: {
      // la chiave corrisponde al nome della proprietà data
      count(newValue, oldValue) { // funzione chiamata quando count cambia
        console.log(`count è cambiato da ${oldValue} a ${newValue}`)
        // esegui effetti collaterali, es. salvare i dati
      }
    },
    methods: {
      increment() {
        this.count++ // cambia il valore di count e chiama il watcher
      }
    }
  }
}
</script>

```

Computed Vs Methods Vs Watchers

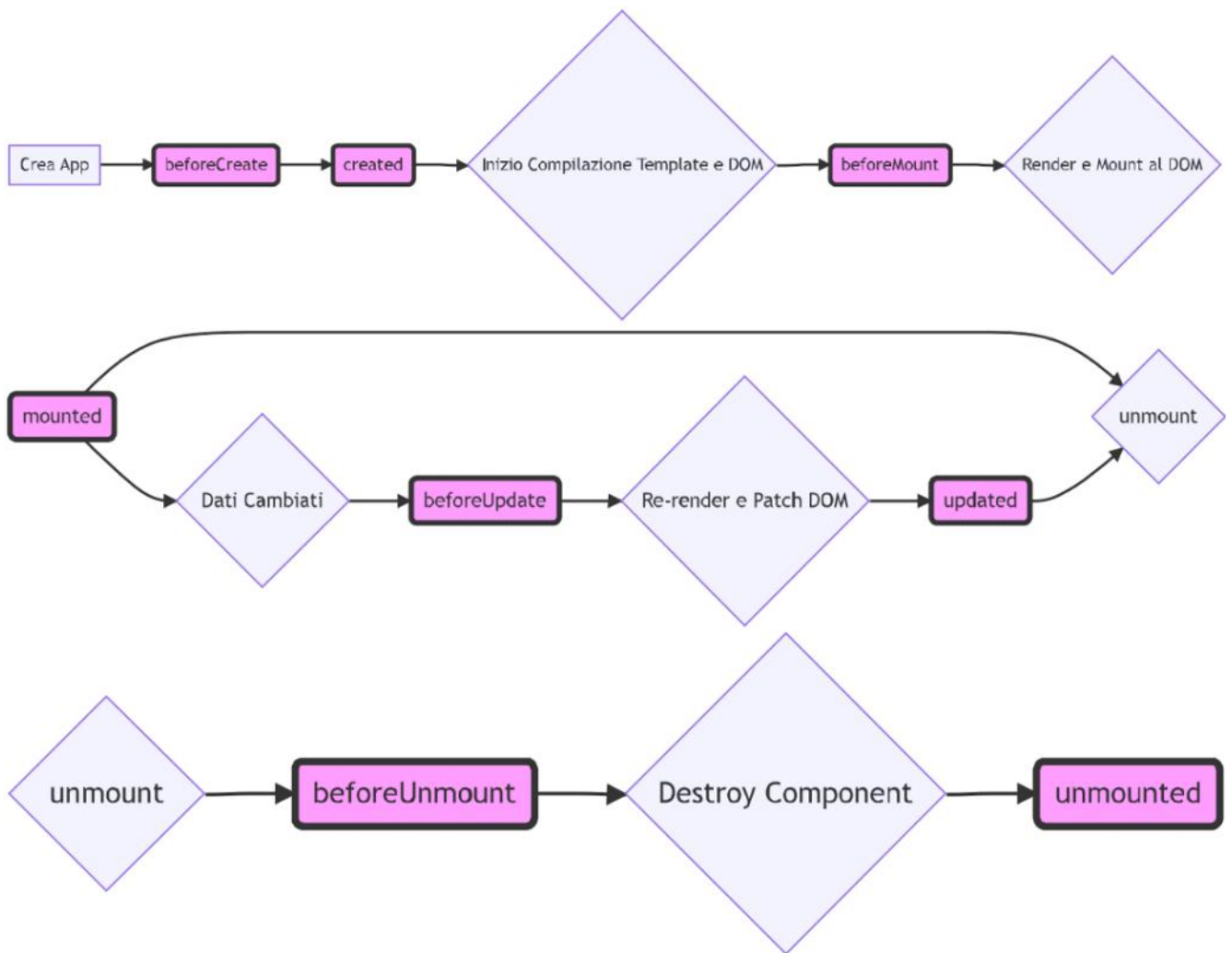
Caratteristica	Computed	Methods	Watchers
Scopo Primario	Calcolare un nuovo valore reattivo	Eseguire un' azione logica	Eseguire una funzione (effetto collaterale) in risposta a un cambiamento
Output	Restituisce per forza un valore	Può restituire o meno un valore	Esegue logica, non restituisce un valore
Caching	Sì, non ricalcola ogni volta i dati se non sono cambiati	No, sono ri-eseguiti ad ogni chiamata	No, ogni volta che il dato cambia, riesegue il codice
Quando Usarlo	Per valori derivati, ogni volta che visualizzare qualcosa che dipende da altri dati	Per eseguire logica o cambiare lo stato	Per operazioni asincrone o costose in risposta a un cambiamento

Ciclo di Vita dei Componenti

Ogni istanza di un componente attraversa diversi stati nel suo **lifecycle** (ciclo di vita) (es: mounted, beforeUpdate, beforeUnmount, ecc).

È possibile eseguire codice JS quando viene raggiunto uno degli stati usando gli **hook** (ganci, cioè gli stati) appropriati

Diagramma del ciclo di vita di un componente Vue:



Gli **hook** sono chiamati quando il componente:

- **mounted()** (il più comune): è stato **renderizzato nel DOM**
È il posto ideale per operazioni che richiedono l'accesso al DOM o per il **fetching dei dati** iniziali da un'API
- **beforeCreate()**: **prima che viene creato** (i dati reattivi non sono ancora pronti)
- **created()**: **dopo la creazione** (dati pronti ma il DOM non è ancora renderizzato). Ottimo per l'**inizializzazione dei dati**
- **beforeUnmount()**: **prima che venga distrutto** e rimosso dal DOM. Ottimo per **pulire listener o timer**
- **unmounted()**: **dopo che è stato rimosso**

Es con mounted()

```

export default {
  data() {
    return {
      loading: true, // loading lo usiamo per indicare quando stiamo caricando i dati
      postTitle: ""
    }
  },
  // "mounted" è eseguito quando il componente è inserito nel DOM
  mounted() {
    console.log("Componente montato!");

    // esempio: carica i dati di un post dall'API
    this.loading = true; // imposta loading a true (caricamento iniziato)
    fetch("https://jsonplaceholder.typicode.com/posts/1")
      .then(response => response.json()) // prende i dati dall'API
      .then(data => {
        this.postTitle = data.title; // carica il titolo del post nella variabile
        this.loading = false; // imposta loading a false (caricamento finito)
      });
  }
}

```

Quando Usare Cosa?

Hook	Momento	Uso Comune
created	Dati pronti, DOM no	Inizializzazione dei dati
mounted	Dati e DOM pronti	Fetching dati (con API), interazioni DOM
beforeUnmount	Prima della distruzione	Pulizia di listener esterni

Router

lunedì 12 gennaio 2026 12:03

Le applicazioni a **pagina singola (SPA)** sono composte da più viste (o pagine) e ogni vista implementa un componente. Le viste hanno un URL univoco e vengono renderizzate sul client, sfruttando JS e il browser stesso. Mentre il server fornisce il codice per la SPA e le API e la SPA invierà e riceverà dati usando l'API tramite JS

Il **router** è un fondamentale per **scambiare i vari componenti** e **gestire le rotte** per le viste nell'app. È composto dalle seguenti parti:

- **Routes (rotte):** Associazione tra **percorsi (paths)** e **componenti**
- **<RouterView></RouterView>**: La vista del router all'interno del componente principale
 - Verrà sostituita col componente indicato dall'URL quando questo cambia
- **<RouterLink to="/link1">Link</RouterLink>**: Crea un link
 - Quando viene cliccato, la vista del router viene sostituita dal componente registrato per **/link1**

Le viste del router sono **impilate nella cronologia** (history) e quando usiamo **<RouterLink>** si va a "pushare" la vista nella cronologia. Si può controllare lo stack con queste azioni:

- **Push: Nuova vista aggiunta** in cima allo stack, e si passa ad essa
- **Go: Spostarsi avanti/indietro** nella cronologia
- **Replace: Sostituire la vista corrente** nello stack della cronologia

Per inizializzare il router bisogna configurare il tipo di navigazione, la cronologia e la lista di **routes**.

Attenzione: bisogna inizializzare il router **prima** di creare e montare l'applicazione

Es: crea un router che si può gestire **modificando gli URL**. Ogni URL è associato a una vista dell'app. Es il link per /link1 sarà tipo <https://example.com/#/link1>

Configurazione Router

```
import {createRouter, createWebHashHistory} from 'vue-router'
import HomeView from '../views/HomeView.vue'
import Page1View from '../views/Page1View.vue'
import Page2View from '../views/Page2View.vue'

const router = createRouter({ // inizializza il router
  history: createWebHashHistory(), // crea la cronologia
  routes: [ // imposta le rotte per i vari path ai vari componenti
    {path: '/', component: HomeView},
    {path: '/page1', component: Page1View},
    {path: '/some/:id/link', component: Page2View},
  ]
});

export default router;
```

Utilizzo del Router in main.js

```
import { createApp } from 'vue'
import App from './App.vue'
// importa la configurazione del router
import router from './router'

const app = createApp(App)
// ... eventuali altri componenti
app.use(router) // usa il router nell'app Vue
app.mount('#app') // monta l'app Vue nell'elemento con id "app"
```

Componente Principale di Vue

```
<script setup>
  import { RouterView, RouterLink } from 'vue-router'
</script>
<script>
  export default {
    // logica del componente principale
  }
</script>
<template>
  <header>Intestazione dell'App</header>
  <main>
    <p>
      <RouterLink to="/">Home</RouterLink>
      <RouterLink to="/link1">Pagina 1</RouterLink>
      <RouterLink to="/some/1/link">Pagina interna con parametro</RouterLink>
      <RouterLink to="/variableWithLinkDest">Link dinamico</RouterLink>
    </p>
    <RouterView></RouterView> <!-- Qui viene renderizzato il componente della rotta corrente -->
  </main>
</template>
```

Cambiare Pagina e Ottenere Parametri

Si può **navigare tra le pagine** in modo programmatico usando l'**oggetto \$router**, da cui si può modificare la cronologia delle rotte. Es: per navigare verso una nuova rotta eseguiamo il push della rotta

Data la rotta **/some/:id/link**, il suo componente può accedere al parametro usando **\$route.params**.

Attenzione: il passaggio allo stesso link con valori diversi (es /some/1/link → some/2/link) **non ricarica il componente**.

Bisogna quindi osservare l'evento con un **watcher**

```

<script>
  export default {
    created() {
      this.$watch(
        () => this.$route.params.id,
        (toParams, previousParams) => {
          console.log('Parametro id cambiato da', previousParams, 'a', toParams);
          // Esegui altre azioni in risposta al cambiamento del parametro
        }
      )
    },
    methods: {
      doSomething() {
        // ...
        // imposta la variabile id
        let id = this.$route.params.id;
        // ...

        // naviga verso la rotta con il parametro
        this.$router.push('/some/' + id + '/link');
      }
    }
  }
</script>

<template>
  <!-- Visualizza il valore del parametro "id" direttamente nel template-->
  <p>Valore del parametro id: {{ $route.params.id }}</p>

  <button @click="doSomething">Fai qualcosa e cambia la pagina</button>
  <button @click="$router.push('/')">Torna all'indice</button>
</template>

```

JS Asincrono

mercoledì 14 gennaio 2026 18:18

Promises (Promesse)

Una **Promise** è un oggetto che si riferisce a un **valore futuro** (ovvero una "proxy"). Se una funzione restituisce una Promise, significa che il valore di ritorno sarà **disponibile in futuro** (esecuzione asincrona).

Per usare una Promise è necessario specificare:

- 1) Una funz. che viene eseguita quando il risultato è **disponibile**
- 2) Una funz. che viene eseguita quando **qualcosa va storto**

Una Promise è in uno di questi stati:

- **Pending (sospeso)**: prima di iniziare, o il codice asincrono è in esecuzione
- **Fulfilled (adempita)**: operazione completata con successo
- **Rejected (rifiutata)**: operazione fallita

```
var txt;
doSomething() // supponiamo che restituisce una Promise
  .then((result) => { // quando la Promise è risolta
    txt = result;
  })
  .catch((error) => { // quando la Promise è rifiutata
    // gestione dell'errore
  });

// il seguente alert può essere eseguito prima che txt sia stato assegnato
alert(txt)
```

Pattern Async-Await

Il pattern **async-await** è un modo per scrivere codice **asincrono** usando lo stile "sincrono".

Le funzioni che devono usare questo stile devono essere dichiarate come **async**

Nota: questo è **syntactic sugar** (zucchero sintattico). Il codice è comunque asincrono e devi esserne consapevole

Es:

Codice con Promise

```
function buttonClicked() {
  doSomething()
    .then((result) => { // Promise risolta
      alert(result);
    })
    .catch((error) => { // Promise rifiutata
      // gestione dell'errore
    });
}
```

Codice async-await

```
async function buttonClicked() {
  try {
    // aspetta che la Promise sia risolta
    let txt = await doSomething();
    alert(txt);
  } catch (error) { // Promise rifiutata
    // gestione dell'errore
  }
}
```

Axios

mercoledì 14 gennaio 2026 18:41

AJAX (Asynchronous JavaScript and XML) è un insieme di tecnologie per aggiornamenti incrementali nelle pagine web. Contiene:

- HTML/XHTML/CSS
- JavaScript
- DOM
- XML/XSLT
- XMLHttpRequests (XHR): un oggetto supportato da ogni browser usato per eseguire richieste web e ottenere il risultato

Per ottenere i dati da un server remoto esiste anche **Fetch API** che è molto usato dalle nuove applicazioni JS

Pero non useremo ne XHR direttamente (troppo verboso e interfaccia non aggiornata) ne Fetch API (ancora work-in-progress, mancano alcune funzioni)

Invece andremo ad usare **Axios**, una libreria JS che semplifica AJAX e permette di eseguire richieste HTTP ai server.

Axios è un client HTTP per browser e node.js, basato sulle Promise

- **Richieste da Browser:** Esegue richieste XMLHttpRequests direttamente dal browser
- **Basato sulle Promise:** Supporta completamente l'API Promise
- **Interceptor:** Intercetta le richieste e le risposte per aggiungere logiche personalizzate
- **Trasformazione dei Dati:** Trasforma automaticamente i dati di richiesta e risposta
- **Annullamento delle Richieste:** Annulla le richieste usando i meccanismi integrati
- **Gestione Automatica del JSON:** Serializza e analizza automaticamente i dati JSON
- **Serializzazione dei Form:** Serializza automaticamente gli oggetti di dati nei formati multipart/form-data o x-www-form-urlencoded
- **Protezione XSRF:** Supporto lato client per la protezione Cross-Site Request Forgery

Esempio

```
import {createApp} from 'vue'
import App from './App.vue'
import axios from 'axios'

const instance = axios.create({
  // URL di base per tutte le richieste
  baseURL: "http://localhost:3000",
  timeout: 1000
});

let app = createApp(App); // inizializza l'app

// aggiunge axios come proprietà globale,
// accessibile in tutti i componenti con this.$axios
app.config.globalProperties.$axios = axios;

app.mount("#app"); // monta l'app sul DOM
```

Esempio GET

```
async refresh() {
  try {
    let response = await this.$axios.get("/");
    // response.data contiene il JSON in response body
  } catch (error) {
    console.error(error.toString());
  }
}
```

Esempio POST

```
async sendData() {
  try {
    let response = await this.$axios.post("/some/api", {
      firstName: "John",
      lastName: "Doe"
    });
    // response.data contiene il JSON nel response body
  } catch (error) {
    console.error(error.toString());
  }
}
```

Dati della Risposta

Un oggetto di risposta tipico da Axios contiene i seguenti campi:

```
{
  "data": {},
  "status": 200,
  "statusText": "OK",
  "headers": {},
  "config": {},
  "request": {}
}
```

Interceptor

Gli **Interceptor** permettono di **intercettare e modificare** le richieste/risposte prima che vengano gestite dal codice **then o catch**

Interceptor di Richiesta

```
axios.interceptors.request.use(function (config) {  
  // Esegui qui modifiche alla configurazione della richiesta  
  // (es: aggiungere header di autenticazione)  
  return config;  
}, function (error) {  
  // Gestisci l'errore della richiesta (prima che venga inviata)  
  return Promise.reject(error);  
});
```

Interceptor di Risposta

```
axios.interceptors.response.use(function (config) {  
  // Esegui qui modifiche alla configurazione della richiesta  
  // (es: aggiungere header di autenticazione)  
  return config;  
}, function (error) {  
  // Gestisci l'errore della richiesta (prima che venga inviata)  
  return Promise.reject(error);  
});
```

Cancellazione

La **cancellazione della richiesta** può essere utile in alcuni casi di **race condition** dove alcune richieste partite prima arrivano in ritardo rispetto ad altre richieste partite dopo

Es:

- 1) Alice fa il GET di /conversation/alice/messages ma ha la connessione lenta e la richiesta arriva dopo 3 secondi
- 2) Poco dopo Bob fa sempre il GET di /conversations/bob/messages e ha la connessione più veloce e la richiesta arriva dopo 0.5 secondi
- 3) La chat di Bob viene mostrata a schermo
- 4) Dopo 3 secondi arriva la risposta di Alice e i dati della response di Bob vengono sovrascritti da quelli di Alice

Con la cancellazione della richiesta possiamo **annullare** la richiesta precedente, così se la richiesta di Alice arriva tardi, viene ignorata

```
// 1) Crea un controller  
const controller = new AbortController();  
  
// 2) Utilizzo del controller per annullare una richiesta  
async function fetchWithCancellation() {  
  try {  
    const response = await this.$axios.get("/user/12345", {  
      signal: controller.signal // passa il signal alla richiesta  
    });  
  } catch (error) {  
    if (axios.isCancel(error)) { // se l'errore è dovuto ad annullamento  
      console.log("Richiesta annullata", error.message);  
    } else {  
      // gestisci errore  
    }  
  }  
}  
  
// 3) Annulla la richiesta  
controller.abort(); // annulla la richiesta in corso
```

Esempio Nasoni

mercoledì 14 gennaio 2026 23:46

Creiamo una applicazione Vue.js che recupera la lista di fontane da un server API.

Lanciando il comando

```
npm create vue@latest
```

Creiamo un nuovo progetto Vue vuoto nella directory (chiederà nome e altre info sul progetto). I file più importanti del progetto sono:

- **index.html**: è la **pagina principale** dell'applicazione SPA. Ci sta `<div id="app"></div>` che è il punto di montaggio dell'intera applicazione
- **packages.json**: è il file di configurazione usato nei progetti che usano Node.js. È usato per definire vari aspetti del progetto (metadata, dipendenze, ecc)
- **vite.config.js**: è la configurazione di **vite**, il build tool di Vue
- **public/**: è la directory che contiene **file statici**: Vue copia questi file in **dist/** senza modificarli
- **src/**: è la directory che contiene il codice dell'applicazione
 - **src/App.vue**: è l'SFC che è usato come componente root dell'applicazione Vue
 - **src/main.js**: è il core JS dell'applicazione Vue. L'app viene creata in questo file e se si usa il riuter, Axios e altre librerie convien e iniziarle qua
 - **src/assets/**: contiene gli **"assets"** (file che subiscono cambiamenti minimi, es minimizzazione), usati comunemente per icone e CSS
 - **src/router/**: contiene l'inizializzazione e la configurazione del **router** (cronologia, rotte, ecc)
 - **src/components/** e **/src/views/**: contiene tutte le **SFC** (Single File Components) (eccetto App.vue)

(la struttura di src è una formalità e tutto si può modificare, a patto che i file rilevanti sono modificati di conseguenza)

Ora possiamo installare **axios** (con **npm install axios** se già non è installato) e lo impostiamo in Vue come variabile globale dentro **src/main.js** dopo **createApp()** e **prima di app.mount()**

```
import {createApp} from 'vue'
import App from './App.vue'
import router from './router'
import axios from './services/axios.js';

const app = createApp(App) // crea l'app Vue
// aggiunge axios e imposta l'URL e il timeout
app.config.globalProperties.$axios = axios.create({
  baseURL: "http://localhost:8090",
  timeout: 1000 * 5
});
app.use(router) // aggiunge il router
app.mount('#app') // monta l'app vue sul DOM
```

Ora possiamo usare Axios, quindi andiamo a in **src/views/HomeView.vue** per inserire il componente della stampa delle fontane


```

<script>
export default {
  data: function() {
    return {
      // dichiara un array reactive di fontanelle
      fountains: [],
    }
  },
  methods: {
    // dichiara una funzione che manda una richiesta HTTP GET. Axios
    async loadFountainsList() {
      try {
        let response = await this.$axios.get("/fountains/", {
          params: {
            // questi valori dovrebbero venire dal browser
            // posizione/GPS o qualche input dell'utente
            latitude: 1,
            longitude: 2,
          }
        });
        // Axios decodifica JSON automaticamente
        this.fountains = response.data;
      } catch (e) {
        alert("Error: " + e);
      }
    }
  }
}
</script>

<template>
  <!-- quando questo bottone è cliccato, la lista viene scaricata -->
  <button @click="loadFountainsList">
    Load fountains
  </button>
  <!-- separatore -->
  <hr />
  <p>List of fountains:</p>
  <ul>
    <!-- cicla il tag <li> per ogni fontanella e stampa la posizione -->
    <li v-for="f in fountains">
      Lat: {{ f.latitude }} -
      Lng: {{ f.longitude }}
    </li>
  </ul>
</template>

```

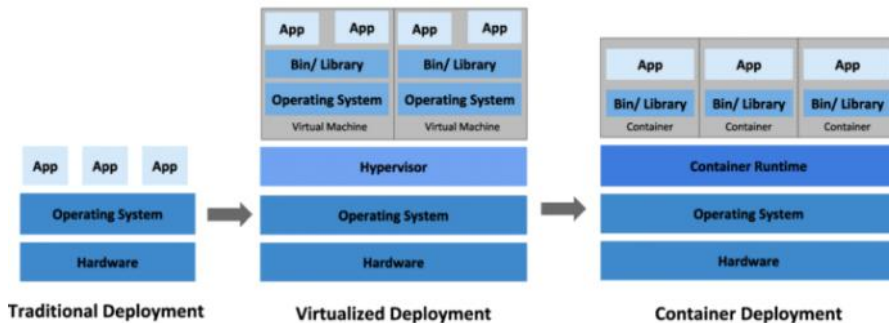

Container

venerdì 13 febbraio 2026 13:27

Per fare in modo che altri utenti possano usare un'applicazione, bisogna eseguire la **distribuzione (deploy)** di essa su un server. Il deploy include la copiatura degli eseguibili dell'app, i file usati, le dipendenze, le directory di produzione, l'interprete dell'app e altri file esterni necessari (es immagini).

Ci sono 3 modi principali per eseguire il deploy:

- **Metodo tradizionale:** i processi dell'app sono a conoscenza e possono interagire con i processi di altre app nell'OS. Il kernel, filesystem, librerie e hardware sono condivisi tra tutti i processi. Questo metodo però ha **molte limitazioni**, ad esempio condividere la stessa libreria per più app è scomodo se si vogliono usare librerie specifiche
- **Distribuzione su Virtual Machine (VM):** l'**host** è l'OS che gira sull'hardware vero e proprio e il **guest** è l'istanza che gira nella VM. L'**hypervisor** (software specializzato che gestisce l'esecuzione e il ciclo di vita della VM) si occupa di eseguire e tradurre le operazioni richieste dal guest così che esso creda che stia girando su una macchina reale e non virtuale. Questo permette **maggiore sicurezza** grazie all'**isolazione**: i processi nella VM sono **separati virtualmente**, quindi un crash di un guest non influisce gli altri guest
- **Distribuzione tramite Container:** I processi containerizzati girano sullo **stesso kernel dell'Host** ma ogni **chiamata di sistema (syscall)** è eseguita in un **contesto specifico** (una lista di processi in un contesto specifico può essere diverso da una lista di processi in un altro contesto). I processi sono **isolati** e si può **configurare** la consumazione di risorse.



Un'immagine container è un **archivio tarball di un filesystem** che costruisce un **ambiente isolato e portatile**

Costruire un'Immagine (docker build)

- 1) Installa il programma + dipendenze
- 2) Configuralo come desideri
- 3) Crea una tarball dell'intero filesystem

Eseguire un'Immagine (docker run)

- 1) Scarica il tarball
- 2) Decomprimi in una directory
- 3) Esegui un programma e fai finta che quella directory sia il suo intero filesystem

Container Docker

I **container Docker** usano queste funzionalità del kernel Linux:

- **pivot_root:** imposta la **dir root** di un processo su una dir con il contenuto dell'immagine container
- **cgroups:** limita l'uso di **CPU/RAM** per un gruppo di processi
- **Namespaces:** fa credere ai processi di essere eseguiti su un filesystem vero dandogli i propri PID, rete, hostname, mount e utenti
- **Capabilities (Sicurezza):** fornisce **permessi specifici**
- **seccomp-BPF (Sicurezza):** previene syscall pericolose
- **overlay filesystem:** la condivisione dei layer fa risparmiare spazio su disco

chroot Vs pivot_root

Per cambiare la directory root di un processo si usa:

chroot

Facendo **chroot** in **fake/root**, quando si apre **/usr/bin/redis** si otterrà **fake/root/usr/bin/redis**

Problema: i programmi possono uscire da un chroot e un processo root può accedere agli altri file

pivot_root

I container usano **pivot_root** invece di chroot perché si può **smontare** il vecchio filesystem in modo che sia impossibile accedervi

cgroups (Control Groups)

Questa funzionalità permette di **limitare le risorse** che un processo può usare (es un gruppo di processi può usare **al massimo** 500MB di RAM e 10% di CPU)

- 1) **Crea un cgroups:** con il comando `cgcreate -g "cpu, memory:nome-container"`
- 2) **Imposta i limiti:** es `cgset -r memory.limit_in_bytes=524288000 nome-container`
- 3) **Esegui il processo:** usa `cgexec -g "cpu,memory:nome-container"`

Namespaces

I **Namespaces** permettono di **isolare** i processi tra di loro, facendogli credere di essere **l'unico processo/rete/utente** sul sistema

Namespace	Isolamento	Cosa rende unico
PID	Processi	La sua tabella di processi
Net	Rete	Interfacce di rete, indirizzo IP, porte
User	Utente	ID utente e ID gruppo
Mount	Filesystem	Punti di mount
UTS	Nome host	Nome host
IPC	Comunicazione interprocesso	Code di messaggi, semafori

PID Namespaces

Creano l'illusione per i processi facendogli credere di essere da soli nel sistema:

- **Nell'host:** l'OS vede il vero PID (es 1234)

- **Nel container:** il processo principale (es server Redis) è sempre **PID 1** (semplifica la scrittura degli script)

In questo modo un container non può vedere/terminare altri processi che girano sull'host

Network Namespaces

Forniscono ad un processo la propria **configurazione di rete** (proprie interfacce, IP, tabelle di routing, regole firewall, ecc)

Ogni componente ha:

- **Interfaccia lo** (localhost): per il loopback
- **Interfaccia virtuale veth:** un'estremità sta nel container e l'altra è nell'host (spesso collegata al bridge), permettendo la comunicazione di rete tra i due

Vantaggio: si possono avere 50 container che ascoltano tutti sulla porta 80 senza che vadano in conflitto, poiché ognuno ha un suo IP interno

Rete in Docker

Docker usa i Network Namespaces per collegare i container a internet:

- 1) **Router virtuale (Bridge):** Docker crea un **bridge di rete** sull'host chiamato **docker0**
- 2) **Collegamento (veth pair):** crea una coppia di interfacce virtuali (veth) una nel container e l'altra collegata al bridge docker0
- 3) **Indirizzo IP:** il bridge assegna un IP privato al container (es 172.17.0.2)
- 4) **Connessione a Internet (NAT):** docker0 usa il **NAT** (Network Address Translation) per **instradare il traffico** dei container verso l'interfaccia di rete reale (es eth0)

Eseguito il comando **docker run -p 8080:80**, Docker fa due cose:

- 1) Avvia il container sulla **porta virtuale 80**
- 2) Aggiunge una regola di firewall all'Host che **reindirizza** il traffico in arrivo sulla **porta 8080** dell'Host alla **porta 80** del container

Il comando **unshare** esegue un programma in un nuovo namespace. Es:

```
# Esegue 'sh' nel proprio namespace per il nome host (UTS)
$ sudo unshare -u sh
# Ora esegui 'hostname'
$ hostname
# Output: [Il nome host del sistema host]
# Cambia il nome host:
$ hostname my-secret-container
# Ora esegui 'hostname' di nuovo
$ hostname
# Output: my-secret-container
# Se apri un'altra shell o sul sistema host, il nome host non è cambiato.
# L'effetto è locale a questa shell (perché è nel suo namespace UTS).
```

Capabilities

Le **Capabilities** suddividono i permessi di **root** in centinaia di funzioni più piccole (es CAP_SETUID per cambiare ID utente)

I contenitori di Docker (di default):

- **Rimuovono** molte capabilities di root (es viene rimosso CAP_SYS_ADMIN che permette di eseguire operazioni di amministrazione di sistema)
- **Mantengono** un set minimo necessario per l'esecuzione (es CAP_NET_BIND_SERVICE che permette di eseguire il bind a porte sotto 1024)

Vantaggio: se un processo **nel contenitore** viene **eseguito come root**, **non ha i permessi** di un vero root **nell'Host**

seccomp-BPF (Secure Computing con Berkeley Packet Filter)

seccomp-BPF permette di **filtrare le syscall**, creando una **whitelist** o una **blacklist** di syscall che un processo può o non può eseguire
I contenitori di Docker (di default):

- **Applicano** un profilo seccomp predefinito che **disabilita circa 44 syscall "pericolose"**

Es: viene disabilitato **mount()** per impedire al contenitore di montare/smontare filesystem dell'Host

Vantaggio: anche se un attaccante ha i permessi per eseguire un'azione, seccomp può bloccare la syscall sottostante

Layer

Immagini diverse hanno spesso file simili e il **riutilizzo dei layer** fa risparmiare spazio su disco

Es: Se due immagini usano entrambe Ubuntu 18.04, invece di creare due layer Ubuntu 18.04, si usa un unico layer per entrambe le immagini

- **ID del layer:** ogni layer ha un ID
- **Struttura:** un layer è una sola directory contenente file (es /bin, /etc)
- **Unione:** se un file si trova in 2 layer, vedrai la versione del **layer superiore**

Se si hanno due o più directory, si usano i **filesystem Overlay** (OverlayFS) per farle sembrare una sola:

- **Layer superiore (Write Layer):** di predefinito, le modifiche vengono scritte su un layer temporaneo e eliminati dopo l'uscita dal container. Per mantenere le modifiche bisogna scrivere su una dir esterna al container
- **Layer inferiore (Base Layer):** tutto in **sola lettura** (read-only)

Es: avvio di un container che esegue la shell fish

```

wget bit.ly/fish-container -O fish.tar           # 1. Scarica l'immagine
mkdir container-root; cd container-root         # 2. Prepara la directory
tar -xf ../fish.tar                             # 2. Decomprimi l'immagine

cgroup_id="cgroup_$(shuf -i 1000-2000 -n 1)"    # 3. Genera un nome per il cgroup
cgcreate -g "cpu,cpuacct,memory:$cgroup_id"    # 4. Crea il cgroup
cgset -r cpu.shares=512 "$cgroup_id"           # 4. Imposta i limiti di CPU
cgset -r memory.limit_in_bytes=1000000000 "$cgroup_id" # 4. Imposta i limiti di
Memoria

cgexec -g "cpu,cpuacct,memory:$cgroup_id" \    # 5. Usa il cgroup
unshare -fmuipn --mount-proc \                 # 6. Crea e usa i namespaces
chroot "$PWD" \                                # 7. Cambia la directory root
/bin/sh -c "                                     # 10. Avvia la shell
    /bin/mount -t proc proc /proc &&           # 8. Usa il /proc corretto
    hostname container-fun-times &&           # 9. Cambia l'hostname
    /usr/bin/fish                             # 10. Avvia fish!
"

```

Docker: Opzioni di Configurazione

Docker e altri runtime di contenitori sono un'interfaccia amichevole per tutte queste funzionalità di Linux. Esempi di comandi Docker:

Comando	Funzionalità del Kernel Abilitata
docker run (senza flag)	pivot_root, cgroups, tutti i Namespaces, Capabilities, seccomp
docker run -p 8080:80	Network Namespaces (e iptables per mappatura)
docker run --rm	Pulizia di OverlayFS e cgroups dopo l'uscita
docker run --cap-add=ALL	Capabilities (aggiunge tutte le capabilities, riducendo la sicurezza)
docker run --network=host	Network Namespaces (riusa il namespace di rete dell'Host, disabilitando l'isolamento di rete)

Gestione Container

venerdì 13 febbraio 2026 16:16

Dockerfile

Possiamo creare un file chiamato **Dockerfile** che indica come bisogna eseguire la **build** dell'immagine del container

Ad esempio vogliamo creare un container che aggiunge il comando **htop** all'immagine di base di Debian e che faccia partire htop di default.

Possiamo creare il seguente Dockerfile

```
# immagine base
FROM debian:stable

# aggiorna i pacchetti di Debian
RUN apt-get update -yq

# installa htop
RUN apt-get install -yq htop

# esegui htop di default
CMD ["/usr/bin/htop"]
```

Ora dobbiamo solo eseguire:

- 1) **Build dell'immagine:** creiamo un pacchetto con tutto il necessario (OS, Python, App)
docker build -t test-debian .
 - Con **-t** definiamo il nome e tag dell'immagine
 - Il **punto alla fine** si riferisce alla directory usata per costruire il contesto del container.
- 2) **Esecuzione**
docker run -it --rm -v path/to/dir test-debian
 - Con **-it** possiamo rendere **interattivo** il container, quindi possiamo **interagire con i processi interni** del processo e con il suo filesystem
Usando il comando
docker exec -it test-debian /bin/bash
Possiamo entrare nel container mentre è in esecuzione tramite shell
 - Con **--rm** diciamo che il container deve essere **eliminato quando viene chiuso**
 - Con **-v path/to/dir** diciamo invece di mappare una **directory esterna all'interno del container** così da salvarci file così che non vengano eliminati all'eliminazione del container (ogni file del container che non è in una cartella montata con -v verrà eliminato all'eliminazione del container)

All'avvio vedremo che htop è **eseguito direttamente** all'avvio del container

Docker run scaricherà l'immagine **di debian:stable** da **Docker Hub** (registro di varie immagini di container) se non è presente nella cache locale del sistema, poi eseguirà i vari comandi presenti nel Dockerfile.

Immagine del Backend

Creiamo un'immagine per eseguire il **backend di Go**.

Usiamo la versione 1.19.1 del compiler di Go e creiamo un nuovo file chiamato **Dockerfile.backend** nella root del progetto di Go (la stessa directory con go.mod):

```
# usiamo l'immagine base di Go
FROM golang:1.19.1

# ci spostiamo nella cartella di lavoro
WORKDIR /src/

# copiamo il codice di Go nell'immagine
COPY . .

# costruiamo l'applicazione
RUN go build -o ./webapi ./cmd/webapi

# esponiamo la porta su cui gira l'applicazione
EXPOSE 3000

# condividiamo la cartella per il database
VOLUME /database

# impostiamo il programma di default al backend di Go
CMD ["/src/webapi"]
```

Costruiamo l'immagine:

- docker build -t backend:latest .
- docker run -it --rm -p 3000:3000 backend:latest
 - Con **-p 3000:3000** diciamo di **forwardare** (instradare i pacchetti) dalla porta 3000 dell'host (1° porta) alla porta 3000 del container (2° porta)

Adesso il server Go è raggiungibile su <http://localhost:3000>

Multi-Stage Build

L'immagine backend:latest creata funziona ma contiene l'eseguibile, il codice sorgente e l'intero compiler di Go.

Per fare in modo che questi file vengano rimossi dopo il comando build possiamo usare una tecnica chiamata **multi-stage build**:

- Creiamo una o più **immagini temporanee** e poi costruiamo quella finale con gli "artefatti" (eseguibili, file, ecc)
- Ogni **fase inizia con FROM** e termina al **prossimo FROM** (o alla fine del file)
- Tutte le immagini temporanee **devono avere un nome**

Nel build del Dockerfile, Docker salverà solo **l'immagine finale** (i comandi che partono dall'ultimo FROM fino alla fine del file), mentre le altre immagini temporanee vengono rimosse.

Modifichiamo il file del backend

```
# creiamo la prima immagine temporanea "builder"
FROM golang:1.19.1 AS builder

# copiamo il codice Go (in "builder")
WORKDIR /src/
COPY . .

# costruiamo l'applicazione (in "builder")
RUN go build -o ./webapi ./cmd/webapi

# creiamo il container finale
FROM debian:bookworm

# esponiamo la porta su cui gira l'applicazione
EXPOSE 3000

# copiamo l'eseguibile dall'immagine di "builder"
WORKDIR /app/
COPY --from=builder /src/webapi .

# impostiamo il programma di default al backend di Go
CMD ["/app/webapi"]
```

Quindi in questo caso l'immagine finale conterrà solo l'immagine di base di **debian:bookworm** e l'eseguibile webapi

Immagine del Frontend

Creiamo un altro file chiamato **Dockerfile.frontend** nella **root del progetto Vue.js**

```
# usa l'ultima versione di Node LTS
FROM node:lts as builder

# copia il codice di Vue.js
WORKDIR /app
COPY . .

# build Vue in semplice HTML/CSS/JS
RUN npm run build-prod

# crea il container finale con nginx
FROM nginx:stable

# copia l'app dall'immagine di "builder"
COPY --from=builder /app/dist /usr/share/nginx/html

# espone la porta 80
EXPOSE 80
```

Non c'è bisogno di CMD alla fine del file perché l'immagine di nginx definisce già il comando da eseguire

Costruiamo l'immagine (nginx ascolta alla porta 80 ma noi usiamo la porta 8080 sull'Host):

- `docker build -t frontend:latest .`
- `docker run -it --rm -p 8080:80 frontend:latest`

Ora possiamo accedere al frontend su <http://localhost:8080>

Docker Compose

Gestire due container manualmente (avviarli e collegarli in rete) è difficile.

Possiamo usare **Docker Compose**, un programma che permette di definire tutti i servizi, networks e volumi richiesti per le immagini create.

Creiamo un file **docker-compose.yml** nella directory del progetto che definisce la **configurazione del container**

```

version: "3" # versione del file docker-compose

▷ Run All Services
services:
  ▷ Run Service
  backend: # configurazione del backend
    build:
      context: .
      dockerfile: Dockerfile.backend # specifica il Dockerfile dell'immagine del backend
    ports:
      - "3000:3000" # mappa la porta 3000 del container a quella del host
    volumes:
      - "./shared:/shared" # mappa la cartella shared del host a quella del container

  ▷ Run Service
  frontend: # configurazione del frontend
    build:
      context: ./webui
      dockerfile: Dockerfile.frontend # specifica il Dockerfile dell'immagine del frontend
    ports:
      - "8080:80" # mappa la porta 8080 del container alla porta 80 del host

```

Per eseguire i container definiti nel file docker-compose.yml bisogna eseguire questi comandi nella stessa cartella del file:

```
docker compose up -d
```

- Con **-d** diciamo di far partire il file in **background**

Docker Compose scaricherà le immagini specificate (se non sono già salvate in locale) e eseguirà i container in background

Per fermare i container e rimuoverli eseguiamo

```
docker compose down
```

Fontanelle

giovedì 16 ottobre 2025 14:11

Define the API for the Fountains project (fountains are called Fontanelle or Nasoni in Rome). The Fountains project includes a mobile app that allows citizens and tourists to explore nearby drinking fountains.

In Rome, drinking water is available thanks to little public fountains, also known as “nasoni” (“large noses”, due to their nose-like shape). They were installed in the late 1800 by the local municipalities in the capital city and nearby areas. More than 2500 are still working. The “Fountains” project includes a mobile app that allows citizens and tourists to explore nearby drinking fountains (see their location and status). Also, they will be able to change the status or location of any fountains, add missing fountains, and remove those that are no longer present. A fountain’s status is “good” when it is in working condition and “faulty” if broken. The app will communicate with a central server via REST and JSON - no information is stored locally in the smartphone. No authentication is needed, nor is user identification.

WASA BOOK section 4.3.

API Moderazione

giovedì 27 novembre 2025 12:48

Progettare le API RESTful per un sistema di moderazione di contenuti (es. commenti, post) in una piattaforma social

Il sistema gestisce:

- 1) **Segnalazioni** (Reports): Un utente segnala un contenuto
- 2) **Contenuti segnalati** (Reported Items): Elementi che attendono revisione
- 3) **Decisioni di moderazione** (Decisions): L'azione finale intrapresa (approvare, rimuovere, bannare)

Fase 1: Risorse e Path

Obiettivo: definire le risorse e i path principali, aderendo al naming REST

<i>Descrizione Interazione</i>	<i>Risorsa Principale</i>	<i>Risorsa Secondaria (necessaria)</i>	<i>Path (esempio)</i>
L'utente crea una segnalazione	reports		/reports
Il moderatore vede tutte le segnalazioni	reports		/reports?status=pending
Il moderatore cerca una segnalazione specifica	reports		/reports/{report_id}
Il moderatore agisce su un contenuto (decisione)	reported/items	decisions	/reported-items/{item_id}/decisions

Fase 2: Mapping dei Metodi

<i>Descrizione Operazione</i>	<i>Path (esempio)</i>	<i>Metodo HTTP</i>
Inviare una nuova segnalazione utente	/reports	POST
Leggere i dettagli di una segnalazione	/reports/{report_id}	GET
Ottenere la lista di tutti i contenuti in attesa	/reported-items	GET
Aggiornare lo stato di una segnalazione a "in revisione"	/reports/{report_id}	PATCH
Prendere una decisione (es. rimuovi contenuto)	/reported-items/{item_id}/decisions	POST
Annullare/Rivedere	/reported-items/{item_id}/decisions/{decision_id}	DELETE

Fase 3: Output e Parametri

Domanda di design cruciale:

- 1) **Filtraggio:** come permettiamo ai moderatori di filtrare la lista di /reported-items per categoria (spam, violenza, haters) e priorità?
 - **Suggerimento:** usare i parametri query (es ?category=spam&priority=high)
- 2) **Payload della decisione:** quali dati minimi devono essere inviati al POST su /reported-items/{item_id}/decisions?
 - **Suggerimento:** moderator_id, action (REMOVE_POST, WARN_USER, APPROVE), reason_code
- 3) **Gestione dello stato:** quale codice HTTP dovrebbe restituire il POST che registra la decisione di moderazione?
 - **Suggerimento:** **201 Created** se viene creato un nuovo record di decisione